

Python for Introductory Statistics

A practical guide to statistical analysis using Python and Google Colaboratory

Purpose and Overview

The purpose of this material is to provide an overview of the free programming language, Python, and a few of its software *libraries* that are most useful for improving Introductory Statistics students' learning and for providing an alternative statistical software package for those who perform statistical data analysis and plotting. This material, along with the companion workbook and a free web-based Python platform, could easily replace the use of other software packages and calculators in an Introductory Statistics course. Part I, Sections 1 to 4, covers the basics of the Python programming language. Part II, Sections 5 to 13, focuses on Python for introductory statistics, and Part III, Section 14, covers selected advanced statistics topics in Python. Additionally, there are demo video links accompanying key steps for further instruction. No prior programming knowledge is required or assumed.

Questions, comments or errors can be reported by sending an email to saman2@ccc.edu. A link to an interactive Colab version of this document can be found in this [interactive Colab workbook](#).

Keyboard Shortcut for HTML (Web Link) and PDF Users: When you click an external link and want to go back to where you were before the click, use the keyboard shortcut `Alt` and `Left-Arrow` for Windows and `Command` and `Left-Arrow` for Mac.

© 2021 Simon Aman, City Colleges of Chicago. Special thanks to reviewers, Professor Phil Yates of DePaul University and Professor Aman Ghebremicael of Arizona Western College.

This material is licensed under [Creative Commons Attribution 4.0 International \(CC BY 4.0\)](#).

Table of Contents

- 0. [Why Learn Python?](#)
- 1. [Introduction to Python](#)
 - [1.1 Getting Started](#)
 - [1.2 Python Arithmetic Operators](#)
 - [1.3 Python Variables](#)
 - [1.4 Python Comments](#)
 - [1.5 The NumPy Library](#)
 - [1.6 `input` Data From Keyboard](#)
- 2. [Python Lists](#)
 - [2.1 Creating and Accessing Lists](#)
 - [2.2 List Operations](#)
- 3. [Python Syntax](#)
 - [3.1 `if` Statements](#)
 - [3.2 Functions](#)
 - [3.3 Syntax Errors](#)
- 4. [Python Loops](#)
 - [4.1 The `for` Loop](#)
 - [4.2 The `while` Loop](#)
- 5. [Frequency Distributions and Graphs](#)
 - [5.1 Frequency Counts](#)
 - [5.2 The Matplot Library](#)
 - [5.3 Bar Graphs and Pie Charts](#)
 - [5.4 Histogram, Polygon and Ogive](#)
 - [5.5 Dot Plot](#)
 - [5.6 Stem-and-leaf](#)
 - [5.7 Scatter Plot](#)
- 6. [Measures of Center and Spread](#)

- 6.1 Mean and Median
- 6.2 Variance
- 6.3 Standard deviation
 - Weighted Mean and std
- 6.4 Grouped mean and standard deviation
- 7. Measures of Position
 - 7.1 Percentiles
 - 7.2 SciPy and Percentile Rank
 - 7.3 Box Plot
- 8. Discrete Probability Distributions
 - 8.1 Mean (Expected Values) and Standard-Deviations
 - 8.2 Binomial Probability Distribution
 - Binomial Plot
- 9. Continuous Probability Distributions
 - 9.1 Uniform Probability Distribution
 - 9.2 Normal Probability Distribution
 - 9.3 Central Limit Theorem - Mean
- 10. Confidence Intervals
 - 10.1 Confidence Intervals for Mean (z distribution)
 - 10.2 Confidence Intervals for Mean (t distribution)
 - 10.3 Confidence Intervals for Proportions
 - Calculating Sample Size n
- 11. Hypothesis Testing
 - 11.1 Hypothesis Testing: Mean z test
 - 11.2 Hypothesis Testing: Mean t test
 - 11.3 Hypothesis Testing: Proportion z test
- 12. Linear Regression and Correlation
 - 12.1 Scatter Plot
 - 12.2 Correlation
 - 12.3 Linear Regression
 - Predictions
- 13. Analysis of Categorical Data
 - 13.1 Goodness of Fit
 - 13.2 Test of Independence
- 14. Advanced Topics
 - 14.1 Descriptive Statistics with Pandas
 - Pandas Library
 - Frequency and Relative Frequency
 - Descriptive Stats with Pandas
 - Mode
 - Percentiles
 - Summary stats, Skewness and Kurtosis

- 14.2 Uploading Excel Data
 - Descriptive Statistics (multiple columns)
- 14.3 Visualization
- 14.4 Regression
 - Multiple Linear Regression
 - Logistic Regression
- 14.5 ANOVA
 - One-way ANOVA
 - Tukey Paired Mean
 - Two-way ANOVA
 - Two-Factor ANOVA
- Appendix 1: Python Installation
- Appendix 2: Random Integer Generator
- Appendix 3: Factorial
- Code References
- Resources Links

0. Why Learn Python?

Artificial intelligence is changing how people write code, but it does not eliminate the need for introductory statistics students to learn Python. In this book, Python is not presented as an end in itself. It is a practical tool for learning how to think clearly about data, statistical methods, and the questions we want analysis to answer.

If AI tools were not available

If students were not allowed to use AI assistants, Python knowledge would be essential because they would need to:

- understand data structures and data cleaning
- translate statistical procedures into computational steps
- debug errors and recognize when code is wrong
- verify that a method actually matches the research question
- carry out analysis without relying on outside automation

In that setting, Python literacy would be foundational rather than optional.

When AI tools are available

In many future workplaces, students will likely have access to AI-powered coding tools. Even then, the real value of learning Python remains strong. Students still need to:

1. understand the statistical question well enough to ask for the right analysis
2. judge whether AI-generated code is correct
3. notice when an answer is incomplete, misleading, or wrong
4. ask better follow-up questions and refine an analysis thoughtfully

AI can help with syntax and code generation, but it cannot replace statistical understanding.

What students gain by learning Python

When students write Python code to load data, compute summary statistics, create graphs, and perform statistical tests, they are also learning how statistical reasoning works in practice. They become better able to:

- recognize common analysis patterns

- connect formulas to data workflows
- interpret output rather than just produce it
- trust but verify automated results
- adapt an analysis to a new situation

In that sense, learning Python supports statistical literacy, not just programming skill.

What AI cannot do for students

AI can generate code, but several parts of statistics still require human judgment:

- **Problem framing:** deciding what question should be asked and what kind of study or model is appropriate
- **Ethical reasoning:** identifying bias, fairness issues, and the consequences of analytical choices
- **Interpretation:** deciding whether results are practically meaningful, not just statistically significant
- **Communication and accountability:** explaining findings clearly and taking responsibility for the conclusions

These are human responsibilities, and they become more important, not less, in an AI-enabled world.

Conclusion

The goal of this material is not to turn every student into a software developer. The goal is to help students become informed users of data and statistics. Python provides a powerful way to practice that thinking. In the age of AI, Python literacy still matters because students must understand enough to guide the tools they use, evaluate what those tools produce, and make responsible statistical decisions.

1. Introduction to Python

1.1 Getting Started

Python is a general-purpose simple-syntax programming language created by Guido van Rossum in 1989. Python is widely used for scientific computing, data science, web development, machine learning, artificial intelligence and other advanced fields. Google, NASA, Netflix, and YouTube are some of the large organizations that use Python.

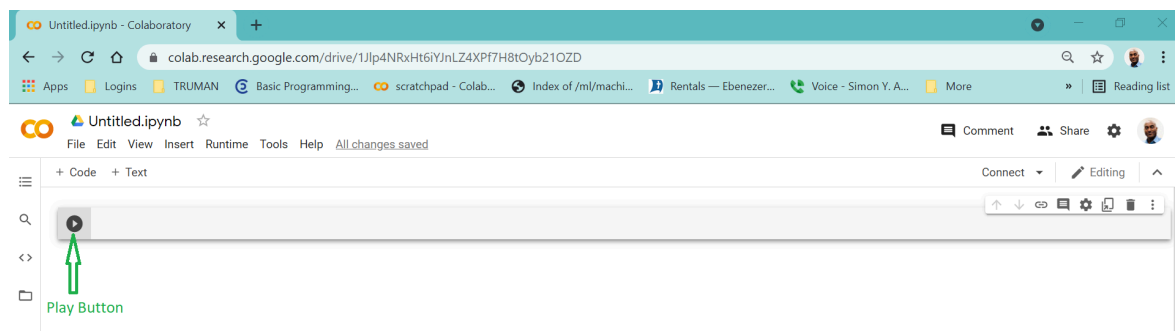
Google Colaboratory Notebook (Colab for short) is a free web-based Google product, similar to Google Docs, that requires no software installation for Python programming. A Colab Notebook is a list of **cells** created by adding a **code** or a **text** cell. When you create your own Colab notebooks, they are stored in your Google Drive account.

To Open a New Colab Notebook

[Watch demo video](#)

1. Click [this link](#)
2. Sign in with your Google account. If you are already signed in, continue to the next step.
You can create a free [Google account](#)
3. Click **New Notebook** from the bottom-right of the pop-up window **Note:** It may open a new notebook automatically sometimes.

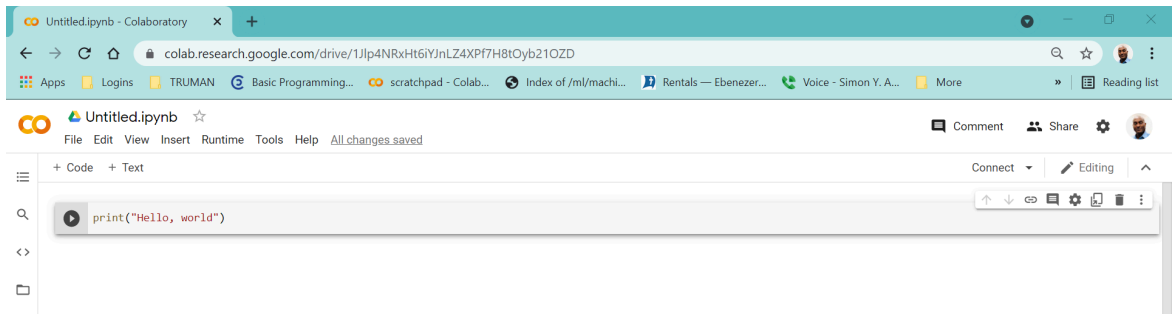
Note: Google Colab Notebook (Picture Below)



To Write Your First Python Code

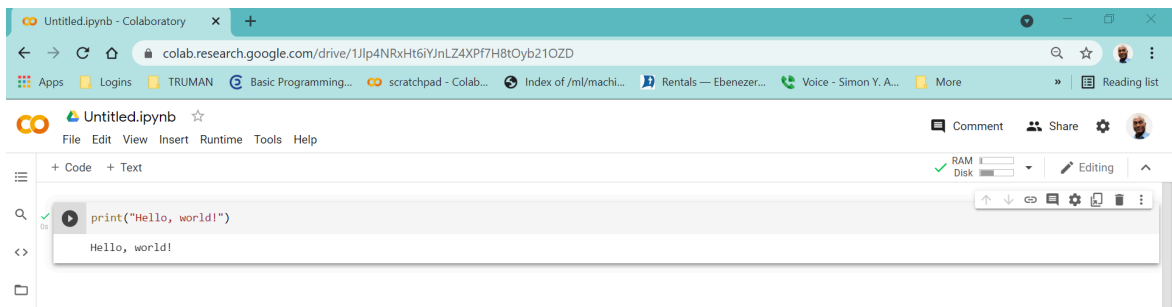
1. Click the code-cell next to the `play` button
2. Type `print("Hello, World!")`

See picture below



To Execute(run) Your First Python Code

1. Click the **play** button located before `print("Hello, World!")`
2. Or click your first Python code and press Ctrl and Enter on your keyboard. **Note:** It is Command and Enter for Mac(Apple) computers.
3. `Hello, world!` is displayed on a new line



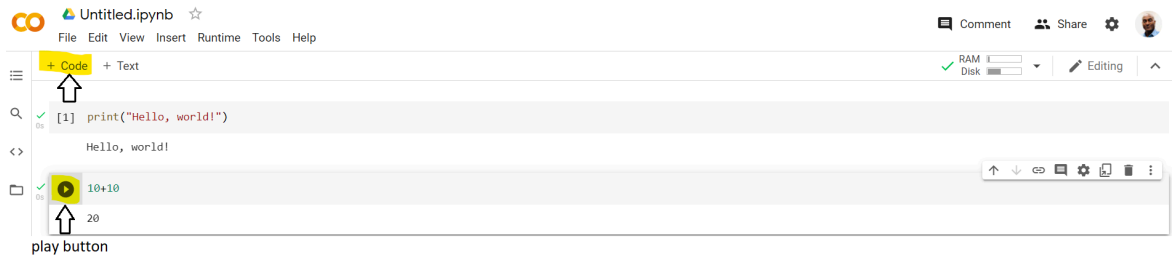
To Edit the Code

1. Click the code-cell with `print("Hello, World!")`
2. Start editing

Note: The Python `print()` function takes in **arguments** inside parenthesis and prints(displays) them in one line. In the above code, what happens if you leave out one of the quotation marks? `print("Hello, World!)` or parentheses `print("Hello, World!"` or misspelled the word print `prnit("Hello, World")` ? In all these cases, you will face error messages. Computer programming languages are literal and unambiguous. It expects *everything* to be accurate. Learning to identify and fix the causes of errors, **debugging**, is an important skill in programming.

To Write Your Second Python Code

1. Click **+ Code** in your notebook
2. Type `10+10` in the new code cell
3. Click the **play** button
4. The output `20` is displayed on a new line



Note: In the first code above `Hello, World!` was in double quotation marks. But in this example `10+10` is not in quotation marks. Python recognizes anything in quote, such as, "Hello, World!" as string **data type**, `str`, whereas `10` is recognized as an integer data type, `int`. Decimal numbers such as `10.1` are recognized as `float` data type.

Note: If you prefer **not** to use Google Colab Notebook, you can install free Python in your computer. See [appendix 1](#) for details.

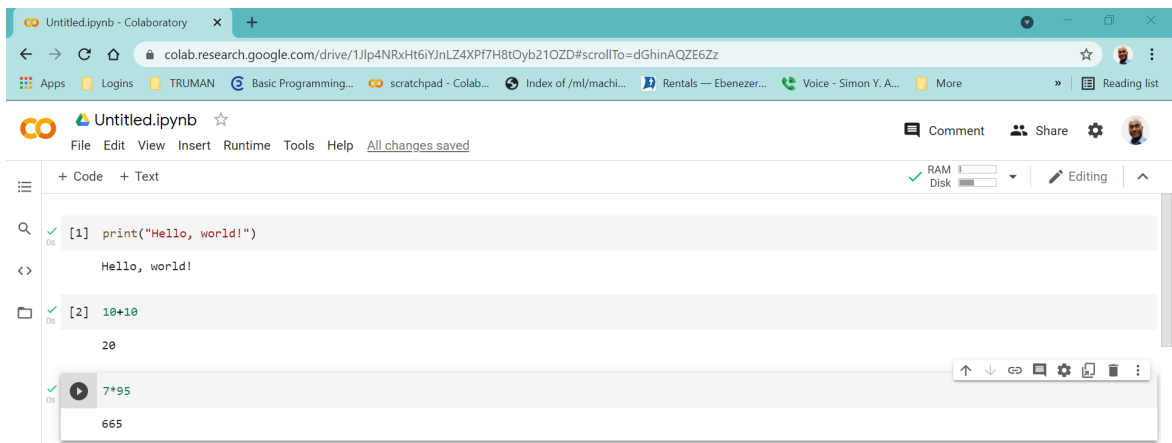
1.2 Python Arithmetic Operators

Name	Operator	Example	Result
Addition	+	5+6	11
Subtraction	-	5-6	-1
Multiplication	*	5*6	30
Division	/	5/6	0.83333
Exponentiation	**	5**2	5 ² = 25

To Write Your Third Python Code

7×95

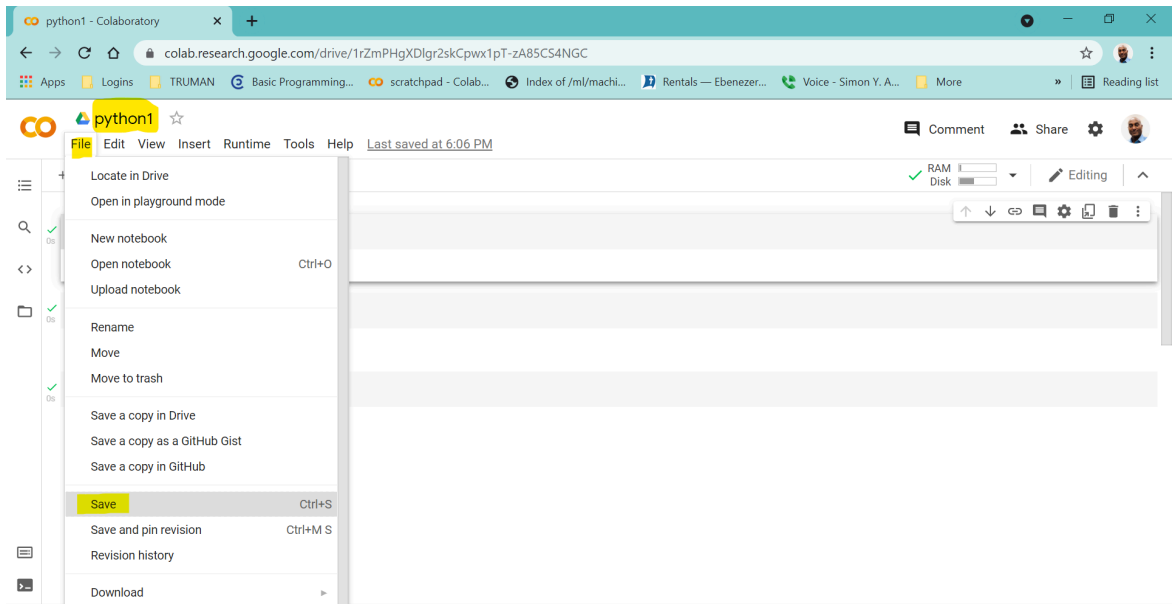
1. Click `+ Code` on a notebook to add a code cell
2. Type `7*95`
3. Press the `play` button to execute the code
4. `665` is the result



To Save Your Colab Notebook

[Watch demo video](#)

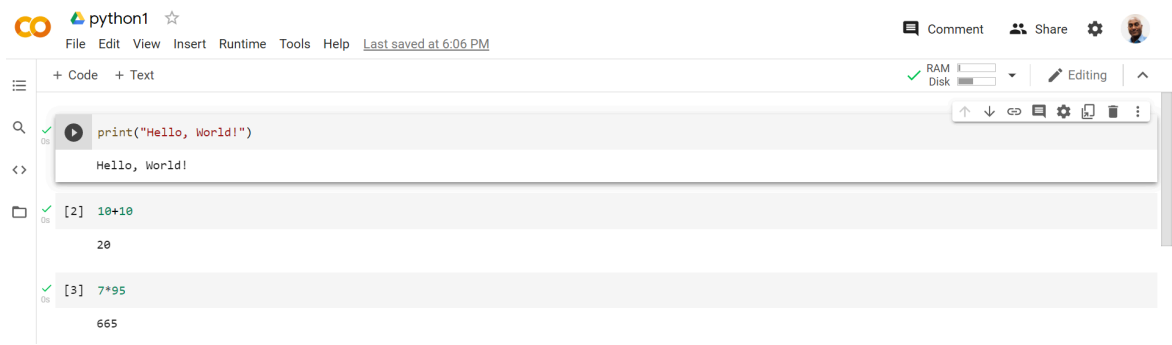
1. Click `Untitled.ipynb` at the top of the notebook
2. Delete the title and replace it by typing `python1`
3. Press the `enter` key
4. The file `python1` is now saved in your Google Drive
5. Click File and then Save
6. Close your browser



To Open Your python1 Notebook

1. Click this link <http://colab.research.google.com#create=true> or go to your [Google Drive](#).
2. Click `python1` from the list on the pop-up window
3. The notebook will open

OR click the File menu then Open Notebook



Note: For the examples below:

1. Add a new code cell on a Colab notebook
2. Type the code based on the statement of the example
3. Execute the code

Example: Multiply 5×6

```
In [ ]: 5*6
```

```
Out[ ]: 30
```

Example: Divide $10 \div 3$

```
In [ ]: 10/3
```

```
Out[ ]: 3.3333333333333335
```

Example: Exponentiate 5^2

```
In [ ]: 5**2
```

```
Out[ ]: 25
```

Example: Add and then divide

$$(2 + 3 + 5 + 4) \div 4$$

```
In [ ]: (2+3+5+4)/4
```

```
Out[ ]: 3.5
```

Note: If you want to add the numbers in the numerator $2 + 3 + 5 + 4$ first and then divide it by the denominator, 4, you will need to **save** the sum of the numerator first. We will cover how to save the sum in a **variable name** in the next [section](#).

Example: Add $\frac{1}{2} + \frac{1}{3} + \frac{2}{5} + 10$

```
In [ ]: 1/2+1/3+2/5+10
```

```
Out[ ]: 11.233333333333334
```

Note: Adding parentheses to the fractions makes it clearer (see below)

```
In [ ]: (1/2)+(1/3)+(2/5)+10
```

```
Out[ ]: 11.233333333333334
```

Example: Perform the operations $6 \div 3 \times 7 - 6.7 + 3.5$

```
In [ ]: 6/3*7-6.7+3.5
```

```
Out[ ]: 10.8
```

Example: Perform the operations $43 + (8 - 5)^3 \div (34 - 5^2)$

```
In [ ]: 43+(8-5)**3/(34-5**2)
```

```
Out[ ]: 46.0
```

Example: Perform the operation $13\frac{1}{2} + 2.4 \times 1.4^2$

```
In [ ]: 13+1/2+2.4*1.4**2
```

```
Out[ ]: 18.204
```

Note: Radicals in Rational Exponents [Watch demo video](#)

If x is a real number, a is integer and b is positive integer then

$$\sqrt[b]{x^a} = x^{\frac{a}{b}}$$

For example:

$$\sqrt{8} = 8^{1/2}$$

$$\sqrt[3]{8} = 8^{1/3}$$

$$\sqrt[4]{8^3} = 8^{3/4}$$

Example: Evaluate $\sqrt{7} = 7^{1/2}$

```
In [ ]: 7**(1/2)
```

```
Out[ ]: 2.6457513110645907
```

```
In [ ]: 7**.5
```

Out[]: 2.6457513110645907

Note: $1/2 = .5$

Example: Evaluate $\sqrt[3]{7}$

$$\sqrt[3]{7} = 7^{1/3}$$

In []: `7**(1/3)`

Out[]: 1.912931182772389

Example: Find $\sqrt{5^2 - 4}$

Note: $\sqrt{5^2 - 4} = (5^2 - 4)^{1/2}$

In []: `(5**2-4)**(1/2)`

Out[]: 4.58257569495584

Example: Follow standard math order of operations to evaluate:

$$\sqrt{5^2 - 4 \times 2 \times (-3)}$$

Note: $\sqrt{5^2 - 4 \times 2 \times (-3)} = (5^2 - 4 \times 2 \times (-3))^{1/2}$

In []: `(5**2-4*2*(-3))**(1/2)`

Out[]: 7.0

Example: Follow standard math order of operations to evaluate:

$$-2 + \sqrt{5^2 - 4 \times 2 \times (-3)}$$

Caution: Everything under the square root is inside a parenthesis raised to 1/2. That is $5^2 - 4 \times 2 \times (-3)$ is raised to 1/2. You can do this in two steps using variables in the next [section](#)

In []: `-2+(5**2-4*2*(-3))**(1/2)`

Out[]: 5.0

Example: Evaluate:

$$\frac{-2 + \sqrt{5^2 - 4 \times 2 \times (-3)}}{2 \times 2}$$

Notice how paired parentheses are used to enforce order of operations.

```
In [ ]: (-2+(5**2-4*2*(-3))**(1/2))/(2*2)
```

```
Out[ ]: 1.25
```

Scientific Notations

Example: Divide $\frac{1}{186000}$

```
In [ ]: 1/186000
```

```
Out[ ]: 5.376344086021505e-06
```

Note: The Python output `5.376344086021505e-06` is in scientific notation.
 $5.376344086021505e-06 = 5.376344086021505 \times 10^{-6}$

1.3 Python Variables

[Watch demo video](#)

A variable is a location that stores temporary data within a program. Variables can be modified, stored or displayed whenever needed. Variables that you define in one cell can later be used in other cells.

In Python, variables are defined, created, and declared once you assign a value to it. Variable names can be arbitrarily long and can contain both letters and numbers, but they have to begin with a letter. Avoid using one of Python's **keywords** (reserved words) such as `print`, `input`, `and`, `or`, `def`, `if` etc. Here is a link to the complete [list](#).

The **equal sign** `=` is called the assignment operator. It takes the value of the right-hand side of the equal sign and *assigns* it to the variable name on the left side of the equal sign.

Example: Find the average of three test scores: 75, 80 and 90? The average is found by adding the three test scores first and then dividing the total by 3.

1. Click `+ Code` to add a code cell
2. Type `total = 75+80+90`
3. Type `total/3`
4. `81.66666666666667` is the result

```
In [ ]: total = 75 + 80 + 90
total/3
```

```
Out[ ]: 81.66666666666667
```

Example: Find the average of three test scores: 75, 80 and 90? The average is found by adding the three test scores first and then dividing the total by 3. Save the average in a variable

named `ave`

1. Click `+ Code` to add a code cell
2. Type `total = 75+80+90`
3. Type `ave = total/3`
4. Type `print(ave)`
5. `81.66666666666667` is the result

```
In [ ]: total = 75+80+90
ave = total/3
print(ave)
```

81.66666666666667

Note:

- The value of `75+80+90` is evaluated and stored in `total`
- The value of `total` from line one is used to evaluate `total/3` in the second line and the new value of `total/3` is stored in `ave`.
- `print(ave)` is used to display the value stored in `ave`

Note: In a code-cell with multiple lines, the program executes one line at a time in sequence from top to bottom.

Note: This time, `aver` is not in quotation marks in the `print(aver)` function. Anything in quotation mark is displayed as is. Variable names are not in quotes and are separated by a comma.

Example: Evaluate $\frac{2+3+5+4}{4}$ by storing the sum of the numerator in a variable named `numerator_sum` first. Then add another line that divides this sum by 4.

1. Click `+ Code` to add a code cell
2. Type `numerator_sum = 2 + 3 + 4 + 5`
3. Type `numerator_sum/4` on a new line. (It will display the value of `numerator_sum/4`)
4. Press the `play` button
5. `3.5`, the value of the last executed line is displayed

```
In [ ]: numerator_sum = 2 + 3 + 5 + 4
numerator_sum/4
```

Out[]: 3.5

Example: $-2 + \sqrt{5^2 - 4 \times 3 \times (-3)}$

```
In [ ]: radicand = 5**2 - 4*3*(-3)
-2 + radicand**(1/2)
```

Out[]: 5.810249675906654

Note: In Python, the same variable name can be used to store different values multiple times, but only the last assignment is stored for future use. The old value is **updated** every time a new value is assigned. See example below.

Note: The `print()` function displays anything in quotes as is but variable names are not in quotes and are separated from the quote by a comma.

```
In [ ]: a = 0
print('The first value of a is',a)
a = 10
print("The updated second value of a is ",a)
a = a + 5
print('The last updated, third value, of a is ', a)
```

The first value of a is 0

The updated second value of a is 10

The last updated, third value, of a is 15

Note:

1. In the first line `a = 0`, zero is stored in `a`.
2. In the third line `a = 10`, 10 is assigned to `a`. Here the value of `0` that was stored in `a` is updated(replaced) by `10`.
3. In the fifth line, `a = a + 5`, the `a + 5` on the right side of the equal sign is evaluated as `10 + 5`. This is because `10` was stored in `a` from line 3. Then `10 + 5` is stored in `a` replacing the previous value of `10`.
4. The last line, `a` is displayed with its last updated value of `15`.

Deleting a Variable

Use the **keyword** `del` to delete a variable. See example below.

```
In [ ]: seconds_in_week = 60*60*24*7
del seconds_in_week
print(seconds_in_week)
```

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-403-cf943aec5ccc> in <module>()
      1 seconds_in_week = 60*60*24*7
      2 del seconds_in_week
----> 3 print(seconds_in_week)

NameError: name 'seconds_in_week' is not defined
```

Note: Both the value of `60*60*24*7` and the variable name `seconds_in_a_week` are deleted from the memory space in line 2. Asking Python to print `seconds_in_a_week` after the delete yields an error. `NameError: name seconds_in_a_week is not defined`.

Collab provides a link to the popular question and answer website <https://stackoverflow.com> to help you look for any possible fixes to the error.

Example: A line that passes through two ordered-points (x_1, y_1) and (x_2, y_2) has slope, $m = \frac{y_1 - y_2}{x_1 - x_2}$ where $x_1 \neq x_2$. Use this formula to find the slope of a line that passes through $(2, 5)$ and $(4, 7)$. See code below.

```
In [ ]: x1 = 2
        y1 = 5
        x2 = 4
        y2 = 7

        m = (y1-y2)/(x1-x2)
        m
```

```
Out[ ]: 1.0
```

1.4 Python Comments

Comment is some text in a computer program that is for the human reader and is ignored by the computer when running the program. In Python one line comments begin with a hash symbol `#`. Anything written after the keyboard key hash `#` till the end of the line is considered *comment* and Python does not execute comments. multiple line comments can be done by enclosing them in triple quotes `"""`. Comments (notes) are useful to explain what the program is doing. See example below.

```
In [ ]: # find the slope of a line that passes through (2,5) and (4,7)
        x1 = 2      # 2 came from the given point (2,5)
        y1 = 5
        x2 = 4      # 4 came from the second point (4,7)
        y2 = 7

        m = (y1-y2)/(x1-x2)    # this is the formula for a slope
        print(m)

        """ Or you can directly substitute the given number in to
        the formula. See the code below """

        m = (5-7)/(2-4)
        print("the second way",m)
```

```
1.0
the second way 1.0
```

1.5 The NumPy Library

[Watch demo video](#)

In Python, a software library is a reusable collection of code. NumPy, pronounced "Num-Pie" for Numerical Python is the fundamental package for scientific computing with Python.

<https://numpy.org/>

Unlike `print()` which is a built-in function, we need to `import` numpy before using its functions such as `numpy.absolute()`. Importing refers to allowing Python to access the script from another Python module or library. The numpy library provides access to an extensive mathematical functions.

```
import numpy
```

To access one of its functions, specify the name (or its alias) of the library and the name of the function, separated by a dot(period).

Some Python Functions (Methods) from the numpy library

Name	Method	Example	Results (rounded)
Square root	<code>numpy.sqrt(x)</code>	$\sqrt{5}$	2.23606797749979
Absolute value	<code>numpy.absolute(x)</code>	-5	5
Rounding	<code>numpy.round(x,2)</code>	1.3333	1.33
the number pi	<code>numpy.pi</code>	π	3.141592653589793
e	<code>numpy.e</code>	e	2.718281828459045
e raised to	<code>numpy.exp(x)</code>	e^2	7.38905609893065
Natural log	<code>numpy.log(x)</code>	$\ln 5$	1.6094379124341003
Common log	<code>numpy.log10(x)</code>	$\log(9)$	0.9542425094393249

Example: Given the formula $E = |z| \sqrt{\frac{p(1-p)}{n}}$, evaluate E , if $z = -2.575$, $p = 0.34$, and $n = 800$.

Option 1: substitute all into the formula

```
In [ ]: import numpy as np
np.absolute(-2.575)*(0.34*(1-0.34)/800)**(1/2)
```

```
Out[ ]: 0.043126445627943884
```

Option 2: use some variables to break-down the steps

```
In [ ]: import numpy as np
a = 0.34*(1-.34)/800
b = a**(1/2)
np.absolute(-2.575)*b
```

Out[]: 0.043126445627943884

Option 3: declare z, p and n as variables and assign the given values first then use the formula to evaluate E using these variables:

```
In [ ]: import numpy as np
z = -2.575
p = 0.34
n = 800
E = np.absolute(z)*(p*(1-p)/n)**(1/2)
E
```

Out[]: 0.043126445627943884

Example: Round the value of $\frac{1}{3}$ to two decimal places.

```
In [ ]: import numpy as np
np.round(1/3,2)
```

Out[]: 0.33

Example: Round the value of $\sqrt{2}$ to three decimal places.

```
In [ ]: import numpy as np
np.round(2**(1/2),3)
```

Out[]: 1.414

Example: What is the (rounded) value of π in Python?

1. Click `+ Code`
2. Type `import numpy`
3. Type `numpy.pi`
4. Click `play` button to execute the code
5. `3.141592653589793` (displayed)

```
In [ ]: import numpy
numpy.pi
```

Out[]: 3.141592653589793

Example: Area of a circle with radius r is given by the formula $A = \pi r^2$

- Find the area of a circle with radius 5.
- Round your answer to a whole number.

1. Type `import numpy`
2. Type `numpy.pi*5**2` on the second line
3. Click `play` to execute the code

```
In [ ]: import numpy
        numpy.pi*5**2
```

Out[]: 78.53981633974483

```
In [ ]: import numpy
        Area = numpy.pi*5**2
        round(Area,0)
```

Out[]: 79.0

Example: what is the rounded value of e^3 ?

```
In [ ]: import numpy
        numpy.exp(3)
```

Out[]: 20.085536923187668

Example: what is the rounded value of $\sqrt{3}$?

```
In [ ]: numpy.sqrt(3)
```

Out[]: 1.7320508075688772

Example: use `**` to find $\sqrt{3}$

```
In [ ]: 3**(1/2)
```

Out[]: 1.7320508075688772

1.6 Input data from Keyboard

[Watch demo video](#)

input()

We used the `print()` function to display program output. The `input()` function is used to read user input from the keyboard. When the built-in-function `input()` is called, the program stops and waits for the user to type from the keyboard. When the user presses Return or Enter, the program resumes and `input` returns what the user typed as a `string` data type by default.

1. Click `+ Code` to add a code-cell
2. Type `x = input()`
3. Type `print(x)`
4. Click `play`

5. Type the number 2 using your keyboard and press `enter` .

See code below

```
In [ ]: x = input()
        print(x)
```

```
2
2
```

Note: The value of `input()` entered from the keyboard is stored in `x`

It is a good practice to print a prompt telling the user what to input. For example, `input("what is your name?")` will make it clear to the user what to enter. See code below.

1. Click `+ Code`
2. Type `x = input("What is your name? ")`
3. Type `print("Your name is", x)`
4. Click `play`
5. Enter `Don` from your keyboard

```
In [ ]: x = input("What is your name? ")
        print("Your name is",x)
```

```
What is your name? Don
Your name is Don
```

Note: The value of `input()` entered by the user is stored in `x` . That is `x = Don`

Input Examples

Example: Write a Python program that accepts an integer x and computes and prints the square-root of x . See code below.

```
In [ ]: x = input("Enter any integer number: ") #ask the user for a number
        x = int(x)                             # converts the input from string to integer (casting)
        y = x**.5
        print("The square-root is", y)
```

```
Enter any integer number: 5
The square-root is 2.23606797749979
```

Example: Write a Python program which asks the user for a radius of a circle and displays the area using the formula $A = \pi r^2$. See code below.

```
In [ ]: r=input("Enter the radius of a circle: ") #ask the user for radius
        r=float(r)                               # converts the string to float
        import numpy                            # we need the number pi
```

```
area = numpy.pi*r**2
print("The area of the circle is", area, 'square units')
```

Enter the radius of a circle: 6

The area of the circle is 113.09733552923255 square units

Example: Write a Python program to calculate adult body mass index for a given weight in pounds and height (feet and inches) using the formula, $BMI = 703 \frac{\text{weight}(lb)}{\text{height}(in)^2}$. See code below.

```
In [ ]: w=input("Enter weight in pounds:") #ask the user for a number
h1 = input("Enter height feet only:")
h2 = input("Enter inches:")
w,h1,h2 = float(w),float(h1),float(h2) # converts the number to float
bmi = 703*(w/(h1*12+h2)**2)
print("The BMI is", bmi)
```

Enter weight in pounds:60

Enter height feet only:5

Enter inches:2

The BMI is 10.972944849115505

2. Python Lists

[Watch demo video](#)

In Python, lists are used to store multiple items in a single variable. Lists are created using square brackets separated by commas. List items, called elements, are ordered, changeable, and allow duplicate values.

2.1 Creating and Accessing Lists

Example: Create a Python list of 24, 22, 5, 34, 81, and 10. Save the list to the variable name `my_list` and display it.

1. Click `+ Code` to add a code-cell
2. Type `my_list = [24,22,5,34,81,10]`
3. Type `my_list`
4. Click `play` button

```
In [ ]: my_list = [24,22,5,34,81,10]
my_list
```

Out[]: [24, 22, 5, 34, 81, 10]

my_list = [24, 22, 5, 34, 81, 10]

list name items

Example: Create a list named x using the following integers: 10, 14, 20 and 15. Print the list.

```
In [ ]: x = [10,14,20,15]
x
```

```
Out[ ]: [10, 14, 20, 15]
```

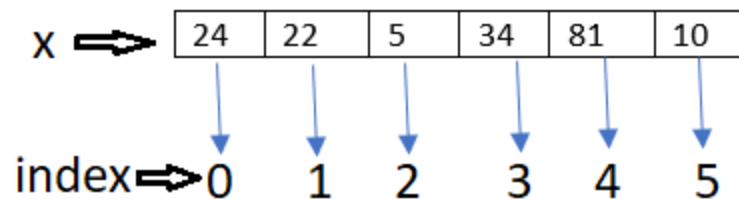
Example: Create a list named x using the following colors: red, yellow, green and blue. Print the list.

```
In [ ]: y = ['red','yellow','green','blue'] # string data in quotes
y
```

```
Out[ ]: ['red', 'yellow', 'green', 'blue']
```

List elements are **indexed**, and we can use the index operator `[]` to access its elements. The first element has index 0, the second element has index 1 etc.

Example: Create a list of 24, 22, 5, 34, 81 and 10. Save the list in `x` and display the entire list .



```
In [ ]: x = [24, 22, 5, 34, 81, 10]
x
```

```
Out[ ]: [24, 22, 5, 34, 81, 10]
```

Example: From the list `x` above, display the first element. See code below.

```
In [ ]: x[0]
```

Out[]: 24

Example: From the list `x` above, display the third element. See code below.

```
In [ ]: x[2]
```

Out[]: 5

Example: From the list `x` above, display the **last** element?

1. Click `+ Code`
2. Type `x = [24,22,5,34,81,10]`
3. Type `x[-1]`
4. Click `play` button

```
In [ ]: x = [24,22,5,34,81,10]
x[-1]
```

Out[]: 10

Note: The index for the last element of a list is -1 , the second from the last is -2 , the third from the last is -3 , etc

Example: Create a list named `z` using the following items: Monday, Tuesday, Wednesday, 4, 5, 6, and 7.0. Here, we have string, integer and float data types.

```
In [ ]: z = ['Monday', 'Tuesday', 'Wednesday', 4, 5, 6, 7.0]
```

To display the whole list, enter its name or use the **print** function.

```
In [ ]: print(z)
['Monday', 'Tuesday', 'Wednesday', 4, 5, 6, 7.0]
```

```
In [ ]: z[0]    # the first item
```

Out[]: 'Monday'

```
In [ ]: z[-1]   # the last item
```

Out[]: 7.0

```
In [ ]: print(z[6])    # the 7 th item
print(z[-2])           # the second from the last
```

7.0

6

Note: To display multiple items of a Python list, use the slicing operator `[a:b]` from `a` to `b-1`. For example, `z[0:3]` will use index 0, 1, and 2. Notice 3 is not included.

Example: Display the first 3 elements of `z`.

```
In [ ]: print(z[0:3])
```

```
['Monday', 'Tuesday', 'Wednesday']
```

An Empty List

A list that contains no elements is called an empty list; you can create one with empty brackets, `[]`

```
x = []
```

```
In [ ]: x=[]  
x
```

```
Out[ ]: []
```

Lists are mutable

Elements of a list can be changed (replaced).

Example: From the list, `x=[10,20,30]`, replace the second element, `x[1]=20`, with the letter `a`.

```
In [ ]: x = [10, 20, 30]  
print(x)  
x[1] = 'a'    # assign 'a' to the second element (replace 20 with 'a')  
print(x)
```

```
[10, 20, 30]  
[10, 'a', 30]
```

2.2 List Operations

1. `.count()` returns the frequency of an item. `my_list.count('Monday')`
2. `.sort()` sorts the list `my_list.sort()` in ascending order
3. `.reverse()` reverses the order `my_list.reverse()` in descending order
4. `min()` returns the smallest item `min(my_list)`
5. `max()` returns the largest item `max(my_list)`
6. `sum()` returns the sum of the items `sum(my_list)`
7. `len()` returns the number of items on the list (size) `len(my_list)`

Example: what is the frequency count for the element "Monday" for the list `x = ['Monday', 'Tuesday', 'Wednesday', 'Monday', 4, 5, 6, 4, 4, 7.0]` ?

```
In [ ]: x = ['Monday', 'Tuesday', 'Wednesday', 'Monday', 4, 5, 6, 4, 4, 7.0]
        x.count('Monday')
```

Out[]: 2

Example: Sort the list `y = [10,14,20,15]` . Print the sorted list.

```
In [ ]: y = [10,14,20,15]
        y.sort()
        y
```

Out[]: [10, 14, 15, 20]

Example: Use the sorted list `y` from above to reverse the order.

```
In [ ]: y = [10,14,20,15]
        y.reverse()  # revers order
        y
```

Out[]: [15, 20, 14, 10]

Example: Use the list `y = [10,14,20,15]` above to find the minimum item of the list.

```
In [ ]: y = [10,14,20,15]
        min(y)
```

Out[]: 10

Example: Use the list `y = [10,14,20,15]` above to find the maximum item of the list.

```
In [ ]: y = [10,14,20,15]
        max(y)
```

Out[]: 20

Example: Use the list `y = [10,14,20,15]` above to find the sum of all items in the list.

```
In [ ]: y = [10,14,20,15]
        sum(y)
```

Out[]: 59

Example: Use the list `y = [10,14,20,15]` above to find the length of the list. That is, how many items are there in the list?

```
In [ ]: y = [10,14,20,15]
        len(y)
```

Out[]: 4

Example: Write a Python program that displays the sum, minimum, maximum and length of the values of the elements in a given list, `x=[1,2,3,4,5,6,7,8,9]`

```
In [ ]: x=[1,2,3,4,5,6,7,8,9]

print('the sum is:',sum(x))
print('the min is:',min(x))
print('the max is:',max(x))
print('the length is:',len(x))
```

```
the sum is: 45
the min is: 1
the max is: 9
the length is: 9
```

3. Python Syntax

"The syntax of a computer language is the set of rules that defines the combinations of symbols that are considered to be correctly structured statements or expressions in that language." Wikipedia

Syntax Summary

1. **Arithmetic Operators:** `+` (add), `-` (subtract), `*` (multiply), `/` (divide), `**` (exponent).
(e.g. `700 * 20` see [Arithmetic Operators](#))
2. **Assignment operator** `=` (e.g. `x = 100`)
3. **Comments:** comment start with `#` and lasts to end-of-line. (e.g. `# this is a comment` see [Comments](#))
4. **Strings:** string (text data) is declared by either single or double quotes. (e.g. `x = 'red'`)
5. **Variable:** A variable is created via initial assignments and can hold integer, float, string etc. (e.g. `y = 10.5` see [Variables](#))
6. **Data Types:** int (integers), float (decimals), str (string), bool (True or False) (e.g. `y = 5` `y` is integer data type `int`)
7. **Statements:** Statements start at the left end (column one) of a line and end with a newline. - Block statements are **indented**. - Python is **case sensitive**.
8. **Comparison Operators:** `==`, `!=` (not equal), `<`, `<=`, `>`, `>=`, `in`, `not in`, `is`, `not is`. (see [Logical Comparisons](#))
9. **Logical Operators:** `and`, `or`, `not`.
10. **if condition:** if line ends with a colon followed by indented block (see [if Statements](#))

11. **While loop:** while line ends with a colon followed by indented block (see `while` [Loop](#))
12. **for loop:** for item in sequence ends with a colon with indented block (see `for` [Loop](#))
13. **List:** `lst = [v1, v2, ..., vn]` variable-size, mutable, dynamic array. (e.g. `x = [22, 17, 50]` see [List](#))
14. **Tuple:** `tup = (v1, v2, v3, ...)` Immutable fix-sized array.(e.g. `x = (22, 17, 50)`)
15. **Dictionary:** `dic = {k1:v1, k2:v2, ...}` mutable key-value pairs, associative array, map. (e.g. `x = {'even': [2,4,6], 'odd' : [1,3,5]}`)

[Reference](#)

3.1 `if` Statements

[Watch demo video](#)

The `if` statement is used for decision making. If a specified condition is true, then a block of code is executed. If not true, the statement in the else if (`elif`) is checked and executed if true. If all are false, the process continues until the (`else`) statement.

Syntax

```
if condition1:           # this line ends with a colon :
    block statements      # this line(s) are indented
elif condition2:
    block statements
...
else:                     # no condition in this line
    block statements
```

Example: Given a number, for example 4, decide if it is positive or negative.

1. Click `+ Code` to add a code-cell
2. Type `a = 4`
3. Type `if a > 0:`
4. Type `print(' a is positive')`
5. Type `else:`
6. Type `a is negative or zero`
7. Click `play` button

```
In [ ]: a = 4
if a > 0:
    print('a is positive')
```

```
else:
    print('a is negative or zero')
```

a is positive

Note: The if condition `a > 0` is either true or false. If true, the statement below the if condition is executed. If false the program goes to the `else` statement. In this case `a=4 > 0` is true and the statement `print('a is positive')` is executed.

Python Comparison (Logical) Operators

A Boolean expression is an expression that is either true or false. The operator `==` compares two operands and produces `True` if they are equal and `False` otherwise.

Name	Operator	Example	Result
Greater than	<code>></code>	<code>5>6</code>	<code>False</code>
Greater than or Equal to	<code>>=</code>	<code>5>=4</code>	<code>True</code>
Less than	<code><</code>	<code>5<4</code>	<code>False</code>
Less than or equal to	<code><=</code>	<code>5<=5</code>	<code>True</code>
Equal	<code>==</code>	<code>5==6</code>	<code>False</code>
Not Equal	<code>!=</code>	<code>5!=6</code>	<code>True</code>

Example: Change the number 4 to -4 in the above example. What is the result. See code below.

```
In [ ]: a = -4
        if a > 0:
            print('a is positive')
        else:
            print('a is negative or zero')
```

a is negative or zero

Example: Given the numbers 2 and 3, decide if 2 is less than 3, greater than 3 or equal to 3.

```
In [ ]: a = 2
        b = 3
        if a < b:                # if statement ends with colon :
            print("2 is less than 3")    # block statement indented
        elif a > b:
            print("2 is greater than 3")
        else:
            print("2 is equal to 3")
```

a is less than b

Example: Use `input()` function to ask the user for two integers and use these numbers in the `if` statement to decide if the first number is less, greater, or equal to the second number. This time we need to convert the string variable to decimal using the `float()` function.

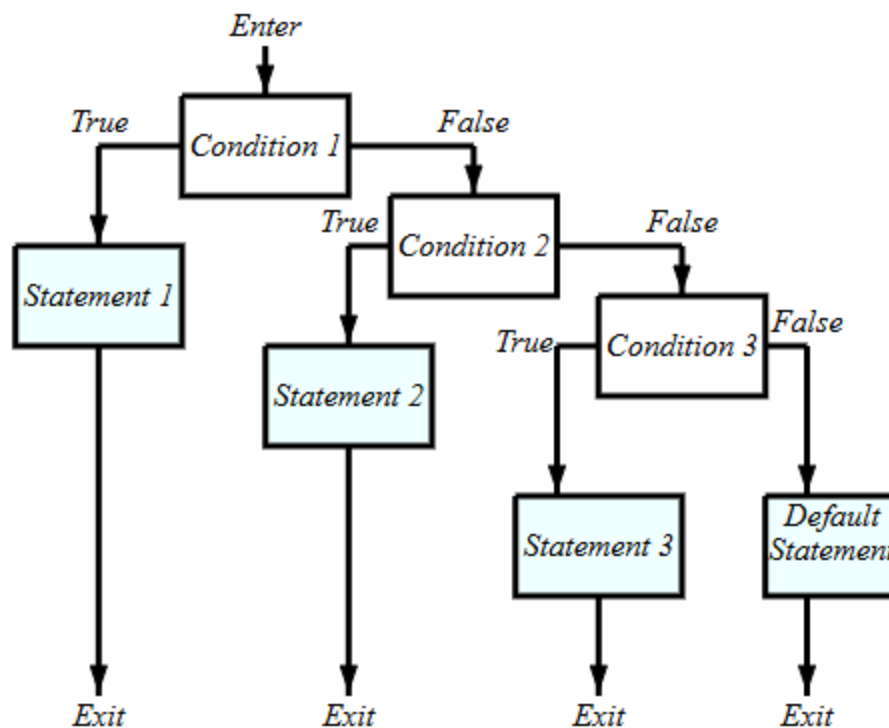
```
In [ ]: x = input("Enter first number :")
        y = input("Enter second number :")

        x = float(x)    # converts (casts) the default string value to an integer value
        y = float(y)

        if x < y:
            print("The first number is less than the second!")
        elif x > y:
            print("The first number is greater than the second!")
        else:
            print("They are equal!")
```

```
Enter first number :6
Enter second number :10
The first number is less than the second!
```

If Statement Flowchart



Example: The slope-intercept equation of a line with slope m and y -intercept b is given as $y = mx + b$. If m is positive, y is increasing, if it is negative y is decreasing and if it is zero y is a constant. Write a program with slope m and use the `if` statement to decide if the line is increasing, decreasing or constant.

```
In [ ]: m=3                                # the slope is selected to be 3

if m>0:
    print ("increasing")
elif m<0:
    print("decreasing")
else:
    print("constant")
```

increasing

Example: Write a Python program that asks a user to enter two numbers. Compute and display the sum of the two numbers. If the sum is greater than 10, compute and display the product of the two numbers. If the sum is less than 10 compute and display the product of the two numbers plus 5. Otherwise display the text 'the sum is 10'.

```
In [ ]: x = input('enter the first number:')
y = input('enter the second number:')

x = float(x)
y = float(y)

sum1 = x + y          # 'sum' is a reserved key word in Python
print('the sum is',sum1)

if sum1 > 10:
    print('the product is',x*y)
elif sum1 < 10:
    print('the product plus 5 is',x*y+5)
```

```
enter the first number:5
enter the second number:6
the sum is 11.0
the product is 30.0
```

You can also use the built-in Python **input** function to read and return a line of string(text). Every time you run the code below, you will be asked to enter the slope of your choice. The function **int()** returns (converts) the value to an integer, whenever possible. [Reference to built-in Python functions](#)

```
In [ ]: m=input("enter any integer number (your choice) for the slope m = ")
m = int(m)      # cast the input as integer

if m>0:          # check if m>0
    print ("increasing")
elif m<0:        # check if m<0 if the if was False
    print("decreasing")
else:
    print("constant")
```

```
enter any integer number (your choice) for the slope m = -9
decreasing
```

3.2 Functions

[Watch demo video](#)

A function is a reusable piece of code. It is a block of code which only runs when it is called. You can pass data, known as arguments, into a function. A function can return data as a result. In Python a function is defined using the `def` keyword. This is similar to built-in functions such as `abs()` or `print()` but "your own".

Syntax

```
def func_name(arguments):  
    block statements  
    return result # optional
```

`def` is a **keyword** in Python. It is used to define a function.

Example: Use the formula $C = (F - 32) \frac{5}{9}$ to convert a temperature of $F = 70$ degree Fahrenheit to C degree Celsius. See code below.

```
In [ ]: F = 70  
        C = (F-32)*5/9  
        C
```

```
Out[ ]: 21.11111111111111
```

Example: Make a Python that provides the equivalent degree Celsius for a given temperature in degree Fahrenheit. Use the formula $C = (F - 32) \frac{5}{9}$.

1. Click `+ Code` to add a code-cell
2. Type `def temp(F):`
3. Type indent `C = (F-32)*5/9`
4. Type indent `return C`
5. Click `play` button

Note: No output is displayed for function. It will display output when it is called with specific argument is provided. See example below.

```
In [ ]: def temp(F):  
        C = (F-32)*5/9  
        return C
```

Note: the name of this function is chosen to be `temp`. There is one argument 'F'. The indented body of the function uses the formula to compute C. The return line displays the value of C.

Example: Use the function `temp(F)` above to convert a 100 degree Fahrenheit to Celsius.

1. Click `play` button on `temp` code cell above
2. Type `temp(100)`
3. Click `play` button

In []: `temp(100)`

Out[]: 37.77777777777778

Example: Use the `temp` function above to convert $F = 0$ to C. See code below.

In []: `temp(0)`

Out[]: -17.77777777777778

Example: Use the quadratic formula to make a function for solving a quadratic equation of the form:

$$ax^2 + bx + c = 0$$

The solution(s):

If $b^2 - 4ac < 0$ No real solutions

If $b^2 - 4ac = 0$ There is one real solution, $x = -\frac{b}{2a}$

If $b^2 - 4ac > 0$ There are two real solutions

$$x1 = \frac{-b - \sqrt{b^2 - 4ac}}{2a} \text{ and } x2 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

See code below:

```
In [ ]: # quadratic equation solver
def solvequad(a,b,c): # the name is solvequad and arguments a,b,c in that order
    a = float(a)      # converts or casts to float
    b = float(b)
    c = float(c)
    if b**2-4*a*c < 0:
        return print("No real solutions")
    elif b**2-4*a*c == 0:
        x = -b/(2*a)
        return print("The solution is", x)
    else:
        x1 = (-b-(b**2-4*a*c)**.5)/(2*a)
        x2 = (-b+(b**2-4*a*c)**.5)/(2*a)
        return print("The solutions are: ",x1," and ",x2)
```

Example: Solve $x^2 - 4 = 0$,

Here $a=1$, $b=0$, $c=-4$ because $1x^2 + 0x + (-4) = 0$

In []: `solvequad(1,0,-4)` # run the solvequad function above first

The solutions are: -2.0 and 2.0

Alternatively, you can pass `a=1,b=0,c=-4` to the function as follows:

```
In [ ]: solvequad(a=1,b=0,c=-4)      # run the solvequad function above first
```

The solutions are: -2.0 and 2.0

Run the function `solvequad(a,b,c)` before you call `solvequad(1,0,-4)`.

Example: Solve $3x^2 + 5x - 7$

Here $a = 3, b = 5, c = -7$

```
In [ ]: solvequad(3,5,-7)
```

The solutions are: -2.5733844181517584 and 0.9067177514850918

3.3 Syntax Errors

[Watch demo video](#)

Incorrectly written command or instruction will cause a failure in execution. For example, **indentation** (any length) and `:` (colon) are part of the syntax for the `if` statement. In the program below, colon `:` is missing in the first line.

Example: In the code below, colon is missing in the first line of the if condition.

```
In [ ]: if 2 < 3                                # colon : is missing
        print("It is less than!")
elif 2 > 3:
    print("It is greater!")
else:
    print("Equal!")
```

```
File "<ipython-input-102-e5ff2ebb180c>", line 1
    if 2 < 3                                # colon : is missing
                                     ^
```

SyntaxError: invalid syntax

Example: In the code below, indentation is missing in the block statement of the if condition in line 2.

```
In [ ]: if 2 < 3:
        print("It is less than!")    # block statement needs indentation
elif 2 > 3:
    print("It is greater!")
else:
    print("Equal!")
```

```
File "<ipython-input-103-3c294b21dd46>", line 2
    print("It is less than!")    # block statement needs indentation
    ^
```

IndentationError: expected an indented block

Example: When an undeclared variable is used in a statement, it causes error messages. In the code below, `slope` was not defined ahead of the `if` condition.

```
In [ ]: if slope > 0:                # slope is not defined can not be compared to zero
        print('increasing')
    else:
        print('decreasing')
```

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-105-942dc7b20bf4> in <module>()
----> 1 if slope > 0:                # slope is not defined can not be compared to zero
ro
      2     print('increasing')
      3 else:
      4     print('decreasing')

NameError: name 'slope' is not defined
```

4. Python Loops

A loop statement allows us to execute statement(s) multiple times. Two types of loops:

1. The iterative `for` loop
2. The conditional `while` loop

4.1 The `for` Loop

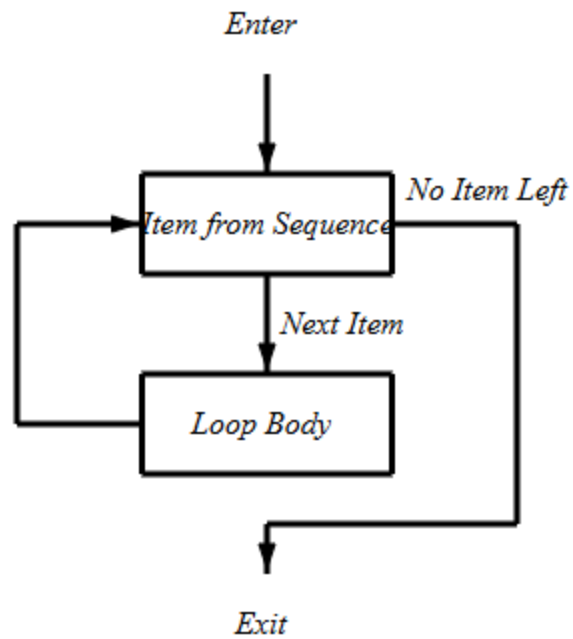
[Watch demo video](#)

The `for` loop can be used to iterate over a sequence such as a list or other iterable objects.

Syntax:

```
for i in sequence_name:    # i will take each value of sequence_name
    block statements
```

For Loop Flowchart



The flow of execution for a `for` statement:

1. Take the first element in the list
2. execute the block statement
3. Go back to step 1
4. Repeat step 1 to 3 until the last element of the list reached

Example: print the values of each element in the list `x=[5, 10, 5, 4]` .

1. Click `+ Code` to add a code-cell
2. Type `x = [5, 10, 5, 4]`
3. Type `for v in x:`
4. Type indented `print(v)`

```
In [ ]: x = [5, 10, 5, 4]
        for v in x:
            print(v)
```

```
5
10
5
4
```

Note: When the program reaches the second line and encounters `for`, it uses `v` to store the first element in `x`. In the third line, it will print the current value of `v`. Then it will go back to the second line and use `v` to store the second element in `x` and prints this updated value of `v`. This pattern repeats till all elements in `x` are reached.

[Watch demo video](#)

Example: Given a list of integers, use the `for` loop to add all values in a list. Same as `sum(x)` from list operators.

```
In [ ]: x = [5, 10, 5, 4]
sum1 = 0
for i in x:
    sum1 = sum1 + i
print("The sum is", sum1)
```

The sum is 24

Note: In the above code, `i` is the variable that takes the value of the elements inside the `x` list on each iteration. The `for` loop continues until the last element in the list is reached. The body of `for` loop is separated from the rest of the code using **indentation**.

Note: The flow of execution:

Begin

- `x = [5, 10, 5, 4]` (line 1)
- `sum1 = 0` (line 2)

Enter `for` loop

- `i=x[0]=5` (line 4)
- `sum1 = sum1 + i = 0 + 5 = 5` . (line 5)

Back to the `for` loop

- `i=x[1]=10` (line 4)
- `sum1 = sum1 + i = 5 + 10 = 15` (line 5)

Back to the `for` loop

- `i=x[2]=5` (line 4)
- `sum1 = sum1 + i = 15 + 5 = 20` (line 5)

Back to the `for` loop

- `i=x[3]=4` (line 4)
- `sum1 = sum1 + i = 20 + 4 = 24` (line 5)

Back to the `for` loop `i=x[4]` no element left in the list

- Exit the `for` loop

Go to line 6

- `print("The sum is", sum1)`.

This could also be done using the `sum(x)` list operator as follows:

```
In [ ]: x = [5,10,5,4]
        sum(x)
```

```
Out[ ]: 24
```

Example: Given a `list` of numbers, find its average. The average is the sum of all the numbers divided by size of the list.

```
In [ ]: x = [2.3,3.5,4,2.6,3.1,1.9]
        sum1 = 0
        for i in x:
            sum1 = sum1 + i
        average = sum1/len(x)
        print("the average is ", average)
```

```
the average is  2.9
```

Note: The total `sum1` is the same as the code above. After the `for` loop in line 4, `average=sum1/len(x)`. `len(x)` is the size or number of elements in `x`.

Example: The ages of current (as of 2020) US Senators are given below. Write a Python program that displays the number of Senators who are older than 65 years.

```
81,81,80,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,70,
70,70,70,68,68,68,68,68,67,67,67,66,66,65,65,65,65,64,64,64,64,64,64,63, 63,
63,63,62,62,62,62,62,61,61,61,60,60,59,59,59,59,59,59,59,59,58,58,57,
57,56,56,55,54,54,54,54,54,53,53,52,52,52,52,52,51,50,49,48,46,45,44,43,43,
43,42,41
```

```
In [ ]: age = [81,81,80,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,
              70,70,70,68,68,68,68,68,67,67,67,66,66,65,65,65,65,64,64,64,64,64,64,63,63,
              63,63,62,62,62,62,62,61,61,61,60,60,59,59,59,59,59,59,59,59,58,58,57,57,
              56,56,55,54,54,54,54,54,53,53,52,52,52,52,52,51,50,49,48,46,45,44,43,43,
              43,42,41]

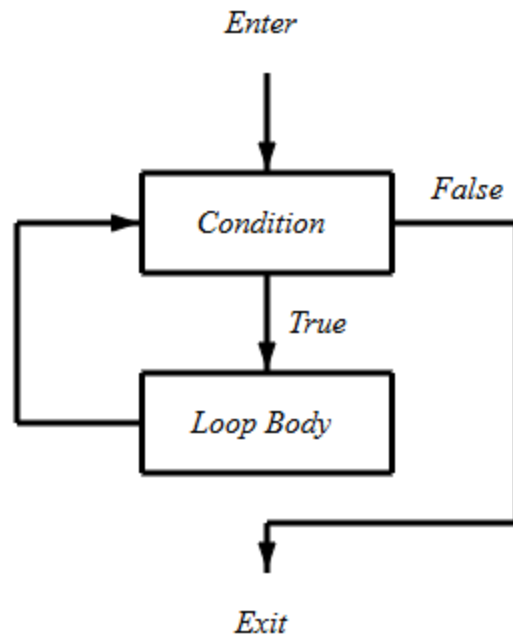
        count = 0
        for i in age:
            if i > 65:
                count = count + 1
        print(count)
        print('Of the',len(age),'Senators,', count,'are over 65 years old')
```

37

Of the 100 Senators, 37 are over 65 years old

Note: `age` and `count` are defined first. In the `for` statement `i` takes the first element of `age` and checks if it is greater than 65. If true `count = 0 + 1` if not true, it will go back to the `for` statement and `i` takes the second element in `age`. The pattern continues till the last element in `age`.

While Loop Flowchart



Syntax

```
while condition:
    block statements
```

The flow of execution for a `while` statement:

1. Evaluate the condition, yielding True or False.
2. If the condition is false, exit the while statement and continue execution at the next statement.
3. If the condition is true, execute the body and then go back to step 1.

Example: Print the first 5 positive integers: 1, 2, 3, 4, and 5.

```
In [ ]: i = 0
while i < 5:
    i = i + 1
    print(i)
```

1
2
3
4
5

Note: In the above code, the flow of execution is:

Begin in line 1

- `i = 0` before the loop

Enter `while` loop

- `i = 0 < 5` is `True` (line 2)
- `i = i + 1 = 0 + 1 = 1` (line 3)
- `print i = 1` (line 4)

Back to the `while` loop

- `i = 1 < 5` is `True` , (line 2)
- `i = i + 1 = 1 + 1 = 2` (line 3)
- `print i = 2` (line 4)

Back to the `while` loop

- `i = 2 < 5` is `True` (line 2)
- `i = i + 1 = 2 + 1 = 3` (line 3)
- `print i = 3` (line 4)

Back to the `while` loop

- `i = 3 < 5` is `True` (line 2)
- `i = i + 1 = 3 + 1 = 4` (line 3)
- `print i = 4` (line 4)

Back to the `while` loop

- `i = 4 < 5` is `True` (line 2)
- `i = i + 1 = 4 + 1 = 5` (line 3)
- `print i = 5` (line 4)

Back to the `while` statement.

- `i = 5 < 5` is `False` Exit the loop

Example: Find the sum of the first 5 positive integers.

```
In [ ]: i = 0
        sum1 = 0
```



```

while i < 5:
    i = i + 1
    sum1 = sum1 + i

print("Total sum:", sum1)

```

Total sum: 15

In the above code:

- `i = 0` and `sum1 = 0` before the loop
- `i = 0 < 5` is true (1st loop)
 $i = i + 1 = 0 + 1 = 1$ and $sum1 = sum1 + i = 0 + 1 = 1$
- `i = 1 < 5` is true (2nd loop) $i = i + 1 = 1 + 1 = 2$ and $sum1 = sum1 + i = 1 + 2 = 3$
- `i = 2 < 5` is true (2nd loop) $i = i + 1 = 2 + 1 = 3$ and $sum1 = sum1 + i = 3 + 3 = 6$
- `i = 3 < 5` is true (3rd loop) $i = i + 1 = 3 + 1 = 4$ and $sum1 = sum1 + i = 6 + 4 = 10$
- `i = 4 < 5` is true (4th loop) $i = i + 1 = 4 + 1 = 5$ and $sum1 = sum1 + i = 10 + 5 = 15$
- `i = 5 < 5` is false exit loop

print sum = 15 (last value of sum)

Note: Fibonacci Numbers: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ... The first two are 1 and 1. The rest are found by adding the two preceding numbers. The list can continue indefinitely.

Example: Write a Python program that lists a user given number of Fibonacci numbers.

```

In [ ]: n = input("How many Fibonacci numbers do you want to list? ")

n = int(n) # convert the default string to integer

i = 0 # initial counter
a = 1 # first number is 1
b = 1 # second number is 1

if n <= 0:
    print("Enter a positive integer")
if n == 1:
    print(a)
else:
    while i < n: # repeat action while i < n is true
        print(a) # print the first number
        i = i + 1 # increase i by 1
        c = a + b # add the previous two numbers and place it in c
        a = b # update the last two numbers
        b = c # update the last two numbers

```

How many Fibonacci numbers do you want to list?10

1
1
2
3
5
8
13
21
34
55

5. Frequency Distributions and Graphs

In statistics, dataset can be summarized in tabular or graphical displays. A frequency distribution can be described as a table that displays several value(s) or categories with their associated frequencies or counts.

5.1 Frequency Distribution

For a unique count of each data value in a dataset use the syntax:

```
import numpy as np
x = [x1,x2,x3,...]
np.unique(x,return_counts=True)
```

Note: Dataset will be entered in Python list as `x = [x1,x2,x3,...]`

Example: The dataset below is the waiting time, in minutes, of customers at a fast food restaurant. Write a program to construct a frequency of unique values.

```
5, 3, 4, 1, 5, 3, 5, 2, 3, 2, 5, 4, 4, 6, 4, 3, 4, 1, 2, 5
```

[Watch demo video](#)

1. Click `+ Code`
2. Type `import numpy as np`
3. Type `x = [5, 3, 4, 1, 5, 3, 5, 2, 3, 2, 5, 4, 4, 6, 4, 3, 4, 1, 2, 5]`
4. Type `np.unique(x,return_counts=True)`
5. Click `play` button
6. Two array output is displayed - unique value and frequency

```
In [ ]: import numpy as np
x = [5, 3, 4, 1, 5, 3, 5, 2, 3, 2, 5, 4, 4, 6, 4, 3, 4, 1, 2, 5]
```

```
np.unique(x, return_counts=True)
```

```
Out[ ]: (array([1, 2, 3, 4, 5, 6]), array([2, 3, 4, 5, 5, 1]))
```

Interpretation: In the above output, the first array is the list of unique elements and the second array is counts (frequencies) for each unique element. For example, the value 1 from the first array has a frequency of 2 from the second array, etc.

value	frequency
1	2
2	3
3	4
4	5
5	5
6	1

Note: We will use the variable name `x` to store the data values in a [Python List](#).

Example: 20 students were asked if they were registered to vote. Their responses are listed below. Write a program to construct a frequency table.

```
NO, NO, YES, YES, NO, YES, NO, NO, YES, NO, YES, NO, NO, YES, YES, YES,  
YES, NO, YES, YES
```

[Watch demo video](#)

```
In [ ]: import numpy as np  
x = ['NO', 'NO', 'YES', 'YES', 'NO', 'YES', 'NO', 'NO', 'YES', 'NO',  
      'YES', 'NO', 'NO', 'YES', 'YES', 'YES', 'YES', 'NO', 'YES', 'YES']  
np.unique(x, return_counts=True)
```

```
Out[ ]: (array(['NO', 'YES'], dtype='<U3'), array([ 9, 11]))
```

Interpretation: from the above output:

value	frequency
NO	9
YES	11

Note: Qualitative data values are entered in 'quotes'. The `dtype='<U3'` refers to a Unicode string of 3 characters.

For making a frequency distribution with k classes (bins), use the following code:

```
import numpy as np
x = [x1,x2,x3,...]
np.histogram(x,bins=k,density=False)
```

If `density=True` the frequencies are replaced by relative frequencies.

Example: The dataset below is test scores of 20 students in a Math class. Write a program to construct a frequency distribution with 5 classes(bins).

```
33, 42, 49, 49, 53, 55, 55, 61, 63, 67, 68, 68, 69, 69, 72, 73, 74, 78, 80,
83, 88, 88, 88, 90, 92, 94, 94, 94, 94, 96, 100
```

[Watch demo video](#)

1. Click `+ Code`
2. Type `import numpy as np`
3. Type `x = [33, 42, 49, 49, 53, 55, 55, 61, 63, 67, 68, 68, 69, 69, 72, 73, 74, 78, 80, 83, 88, 88, 88, 90, 92, 94, 94, 94, 94, 96, 100]`
4. `np.histogram(x, bins=5,density=False)`
5. Two array output - frequencies and classes

```
In [ ]: import numpy as np
x = [33, 42, 49, 49, 53, 55, 55, 61, 63, 67, 68, 68, 69, 69, 72, 73, 74,
      78, 80, 83, 88, 88, 88, 90, 92, 94, 94, 94, 94, 96, 100]
np.histogram(x, bins=5,density=False)
```

```
Out[ ]: (array([ 2,  5,  9,  4, 11]),
        array([ 33. , 46.4, 59.8, 73.2, 86.6, 100. ]))
```

Interpretation from the above output:

classes	freq
33 - 46.4	2
46.4 - 59.8	5
59.8 - 73.2	9
73.2 - 86.6	4
86.6 - 100	11

key: 33-46.4 is `[33,46.4)` or $33 \leq x < 46.4$

Example: The dataset below is test scores of 20 students in a Math class. Write a program to construct a frequency distribution with 5 classes(bins). Save the two arrays as `freq` and `class_limits`

```
33, 42, 49, 49, 53, 55, 55, 61, 63, 67, 68, 68, 69, 69, 72, 73, 74, 78, 80,
83, 88, 88, 88, 90, 92, 94, 94, 94, 94, 96, 100
```

1. Click + Code
2. Type `import numpy as np`
3. Type `x = [33, 42, 49, 49, 53, 55, 55, 61, 63, 67, 68, 68, 69, 69, 72, 73, 74, 78, 80, 83, 88, 88, 88, 90, 92, 94, 94, 94, 94, 96, 100]`
4. Type `freq, class_limit = np.histogram(x, bins=5, density=False)`
5. Type `print(class_limit)`
6. Type `print(freq)`

```
In [ ]: import numpy as np
x = [33, 42, 49, 49, 53, 55, 55, 61, 63, 67, 68, 68, 69, 69, 72, 73, 74,
      78, 80, 83, 88, 88, 88, 90, 92, 94, 94, 94, 94, 96, 100]
freq, class_limit = np.histogram(x, bins=5, density=False)
print(class_limit)
print(freq)
```

```
[ 33.  46.4  59.8  73.2  86.6 100. ]
[ 2  5  9  4 11]
```

Interpretation from the above output:

classes	freq
33 - 46.4	2
46.4 - 59.8	5
59.8 - 73.2	9
73.2 - 86.6	4
86.6 - 100	11

key: 33-46.4 is `[33,46.4)` or $33 \leq x < 46.4$

Example: Use the US Senators age (as of 2021) dataset given below to construct a relative frequency distribution with 5 bins (classes).

```
81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,70,70,
```

```
In [ ]: import numpy as np
x = [81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,
      71,70,70, 70,70,68,68,68,68,68,67,67,67,66,66,65,65,65,65,64,64,64,
      64,64,64,63,63,63,63,62,62,62,62,62,61,61,61,60,60,59,59,59,59,
      59,59,59,58,58,57,57,56,56,55,54,54,54,54,54,53,53,52,52,52,52,52,
      51,50,49,48,46,45,44,43,43,43,42,41]
freq, bin_edges = np.histogram(x, bins=5, density=True)
print(bin_edges)
print(freq)
```

```
[41. 49. 57. 65. 73. 81.]  
[0.01125 0.0225 0.04 0.03 0.02125]
```

Interpretation from the above output:

classes	rel-freq
41 - 49	0.01125
49 - 57	0.0225
57 - 65	0.04
65 - 73	0.03
73 - 81	0.02125

key: 41 - 49 is [41,49)

0.04=4% of the Senators are aged between 75 to 65

5.2 The Matplotlib Library

`Matplotlib.pyplot` is a graph plotting software library in python that serves as a visualization utility. `matplotlib.pyplot` is typically imported with the alias `plt`.

`Matplotlib.pyplot` Functions

Function	Description
<code>plt.bar(x,y)</code>	Bar Graph
<code>plt.pie(y, labels=x)</code>	Pie Chart
<code>plt.hist(x, bins=k)</code>	Histogram with k classes
<code>plt.scatter(x,y)</code>	Scatter Plot
<code>plt.suptitle("Main Title")</code>	Main Title
<code>plt.xlabel("Horizontal Title")</code>	Horizontal Label
<code>plt.ylabel("Vertical Title")</code>	Vertical Label

5.3 Bar Graphs and Pie Charts

Given frequency tables with two columns: categories/values and frequencies:

Syntax for bar graph

```
import matplotlib.pyplot as plt
value = [x1,x2,...]
freq = [y1,y2,...]
plt.bar(value,freq)
```

Syntax for pie chart

```
import matplotlib.pyplot as plt
value = [x1,x2,...]
freq = [y1,y2,...]
plt.pie(freq, labels=value,autopct='%1.1f%%' )
```

Example: 20 students were asked if they were registered to vote. Their responses are given below. Plot a bar graph.

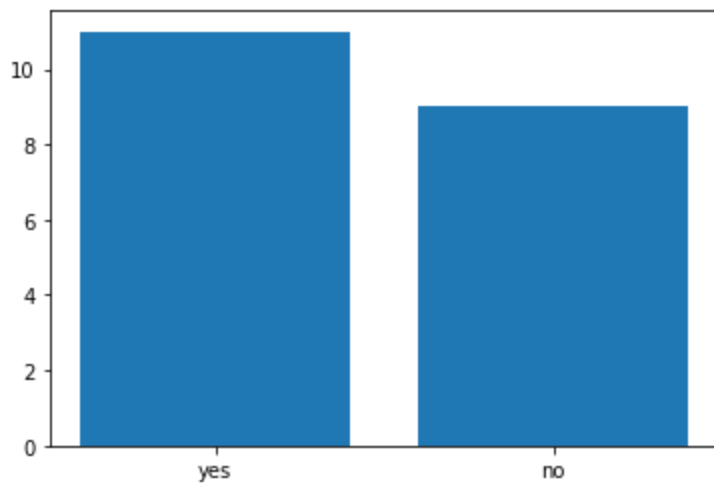
vote	freq
Yes	11
No	9

[Watch demo video](#)

1. Click **+ Code**
2. Type `import matplotlib.pyplot as plt`
3. Type `v = ['Yes','No']`
4. Type `f = [11,9]`
5. Type `plt.bar(v,f)`
6. Click **play** button
7. Right click image to save it to your device

```
In [ ]: import matplotlib.pyplot as plt
v = ['yes','no']
f = [11,9]
plt.bar(v,f)
```

```
Out[ ]: <BarContainer object of 2 artists>
```



Example: 20 students were asked if they were registered to vote. Their responses are given below. Plot a pie chart (percent).

vote	freq
Yes	11
No	9

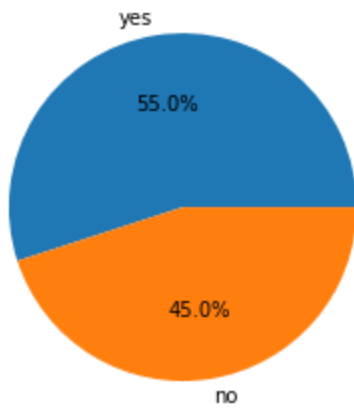
[Watch demo video](#)

1. Click `+ Code`
2. Type `import matplotlib.pyplot as plt`
3. Type `v = ['Yes', 'No']`
4. Type `f = [11,9]`
5. Type `plt.bar(f, labels=f, autopct='%1.1f%%')`
6. Click `play` button
7. Right click on the image to save it to your device

Note: `autopct='%1.1f%%'` changes the frequency to percent rounded to one decimal place, and adds the percent sign %.

```
In [ ]: import matplotlib.pyplot as plt
v = ['yes', 'no']
f = [11,9]
plt.pie(f, labels=v, autopct='%1.1f%%')
```

```
Out[ ]: ([<matplotlib.patches.Wedge at 0x7fcbc206d1d0>,
<matplotlib.patches.Wedge at 0x7fcbc202f3d0>],
[Text(-0.17207795223283848, 1.086457168210212, 'yes'),
Text(0.17207795223283862, -1.086457168210212, 'no')],
[Text(-0.09386070121791189, 0.5926130008419338, '55.0%'),
Text(0.09386070121791196, -0.5926130008419338, '45.0%')])
```

Example: 20 students were asked if they were registered to vote. Their responses are given below. Plot a bar graph.

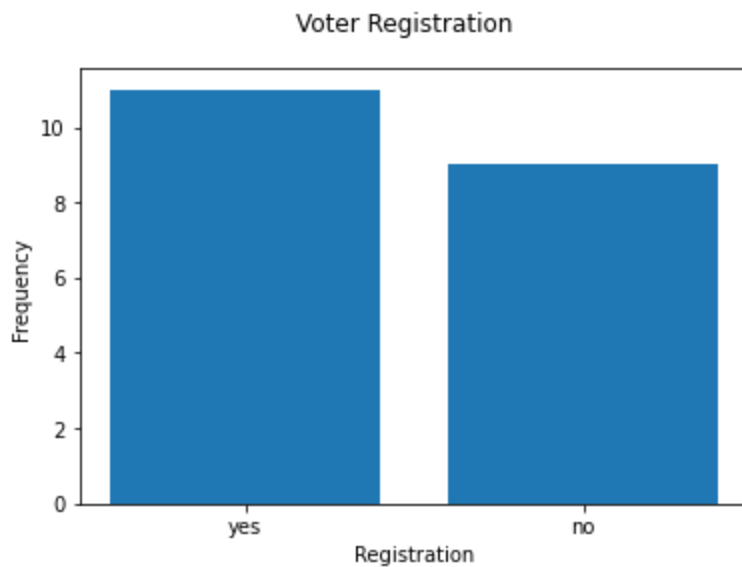
vote	freq
Yes	11
No	9

Add title, vertical and horizontal labels.

1. Click + Code
2. Type `import matplotlib.pyplot as plt`
3. Type `v = ['Yes','No']`
4. Type `f = [11,9]`
5. Type `plt.bar(v,f)`
6. Type `plt.suptitle("Voter Registration")`
7. Type `plt.xlabel("Registration")`
8. Type `plt.ylabel("Frequency")`
9. Click play button

```
In [ ]: import matplotlib.pyplot as plt
v = ['yes','no']
f = [11,9]
plt.bar(v,f)
plt.suptitle("Voter Registration")
plt.xlabel("Registration")
plt.ylabel("Frequency")
```

```
Out[ ]: Text(0, 0.5, 'Frequency')
```



Example: The frequency distribution below is the waiting time, in minutes, of 20 customers at a fast food restaurant.

Waiting_time	Frequency
1	2
2	3
3	4
4	5
5	5
6	1

Plot a bar graph and a pie chart.

Note: First the data values in the two columns are entered separately using Python lists:

`waiting_time` and `frequency` .

1. Click `+ Code`
2. Type `import matplotlib.pyplot as plt`
3. Type `waiting_time = [1,2,3,4,5,6]`
4. Type `frequency = [2,3,4,5,5,1]`
5. Type `plt.bar(waiting_time,frequency)`
6. Type `plt.xlabel("Waiting Time")`
7. Type `plt.ylabel("Frequency")`
8. Click `play` button

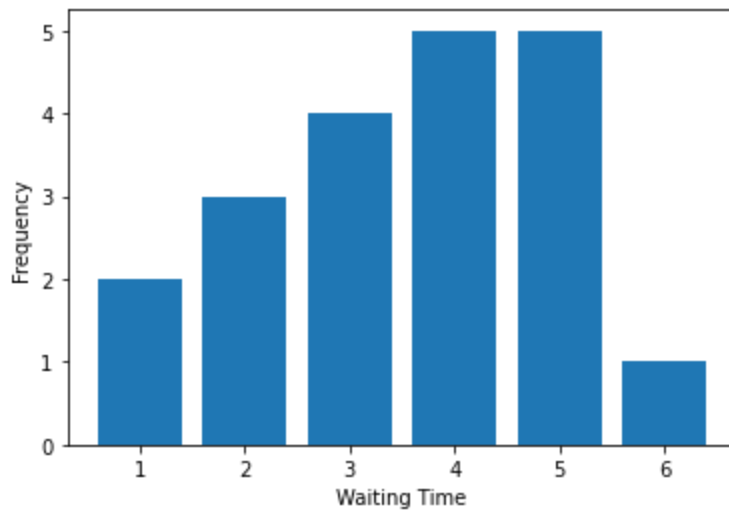
```
In [ ]: import matplotlib.pyplot as plt
```

```
waiting_time = [1,2,3,4,5,6]
frequency = [2,3,4,5,5,1]
```

```
plt.bar(waiting_time,frequency)

plt.xlabel("Waiting Time")
plt.ylabel("Frequency")
```

Out[]: Text(0, 0.5, 'Frequency')



Example: The frequency distribution below is the waiting time, in minutes, of 20 customers at a fast food restaurant.

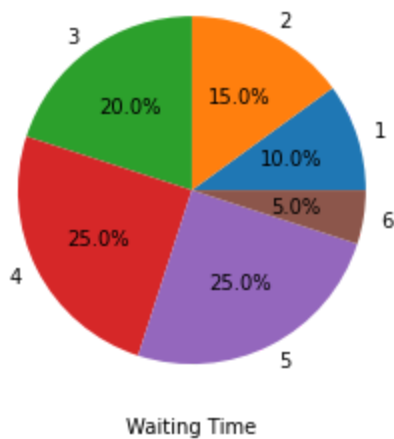
Waiting_time	Frequency
1	2
2	3
3	4
4	5
5	5
6	1

Graph a Pie Chart

```
In [ ]: import matplotlib.pyplot as plt

waiting_time = [1,2,3,4,5,6]
frequency = [2,3,4,5,5,1]
plt.pie(frequency, labels=waiting_time,autopct='%1.1f%%')
plt.xlabel("Waiting Time")
```

Out[]: Text(0.5, 0, 'Waiting Time')



5.4 Histogram, Polygon and Ogive

A histogram uses connected bars to represent the frequency distribution of a continuous dataset.

Syntax for n bins(classes)

```
import matplotlib.pyplot as plt
plt.hist(x,bins=n,ec='k')
```

Example: The dataset below is scores of 20 students in a Math class. Write a program to construct a histogram with 5 classes(bins).

```
33, 42, 49, 49, 53, 55, 55, 61, 63, 67, 68, 68, 69, 69, 72, 73, 74,78, 80,
83, 88, 88, 88, 90, 92, 94, 94, 94, 94, 96, 100
```

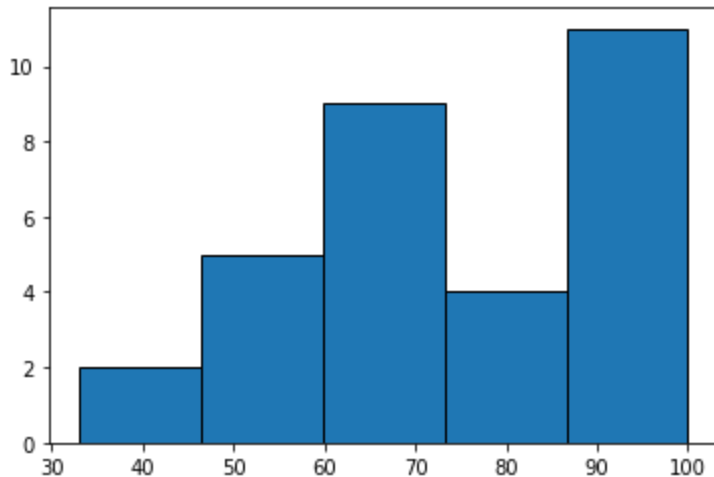
[Deom Video Link](#)

1. Click `+ Code`
2. Type `import matplotlib.pyplot as plt`
3. Type `x = [33, 42, 49, 49, 53, 55, 55, 61, 63, 67, 68, 68, 69, 69, 72, 73, 74,78, 80, 83, 88, 88, 88, 90, 92, 94, 94, 94, 94, 96, 100]`
4. Type `plt.hist(x,bins=5,ec='k')`
5. Click `play` button

Note: `ec='k'` adds solid black lines to the bars.

```
In [ ]: import matplotlib.pyplot as plt
x = [33, 42, 49, 49, 53, 55, 55, 61, 63, 67, 68, 68, 69, 69, 72, 73, 74,78, 80,
      83, 88, 88, 88, 90, 92, 94, 94, 94, 94, 96, 100]
plt.hist(x,bins=5,ec='k')
```

```
Out[ ]: (array([ 2.,  5.,  9.,  4., 11.]),
        array([ 33., 46.4, 59.8, 73.2, 86.6, 100. ]),
        <a list of 5 Patch objects>)
```



Example: The dataset below is scores of 20 students in a Math class. Write a program to construct a histogram with 5 classes(bins). Use class limits for the x-axis.

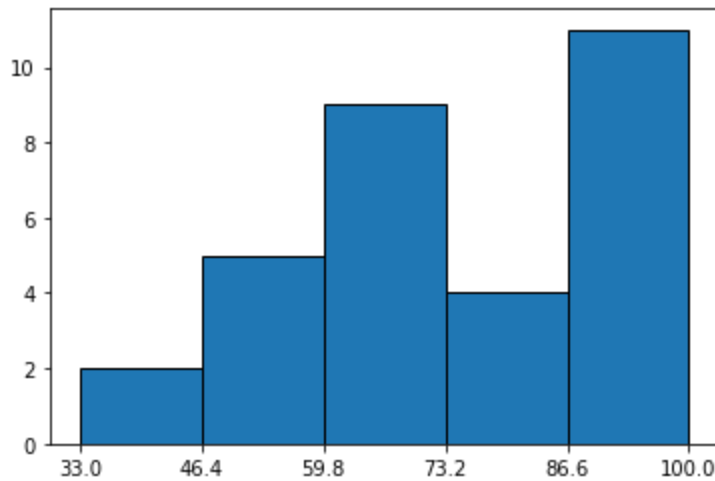
```
33, 42, 49, 49, 53, 55, 55, 61, 63, 67, 68, 68, 69, 69, 72, 73, 74, 78, 80,
83, 88, 88, 88, 90, 92, 94, 94, 94, 94, 96, 100
```

1. Click + Code
2. Type `import matplotlib.pyplot as plt`
3. Type `x = [33, 42, 49, 49, 53, 55, 55, 61, 63, 67, 68, 68, 69, 69, 72, 73, 74, 78, 80, 83, 88, 88, 88, 90, 92, 94, 94, 94, 94, 96, 100]`
4. Type `(freq, class_limit, patches) = plt.hist(x, bins=5)`
5. Type `plt.xticks(class_limit)`
6. Click play button

```
In [ ]: import matplotlib.pyplot as plt
x = [33, 42, 49, 49, 53, 55, 55, 61, 63, 67, 68, 68, 69, 69, 72, 73, 74,
     78, 80, 83, 88, 88, 88, 90, 92, 94, 94, 94, 94, 96, 100]

(freq, class_limit, patches) = plt.hist(x, bins=5, ec='k')
plt.xticks(class_limit)
```

```
Out[ ]: ([<matplotlib.axis.XTick at 0x7fcbbda5b910>,
          <matplotlib.axis.XTick at 0x7fcbbda5b490>,
          <matplotlib.axis.XTick at 0x7fcbbda5b950>,
          <matplotlib.axis.XTick at 0x7fcbbda3bad0>,
          <matplotlib.axis.XTick at 0x7fcbbda3bc50>,
          <matplotlib.axis.XTick at 0x7fcbbda45590>],
          <a list of 6 Text major ticklabel objects>)
```



Example: Histograms from raw data

Use the US Senators age data given below and write a program to:

1. Make a histogram with 5 bins (classes) and age range of 40 to 90
2. Display the frequencies and bins(classes)
3. Add titles and labels

81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,70,70,

In []: `import matplotlib.pyplot as plt`

```
age = [81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,70,70,
       70,70,68,68,68,68,68,67,67,67,66,66,65,65,65,65,64,64,64,64,64,64,63,63,
       63,63,62,62,62,62,62,61,61,61,60,60,59,59,59,59,59,59,59,59,58,58,57,57,
       56,56,55,54,54,54,54,54,53,53,52,52,52,52,52,51,50,49,48,46,45,44,43,43,
       43,42,41]
```

```
(freq, bins, patches) = plt.hist(age,bins=5,edgecolor='white', range=[40,90])
```

```
plt.suptitle("Histogram")
```

```
plt.xlabel("Age")
```

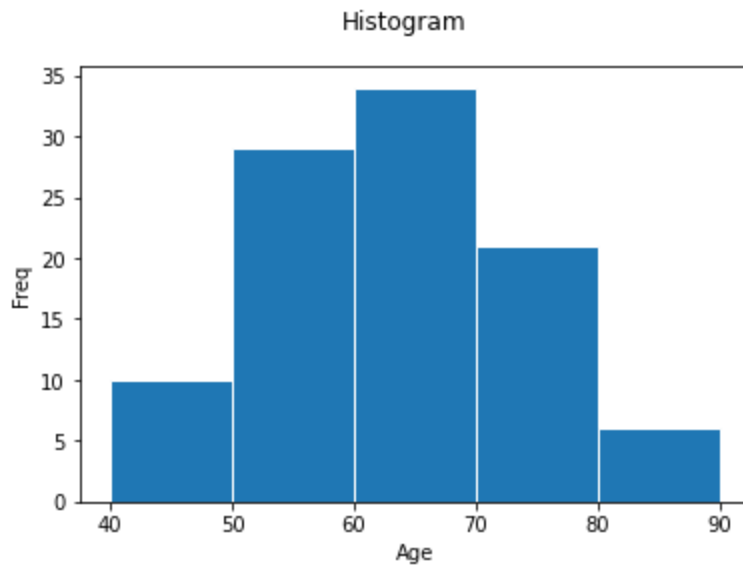
```
plt.ylabel("Freq")
```

```
print("bins", bins)
```

```
print("freq",freq)
```

bins [40. 50. 60. 70. 80. 90.]

freq [10. 29. 34. 21. 6.]



Interpretation from the above output for bins and frequencies:

bins	frequency
40-50	10
50-60	29
60-70	34
70-80	21
80-90	6

key: 40-50 is [40,60)

Frequency Polygon

A frequency polygon is a line graph connecting midpoints of each class/bin.

Example: Use the above example (Senators age) to construct a frequency *polygon*.

We will use the function below to construct a polygon. The arguments (input) to the function are the data values, x , and number of classes(bins), n .

Python Functions

A function is a block of code which only runs when it is called. You can pass data, known as **arguments**, into a function. A function can return data as a result. In Python a function is defined using the `def` keyword.

Syntax

```
def func_name(data):
    statement(s)
    return result # optional
```

When the function `func_name(x)` is called it will return `result` .

To graph a frequency polygon, use the function `polygon(x,n)` given below. `x` is the dataset list and `n` is the number of classes/bins.

```
In [ ]: def polygon(x,n):
import matplotlib.pyplot as plt
import numpy as np

freq, bins, edges = plt.hist(x,bins=n,ec="k",range=[min(x),max(x)+1])
plt.title("Frequency Polygon")
plt.xlabel('Midpoint data values')
plt.ylabel('Frequency')

a1=bins[0]-(bins[1]-bins[0])
a2=bins[-1]+(bins[1]-bins[0])
bin1=np.insert(bins,[0],a1)
bin1=np.append(bin1,a2)

freq,edges,_=plt.hist(x,bins=bin1,ec='k', facecolor='blue', range=[min(x),max(x)+1]
midpoints=0.5*(edges[1:]+edges[:-1])
plt.plot(midpoints,freq)
# plt.xticks(bin1)
plt.xticks(midpoints)
plt.show()
```

Example: Use the the `polygon(x,n)` function above to plot a frequency polygon of the US Senators age data given below with 5 bins(classes).

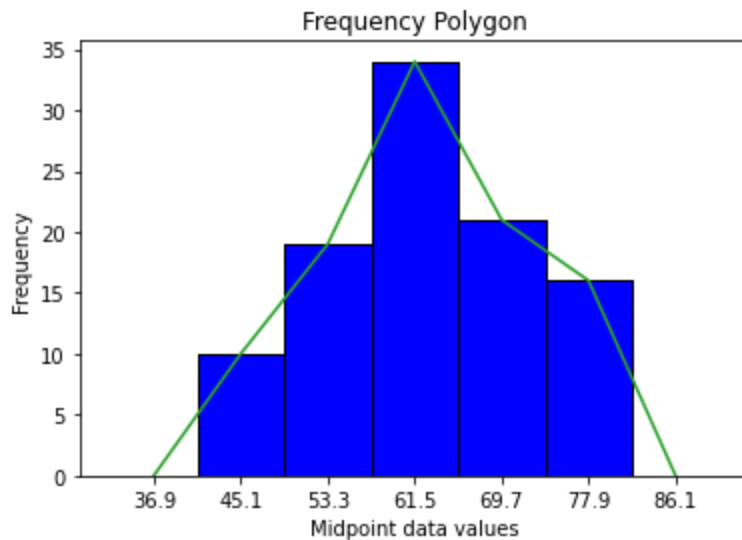
```
81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,70,70,
```

[Watch demo video](#)

1. Click `play` button on the above code `def polygon(x,n)`
2. Type `x =`

```
[81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,70,
```
3. Type `polygon(x,5)`
4. Click on `play` button

```
In [ ]: x = [81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,70,
70,70,68,68,68,68,68,67,67,67,66,66,65,65,65,65,64,64,64,64,64,64,63,63,
63,63,62,62,62,62,62,61,61,61,60,60,59,59,59,59,59,59,59,59,58,58,57,57,
56,56,55,54,54,54,54,54,53,53,52,52,52,52,52,51,50,49,48,46,45,44,43,43,
43,42,41]
polygon(x,5)
```

Ogive

Ogive is a line graph that displays cumulative frequency of each (upper) class. To graph an ogive, use the function `ogive(x,n)` given below. `x` is the dataset list and `n` is the number of classes/bins.

```
In [ ]: def ogive(x,n):
import matplotlib.pyplot as plt
import numpy as np

freq, bins, edges = plt.hist(x,bins=n,ec="k",range=[min(x),max(x)+1])
plt.title("Ogive")
plt.xlabel('data values')
plt.ylabel('Cumulative frequency')

a1=bins[0]-(bins[1]-bins[0])
a2=bins[-1]+(bins[1]-bins[0])
bin1=np.insert(bins,[0],a1)
bin1=np.append(bin1,a2)

freq,edges,_=plt.hist(x,bins=bin1,ec='k', facecolor='blue', range=[min(x),max(x)+1])

ogive1 = edges[:-1]
cumfreq = np.cumsum(freq)
plt.plot(ogive1,cumfreq)
plt.xticks(bin1)
plt.show()
```

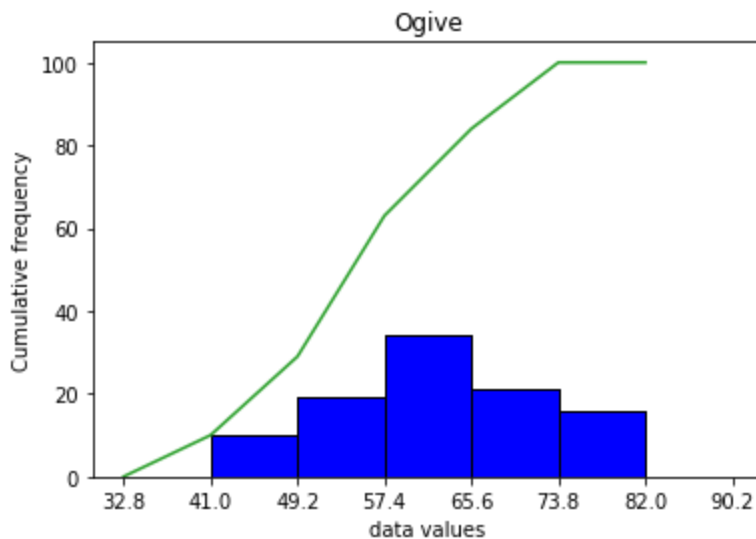
Example: Use the `ogive(x,n)` function above to plot an ogive of the US Senators age data given below with 5 bins(classes).

81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,70,70,

[Watch demo video](#)

1. Click `play` button on the above code `def ogive(x,n)`
2. Type `x =`
`[81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,70,`
`70,70,68,68,68,68,68,67,67,67,66,66,65,65,65,65,64,64,64,64,64,64,63,63,`
`63,63,62,62,62,62,62,61,61,61,60,60,59,59,59,59,59,59,59,59,58,58,57,57,`
`56,56,55,54,54,54,54,54,53,53,52,52,52,52,52,52,51,50,49,48,46,45,44,43,43,`
`43,42,41]`
3. Type `ogive(x,5)`
4. Click on `play` button

```
In [ ]: x = [81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,70,
70,70,68,68,68,68,68,67,67,67,66,66,65,65,65,65,64,64,64,64,64,64,63,63,
63,63,62,62,62,62,62,61,61,61,60,60,59,59,59,59,59,59,59,59,58,58,57,57,
56,56,55,54,54,54,54,54,53,53,52,52,52,52,52,52,51,50,49,48,46,45,44,43,43,
43,42,41]
ogive(x,5)
```



5.5 Dot Plots

Dot plot is like bar graph with dots instead of bars. To graph an dot plot , use the function `dotplot(x)` given below. `x` is the dataset list.

```
In [ ]: # dotplot by Patrick Fitzgerald
def dotplots(x, x_label="Data Values"):
    # takes a x and x_label (by default it is Data Values)

    import numpy as np
    import matplotlib.pyplot as plt

    data = np.array(x) # convert to array
    values, counts = np.unique(data, return_counts=True)

    # dot plot with appropriate figure size and y-axis limits
    fig, ax = plt.subplots(figsize=(6, 2.25))
    for value, count in zip(values, counts):
        ax.plot([value]*count, list(range(count)), 'co', ms=10, linestyle='')
    for spine in ['top', 'right', 'left']:
        ax.spines[spine].set_visible(False)
```

```

ax.yaxis.set_visible(False)
ax.set_ylim(-1, max(counts))
ax.set_xticks(range(min(values), max(values)+1))
ax.tick_params(axis='x', length=0, pad=8, labels=12)

plt.suptitle("Dot Plot")
plt.ylabel("frequency")
plt.xlabel(x_label)

return plt.show()

```

Example: Use the `dotplot(x)` function above to plot a dot plot of the dataset given below.

1, 2, 2, 3, 3, 3, 3, 4, 7, 9

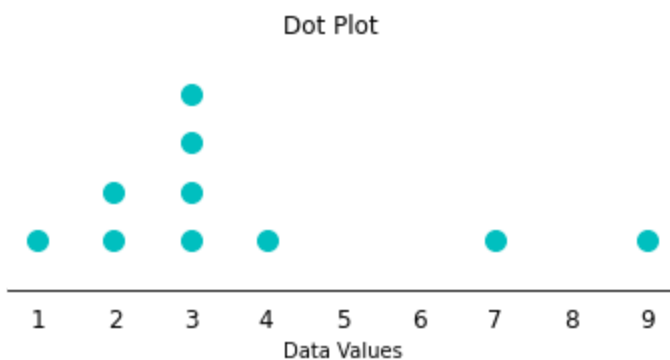
[Watch demo video](#)

1. Click `play` button on the code above `def dotplot(x)`
2. Type `x = [1,2,2,3,3,3,3,4,7,9]`
3. Type `dotplot(x)`
4. Click `play` button

```

In [ ]: x=[1, 2, 2, 3, 3, 3, 3, 4, 7, 9]
dotplots(x)

```



Example: The dataset below is the waiting time, in minutes, of customers at a fast food restaurant. Construct a dot plot.

2, 4, 7, 5, 3, 9, 6, 4, 2, 7, 7, 7, 4, 7, 9

First run(execute) the `dotplots(x)` above.

```

In [ ]: x = [2, 4, 7, 5, 3, 9, 6, 4, 2, 7, 7, 7, 4, 7, 9]
dotplots(x, "Waiting Time")

```



5.6 Stem and leaf Plots

We need to install **Stemgraphic** for plotting stem-and-leaf plot. This is because Stemgraphic is not included in any of the Python Libraries. To install Stemgraphic type the following syntax. [Stemgraphic Modules Link](#)

Note: Stemplots (Stem-and-Leaf plots) should only be used for small datasets.

```
In [ ]: pip install stemgraphic
```

The output generated by the installation process has been deleted above. It is not important.

After stemgraphic is properly installed use the syntax below to plot a stem-and-leaf plot with dataset saved in `x`.

```
import stemgraphic
stemgraphic.stem_graphic(x, scale = 10, aggregation=False)
```

Note: We need to execute(run) `pip install stemgraphic` for every new session to graph stem-and-leaf plot.

Example: The dataset below is exam scores. Construct stem-and-leaf plot.

```
33, 42, 49, 49, 53, 55, 55, 61, 63, 67, 68, 68, 69, 69, 72, 73, 74, 78, 80,
83, 88, 88, 88, 90, 92, 94, 94, 94, 94, 96, 100
```

[Watch demo video](#)

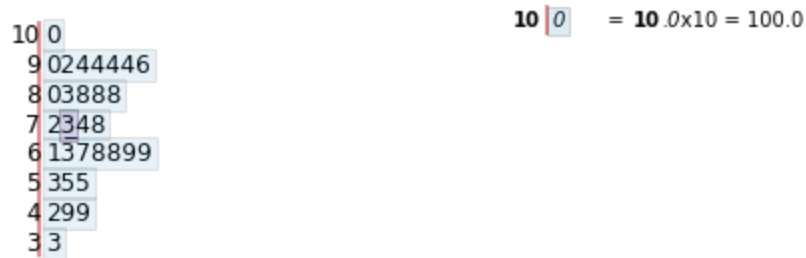
1. Click `+ Code`
2. Type `pip install stemgraphic`
3. Click `play` button
4. Click `+ Code`
5. Type `import stemgraphic`
6. Type `x = [33, 42, 49, 49, 53, 55, 55, 61, 63, 67, 68, 68, 69, 69, 72, 73, 74, 78, 80, 83, 88, 88, 88, 90, 92, 94, 94, 94, 94, 96, 100]`

7. Type `stemgraphic.stem_graphic(x,scale=10,aggregation=False)`

8. Click `play` button

```
In [ ]: import stemgraphic
x = [33, 42, 49, 49, 53, 55, 55, 61, 63, 67, 68, 68, 69, 69, 72, 73, 74,
      78, 80, 83, 88, 88, 88, 90, 92, 94, 94, 94, 94, 96, 100]
stemgraphic.stem_graphic(x,scale=10,aggregation=False)
```

```
Out[ ]: (<Figure size 540x198 with 1 Axes>,
        <matplotlib.axes._axes.Axes at 0x7fcbacfd48d0>)
```



Example: The dataset below is scores of 20 students in a stat class. Construct a stem-and-leaf plot.

```
[81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52,
85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]
```

```
In [ ]: import stemgraphic

x = [81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32,
      52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]

stemgraphic.stem_graphic(x,scale = 10,aggregation=False)
```

```
Out[ ]: (<Figure size 540x198 with 1 Axes>,
        <matplotlib.axes._axes.Axes at 0x7f9c42819fd0>)
```



5.7 Scatter Plots

[Watch demo video](#)

A data set with a pair of quantitative variables can be displayed with a scatter plot.

Example: Age in years and the corresponding height in centimeters is given below. Draw a scatter plot.

age(yrs)	Height(cm)
5	95
6	100
7	105
8	107
9	110
10	115

1. Click `+ Code`
2. Type `import matplotlib.pyplot as plt`
3. Type `x = [5,6,7,8,9,10]`
4. Type `y = [95,100,105,107,110,115]`
5. Type `plt.scatter(x,y)`
6. Type `plt.suptitle('Scatter Plot Age vs Height')`
7. Type `plt.xlabel('Age(yrs)')`
8. Type `plt.ylabel('Height(cm)')`
9. Click `play` button

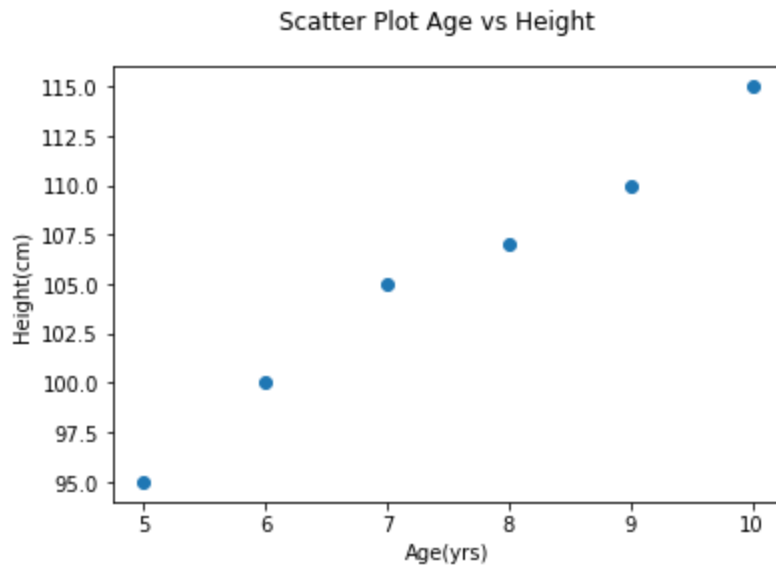
```
In [ ]: import matplotlib.pyplot as plt

x = [5,6,7,8,9,10]
y = [95,100,105,107,110,115]

plt.scatter(x,y)

plt.suptitle('Scatter Plot Age vs Height')
plt.xlabel('Age(yrs)')
plt.ylabel('Height(cm)')
```

```
Out[ ]: Text(0, 0.5, 'Height(cm)')
```



6. Measures of Center and Variation

[Watch demo video](#)

A numerical value that is representative of the dataset located at the "center". The three most widely used measures of the center of a data are the mean (average), the median and mode.

6.1 Mean and Median

The Mean

The sample mean of a data set $x_1, x_2, x_3, \dots, x_n$ is $\bar{x} = \frac{\sum x_i}{n}$

The population mean of the a set $x_1, x_2, x_3, \dots, x_N$ is $\mu = \frac{\sum x_i}{N}$

numpy syntax for the mean:

```
import numpy as np
x = [x1,x2,x3,...,xn]
np.mean(x)
```

Example: find the mean of the data set `1,2,3` using `numpy.mean()` .

1. Click `+ Code`
2. Type `import numpy as np`
3. Type `x = [1,2,3]`
4. Type `np.mean(x)`

5. Click `play` button

```
In [ ]: import numpy as np
x = [1,2,3]
np.mean(x)
```

Out[]: 2.0

Interpretation: the mean is 2.0

The Median

The median of a data set is the middle number after the data is ordered from smallest to highest.

numpy syntax for median:

```
import numpy as np
x = [x1,x2,x3,...,xn]
np.median(x)
```

Example: find the median of the data set `1,2,3,4`

```
In [ ]: import numpy as np
x = [1,2,3,4]
np.median(x)
```

Out[]: 2.5

Interpretation: the median is 2.5

Example: A random sample of 35 annual salaries of Animal Control employees of the City of Chicago is listed below. Write a program to compute the mean, mode and median salaries.

```
45288, 96096, 69468, 79872, 92520, 49968, 76248, 110076, 84324, 101496,
100668, 66864, 49968, 101496, 49968, 69468, 69468, 96096, 100716, 73380,
52044, 76248, 66864, 66864, 76848, 66864, 139392, 56280, 69468,
66864,73380, 87564, 49968, 49968, 76848
```

```
In [ ]: import numpy as np

x= [45288, 96096, 69468, 79872, 92520, 49968, 76248, 110076, 84324, 101496,
    100668, 66864, 49968, 101496, 49968, 69468, 69468, 96096, 100716, 73380,
    52044, 76248, 66864, 66864, 76848, 66864, 139392, 56280, 69468, 66864,73380,
    87564, 49968, 49968, 76848]

print("the mean and median using numpy")
print(np.mean(x))
print(np.median(x))
```



```
the mean and median using numpy
75968.91428571429
73380.0
```

Interpretation: The mean is 75968.914286 and the median is 73380.0.

Mode

For mode use `np.unique(x,return_counts=True)` to display each value with its frequency. Select values with highest frequencies.

Example: What is the mode of `x = [1,1,2,2,2,3,3,5,5,5]`

```
In [ ]: import numpy as np
x = [1,1,2,2,2,3,3,5,5,5]
np.unique(x,return_counts=True)
```

```
Out[ ]: (array([1, 2, 3, 5]), array([2, 3, 2, 3]))
```

Interpretation: From the frequency array ([2, 3, 2, 3]), the second and fourth values have the highest frequency 3. Thus, the mode are 2 and 5.

Note: Mode using pandas library is covered in advanced topics [here](#).

6.2 The Variance

[Watch demo video](#)

How spread out are the data values from the "center"? The most common measures of variation, or spread, of data are the standard deviation and variance.

The variance of a data set is the average of the squared deviations(distances) of the data values from the mean.

1. The sample variance of the data set $x_1, x_2, x_3, \dots, x_n$ is given by $s^2 = \frac{\sum(x_i - \mu)^2}{n-1}$
2. The population variance of the data set $x_1, x_2, x_3, \dots, x_N$ is given by $\sigma^2 = \frac{\sum(x_i - \mu)^2}{N}$

numpy syntax for sample variance:

```
import numpy as np
x = [x1,x2,x3,...]
np.var(x,ddof=1)    is (n-ddof)
```

numpy syntax for population variance:

```
import numpy as np
x = [x1,x2,x3,...]
np.var(x)
```

Note: `ddof` degrees of freedom for numerator is 1 for sample and 0 for population variance.
`ddof=0` by default in numpy.

Example: find the sample variance of the dataset `1,2,3,4,5`

1. Click `+ Code`
2. Type `import numpy as np`
3. Type `x = [1,2,3,4,5]`
4. Type `np.var(x,ddof=1)`
5. Click `play` button

```
In [ ]: import numpy as np
x = [1,2,3,4,5]
np.var(x,ddof=1)
```

Out[]: 2.5

Interpretation: The sample variance is 2.5

Example: find the population variance of the data set `1,2,3,4,5`

```
In [ ]: import numpy as np
x = [1,2,3,4,5]
np.var(x)      #in numpy ddof=0 by default
```

Out[]: 2.0

Interpretation: The population variance is 2.0

Example: The salaries for the eight Board of Ethics workers for the City of Chicago are listed below. Write a `numpy` program to calculate the **population variance**.

`139740, 100716, 80844, 124056, 79812, 87564, 96096, 96096`

```
In [ ]: import numpy as np
x = [139740, 100716, 80844, 124056, 79812, 87564, 96096, 96096]

np.var(x,ddof=0)
```

Out[]: 389385609.75

Interpretation: The population variance for the given salaries is 389385609.75

6.3 The Standard Deviation

The standard deviation is the square root of the variance.

1. The sample variance of the data set $x_1, x_2, x_3, \dots, x_n$ is given by $s = \sqrt{\frac{\sum (x_i - \mu)^2}{n-1}}$

numpy for sample standard deviation:

```
import numpy as np
x = [x1,x2,x3,...]
np.std(x,ddof=1)
```

2. The population variance of the data set $x_1, x_2, x_3, \dots, x_N$ is given by $\sigma = \sqrt{\frac{\sum (x_i - \mu)^2}{N}}$

numpy for population standard deviation:

```
import numpy as np
x = [x1,x2,x3,...]
np.std(x)
```

Example: find the sample standard deviation (std) of the data set 1,2,3,4,5

```
In [ ]: import numpy as np
x = [1,2,3,4,5]
np.std(x,ddof=1)
```

```
Out[ ]: 1.5811388300841898
```

Interpretation: The sample standard deviation is 1.5811388300841898

Example: find the population standard deviation of the data set 1,2,3,4,5

```
In [ ]: import numpy as np
x = [1,2,3,4,5]
np.std(x)
```

```
Out[ ]: 1.4142135623730951
```

Interpretation: The population standard deviation is 1.4142135623730951

Example: The 2010 life expectancy of the 77 Chicago community areas are listed below. Write a `numpy` program to compute the **population standard deviation**.

```
77.3, 80.3, 76, 80.5, 81.5, 81.9, 81.3, 85.2, 81, 80.9, 79.7, 83.4, 84.4,
80.6, 80.3, 79.6, 79.8, 79.6, 79.5, 80.5, 79.8, 80.3, 74.5, 79.5, 71.9,
68.8, 71.7, 78.9, 72.1, 82.2, 81.4, 85, 80.6, 81.9, 74.1, 73.6, 69.8, 74.1,
80.5, 68.9, 82.3, 74.5, 72.5, 74, 74.7, 72.7, 71.9, 77.1, 72.9, 76.4, 76.2,
78.4, 71.6, 76.4, 77.1, 79.8, 79.5, 80.8, 80.4, 80.2, 74.4, 81, 79.2, 77.5,
80.5, 75.2, 70.1, 70.7, 71.1, 78.2, 72.6, 80.5, 74.9, 79.6, 75.3, 82.2,
79.8, 77.8
```

```
In [ ]: import numpy as np
x = [77.3, 80.3, 76, 80.5, 81.5, 81.9, 81.3, 85.2, 81, 80.9, 79.7, 83.4, 84.4,
      80.6, 80.3, 79.6, 79.8, 79.6, 79.5, 80.5, 79.8, 80.3, 74.5, 79.5, 71.9,
      68.8, 71.7, 78.9, 72.1, 82.2, 81.4, 85, 80.6, 81.9, 74.1, 73.6, 69.8, 74.1,
      80.5, 68.9, 82.3, 74.5, 72.5, 74, 74.7, 72.7, 71.9, 77.1, 72.9, 76.4, 76.2,
      78.4, 71.6, 76.4, 77.1, 79.8, 79.5, 80.8, 80.4, 80.2, 74.4, 81, 79.2, 77.5,
      80.5, 75.2, 70.1, 70.7, 71.1, 78.2, 72.6, 80.5, 74.9, 79.6, 75.3, 82.2,
      79.8, 77.8]

np.std(x, ddof=0)
```

Out[]: 4.036198185451496

Example: A random sample of 35 annual salaries of Animal Control employees of the City of Chicago is listed below. Write a `numpy` program to compute the sample variance and standard deviation.

```
45288, 96096, 69468, 79872, 92520, 49968, 76248, 110076, 84324, 101496,
100668, 66864, 49968, 101496, 49968, 69468, 69468, 96096, 100716, 73380,
52044, 76248, 66864, 66864, 76848, 66864, 139392, 56280, 69468,
66864, 73380, 87564, 49968, 49968, 76848
```

```
In [ ]: import numpy as np

x = [45288, 96096, 69468, 79872, 92520, 49968, 76248, 110076, 84324, 101496,
      100668, 66864, 49968, 101496, 49968, 69468, 69468, 96096, 100716, 73380,
      52044, 76248, 66864, 66864, 76848, 66864, 139392, 56280, 69468, 66864,
      73380, 87564, 49968, 49968, 76848]

print('The sample variance and standard deviation are')
print(np.var(x, ddof=1))
print(np.std(x, ddof=1))
```

The sample variance and standard deviation are
438099876.7865547
20930.835549173727

Interpretation: The sample variance and standard deviation are 438099876.7865547 and 20930.835549173727 respectively.

Interpretation: The population standard deviation is 4.036198185451496

Weighted Mean and std

[Watch demo video](#)

numpy syntax for weighted mean and std:

```
import numpy as np
x = [x1, x2, x3, ...]
```

```
w = [w1,w2,w3,...]
```

```
np.average(x,weights=w)
```

numpy sample standard deviation:

```
import numpy as np  
(np.cov(x,aweights=w))**(1/2)
```

Example: At the end of a term a student received a B in Biology (3 credits), A in History (3 credits), B in English Literature (4 credits), and C in Mathematics (5 credits). Find the students grade point average (GPA). Note: A = 4, B = 3, C = 2, D = 1 F = 0 grade points). GPAs are weighted mean. Find the weighted standard-deviation.

```
In [ ]: import numpy as np  
x = [3,4,3,2]  
w = [3,3,4,5]  
  
np.average(x,weights=w)
```

```
Out[ ]: 2.8666666666666667
```

```
In [ ]: (np.cov(x,aweights=w))**.5
```

```
Out[ ]: 0.8359396992145376
```

Interpretation: The grade point average (GPA) is 2.87 with standard deviation 0.84

Mean and Standard deviation of Grouped Frequency Tables

Sometimes instead of the raw data, we are given a summary frequency table.

Example: Below is a frequency table of quiz scores from 26 students. Find the mean and standard deviation. In this case, we take the midpoint of each class to represent the class score.

class	midpoint	freq
0-2	1	1
3-5	4	6
6-8	7	10
9-11	10	7
12-14	13	0
15-17	16	2

numpy syntax for mean and std with frequency as weight:

```
import numpy as np
np.average(x,weights=freq)
```

numpy sample standard deviation:

```
import numpy as np
(np.cov(x,fweights=freq),ddof=1)**(1/2)
```

Example: Below is a frequency table of quiz scores from 26 students. Find the mean and standard deviation. In this case, we take the midpoint of each class to represent the class score.

class	midpoint	freq
0-2	1	1
3-5	4	6
6-8	7	10
9-11	10	7
12-14	13	0
15-17	16	2

1. Click + Code
2. Type `import numpy as np`
3. Type `mid_points = [1,4,7,10,13,16]`
4. Type `freq = [1,6,10,7,0,2]`
5. Type `np.average(mid_points,weights=freq)`
6. Click play button

```
In [ ]: import numpy as np
mid_points = [1,4,7,10,13,16]
freq = [1,6,10,7,0,2]
np.average(mid_points,weights=freq)
```

```
Out[ ]: 7.576923076923077
```

Interpretation: the mean is 7.576923076923077

```
In [ ]: import numpy as np
mid_points = [1,4,7,10,13,16]
freq = [1,6,10,7,0,2]
(np.cov(mid_points,fweights=freq,ddof=1))**.5
```

```
Out[ ]: 3.5005494074282333
```

Interpretation: the sample standard deviation is 3.5005494074282333

```
In [ ]: import numpy as np

mid_points = [1,4,7,10,13,16]
freq = [1,6,10,7,0,2]
np.sqrt(np.cov(mid_points,fweights=freq,ddof=0))
```

Out[]: 3.432571103225427

Interpretation: The population standard deviation is 3.432571103225427

7. Measures of Position

Measure of the relative location of a data value in the dataset.

7.1 Percentiles (Quantiles)

[Watch demo video](#)

The p^{th} percentile is the value that is greater than (or equal to) p percent of the ordered data.

- 1st quartile is equal to 25th percentile
- 2nd quartile is equal to 50th percentile
- 3rd quartile is equal to 75th percentile

In Python percentiles are called **quantiles**

numpy syntax for percentile

```
import numpy as np
np.quantile(x,p)
```

Note: p is the percentile in decimals. For example, the 70th percentile has p value of 0.70, for 1st Quartile which is the same as 25th Percentile, $p = .25$

To display the 25th, 50th and 75th percentiles in one line, use the syntax

```
np.quantile(x,[.25,0.5,0.75])
```

1. Click **+ Code**
2. Type `import numpy as np`
3. Type `x=[81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]`

4. Type `np.quantile(x,.7)`

5. Click `play` button

```
In [ ]: import numpy as np
x=[81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32,
    52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]
np.quantile(x,.7)
```

```
Out[ ]: 50.599999999999994
```

Interpretation: The 70th percentile is approximately 50.6

Example: The dataset below is scores of 20 students in a Math class.

81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52, 85,
28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19

1. Find the 70th percentile.
2. Find the 83rd percentile.
3. Find the 1st quartile.
4. Find the 3rd quartile.

Note: These results could be different from other results that use a different method of computing percentiles.

```
In [ ]: import numpy as np
x=[81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32,
    52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]
np.quantile(x,[.70,.83,.25,.75])
```

```
Out[ ]: array([50.6 , 79.07, 25.25, 52.75])
```

Interpretation:

Percentile	score
70th	50.60
83rd	79.07
25th	25.25
75th	52.75

Example: The dataset below is scores of 20 students in a Math class.

81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52, 85,
28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19

- Find the Interquartile range.

Note: These results could be different from other results that use a different method of computing percentiles.

```
In [ ]: import numpy as np
x=[81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32,
    52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]
np.quantile(x,[.70,.83,.25,.75])
IQR = np.quantile(x,.75)-np.quantile(x,.25)    # IQR = Q3 - Q1
IQR
```

Out[]: 27.5

Example: A random sample of 35 annual salaries of Animal Control employees of the City of Chicago is listed below. Write a `numpy` program to compute the 90th percentile, 2nd quartile and 10th percentile.

```
45288, 96096, 69468, 79872, 92520, 49968, 76248, 110076, 84324, 101496,
100668, 66864, 49968, 101496, 49968, 69468, 69468, 96096, 100716, 73380,
52044, 76248, 66864, 66864, 76848, 66864, 139392, 56280, 69468,
66864,73380, 87564, 49968, 49968, 76848
```

```
In [ ]: import numpy as np

x = [45288, 96096, 69468, 79872, 92520, 49968, 76248, 110076, 84324, 101496,
    100668, 66864, 49968, 101496, 49968, 69468, 69468, 96096, 100716, 73380,
    52044, 76248, 66864, 66864, 76848, 66864, 139392, 56280, 69468, 66864,
    73380, 87564, 49968, 49968, 76848]

np.quantile(x,[0.90,0.50,0.10])
```

Out[]: array([101184., 73380., 49968.])

Percentile of a Value in a Data Set

In a given dataset, what is the percentile (rank) of a given data value? That is, how many or what percent of the data values are below or at the given data value.

For this, we need to import the stats function from the SciPy library.

7.2 The SciPy Library

SciPy (pronounced "Sigh Pie") is a Python-based ecosystem of open-source software library for mathematics, science, and engineering. In addition, `stats` is a module in `scipy` library given as `scipy.stats`.

The `scipy.stats` module This module from SciPy contains a large number of *probability distributions* as well as a growing library of *statistical functions*. We will use it extensively in our

probability and inferential statistics sections later.

stats syntax for percentile rank

```
from scipy import stats
stats.percentileofscore(x, data_value, kind='rank')
```

Example: The dataset below is scores of 20 students in a stats class. Find the percentile rank of the score 50 .

```
81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52, 85,
28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19
```

[Watch demo video](#)

```
In [ ]: from scipy import stats

x = [81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40,
      32, 52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]

stats.percentileofscore(x, 50, kind='rank')
```

Out[]: 70.0

Interpretation: A student with an exam score of 50 is in the 70th percentile. This student's score is at or better than 70 percent of the students. Or, 70% of the scores are at or below 50.

Example: Below is the list of ages of US Senators (2020). Write a program to compute the percentile age rank of a Senator aged 50. [Watch demo video](#)

```
81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70
,70,
70,70,68,68,68,68,68,67,67,67,66,66,65,65,65,65,64,64,64,64,64,64,63,63,
63,63,62,62,62,62,62,61,61,61,60,60,59,59,59,59,59,59,59,59,58,58,57,57,
56,56,55,54,54,54,54,54,53,53,52,52,52,52,52,51,50,49,48,46,45,44,43,43,
43,42,41
```

```
In [ ]: import numpy as np
from scipy import stats

age = [81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,70,
       70,70,68,68,68,68,68,67,67,67,66,66,65,65,65,65,64,64,64,64,64,64,63,63,
       63,63,62,62,62,62,62,61,61,61,60,60,59,59,59,59,59,59,59,59,58,58,57,57,
       56,56,55,54,54,54,54,54,53,53,52,52,52,52,52,51,50,49,48,46,45,44,43,43,
       43,42,41]

age_array = np.array(age)  # convert the list to a numpy array

stats.percentileofscore(age_array, 50, kind='rank')
```

Out[]: 11.0

Interpretation: The percentile age rank of a Senator aged 50 is 11.

Min and Max

To find the minimum and maximum values of a data set, you can use the list method or numpy as follows:

```
x = [x1, x2, x3, ...]

max(x)          # list max x is a Python list
min(x)          # list min

np.amax(x)      #Numpy max x is numpy array
np.amin(x)      #Numpy min
```

Example: The dataset below is scores of 20 students in a stat class. Find the minimum and maximum scores.

```
81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52, 85,
28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19
```

```
In [ ]: import numpy as np
x = [81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40,
      32, 52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]

print('min is', np.amin(x))
print('max is', np.amax(x))
```

```
min is 13
max is 87
```

Example: Below is the list of ages of US Senators (2020). Write a `pandas` program to compute the minimum, 1st quartile, 2nd quartile, 3rd quartile and maximum age.

```
81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70
,70,
70,70,68,68,68,68,68,67,67,67,66,66,65,65,65,65,64,64,64,64,64,64,63,63,
63,63,62,62,62,62,62,61,61,61,60,60,59,59,59,59,59,59,59,59,58,58,57,57,
56,56,55,54,54,54,54,54,53,53,52,52,52,52,52,51,50,49,48,46,45,44,43,43,
43,42,41
```

```
In [ ]: import numpy as n
x = [81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,70,
      70,70,68,68,68,68,68,67,67,67,66,66,65,65,65,65,64,64,64,64,64,64,63,63,
      63,63,62,62,62,62,62,61,61,61,60,60,59,59,59,59,59,59,59,59,58,58,57,57,
      56,56,55,54,54,54,54,54,53,53,52,52,52,52,52,51,50,49,48,46,45,44,43,43,
```

```

    43,42,41]
print('the min age is', np.min(x))
print('the max age is', np.max(x))
print('the first quartile is',np.quantile(x,.25))
print('the second quartile is',np.quantile(x,.5))
print('the third quartile is',np.quantile(x,.75))

```

```

the min age is 41
the max age is 81
the first quartile is 55.75
the second quartile is 63.0
the third quartile is 70.0

```

Interpretation: the minimum, 1st quartile, 2nd quartile, 3rd quartile and maximum age are 41, 55.75, 63, 70 and 81 respectively.

Interpretation: The minimum score is 13 and the maximum score is 87

7.3 Box Plot

[Watch demo video](#)

Box plots (also called box-and-whisker plots or box-whisker plots) give a good graphical image of the concentration of the data. A box plot is constructed from five values: the minimum value, the first quartile, the median, the third quartile, and the maximum value.

Syntax

```

import matplotlib.pyplot as plt
plt.boxplot(x)

```

Example: The dataset below is scores of 20 students in a stat class. Construct a box plot.

```

81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52, 85,
28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19

```

```

In [ ]: import matplotlib.pyplot as plt

x = [81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40,
      32, 52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]

plt.boxplot(x)

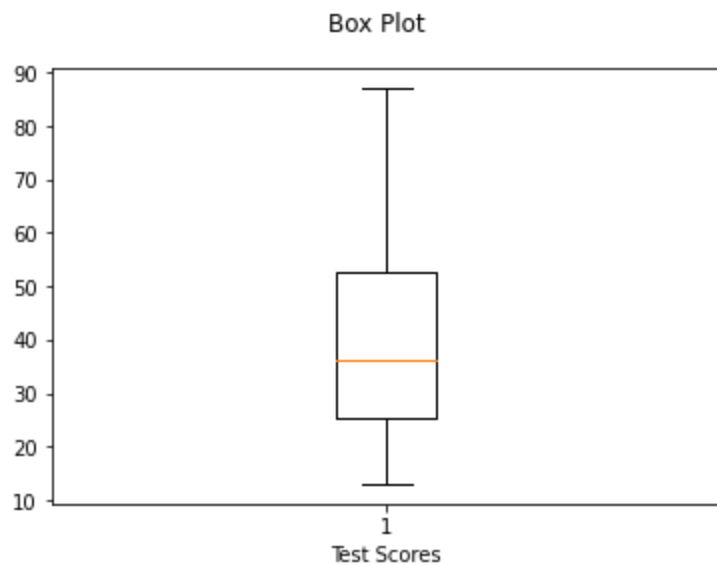
plt.suptitle("Box Plot")
plt.xlabel("Test Scores")

```

```

Out[ ]: Text(0.5, 0, 'Test Scores')

```



Example: The dataset below is scores of 20 students in a stat class. Construct a **horizontal** box plot. Horizontal box plot is useful for identifying the shape of the distribution.

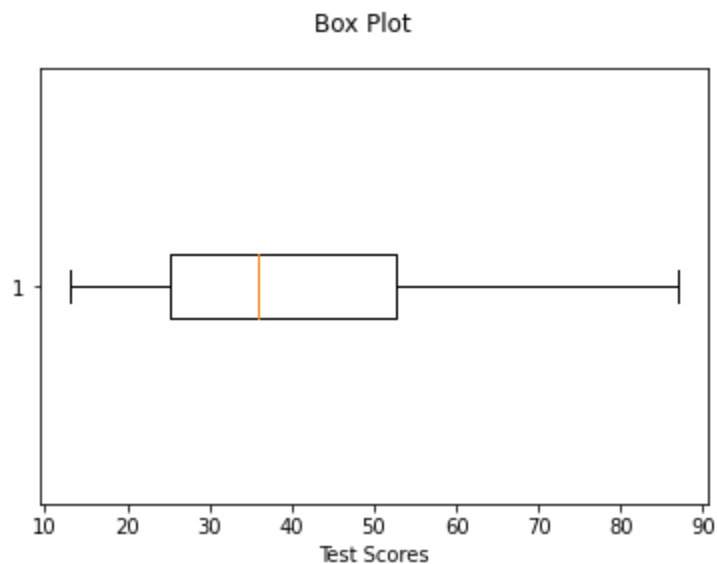
81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19

```
In [ ]: import matplotlib.pyplot as plt

x = [81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40,
      32, 52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]

plt.boxplot(x,vert=False) # add vert=False
plt.suptitle("Box Plot")
plt.xlabel("Test Scores")
```

Out[]: Text(0.5, 0, 'Test Scores')



Example: Below is the list of ages of US Senators (2020). Write a program to draw a box plot.

```
81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,70,70,70,68,68,68,68,68,67,67,67,66,66,65,65,65,65,64,64,64,64,64,64,63,63,63,63,62,62,62,62,62,62,61,61,61,60,60,59,59,59,59,59,59,59,59,59,59,58,58,57,57,56,56,55,54,54,54,54,54,53,53,52,52,52,52,52,52,51,50,49,48,46,45,44,43,43,43,42,41
```

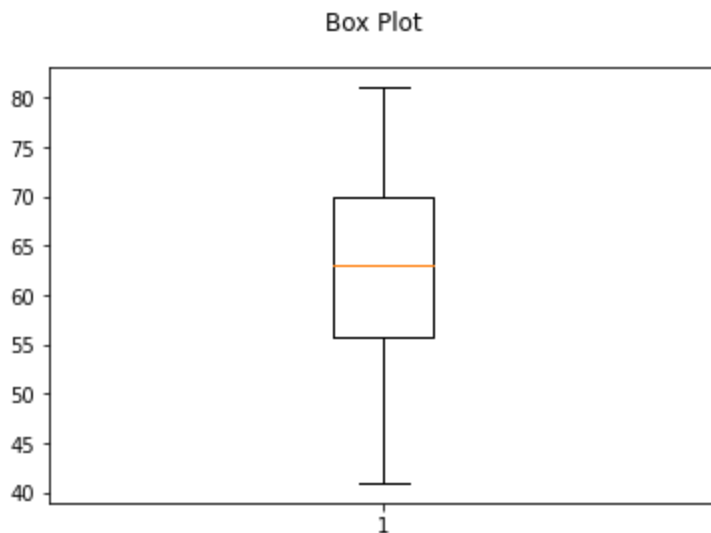
```
In [ ]: import matplotlib.pyplot as plt
```

```
x = [81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,70,70,70,68,68,68,68,68,67,67,67,66,66,65,65,65,65,64,64,64,64,64,64,63,63,63,63,62,62,62,62,62,62,61,61,61,60,60,59,59,59,59,59,59,59,59,59,59,58,58,57,57,56,56,55,54,54,54,54,54,53,53,52,52,52,52,52,52,51,50,49,48,46,45,44,43,43,43,42,41]

plt.boxplot(x)

plt.suptitle("Box Plot")
```

```
Out[ ]: Text(0.5, 0.98, 'Box Plot')
```



```
In [ ]: import matplotlib.pyplot as plt
```

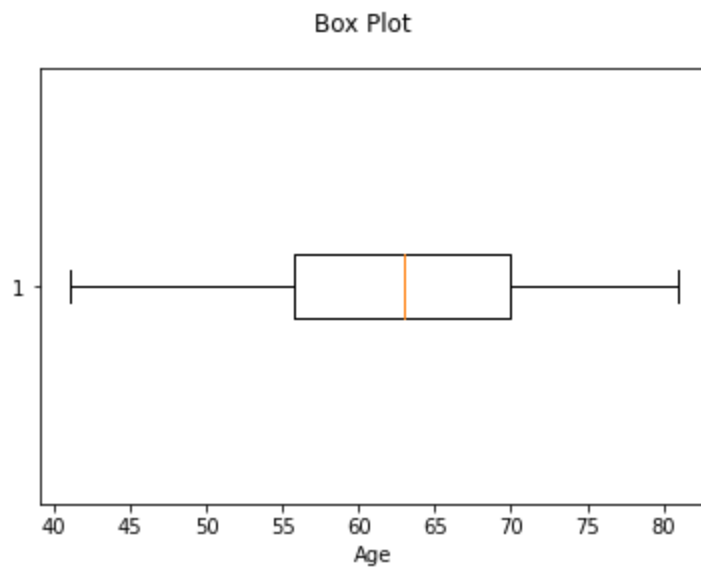
```
x = [81,81,80,80,80,80,78,78,78,77,77,75,75,74,74,74,73,72,72,71,71,71,71,70,70,70,70,68,68,68,68,68,67,67,67,66,66,65,65,65,65,64,64,64,64,64,64,63,63,63,63,62,62,62,62,62,62,61,61,61,60,60,59,59,59,59,59,59,59,59,59,59,58,58,57,57,56,56,55,54,54,54,54,54,53,53,52,52,52,52,52,52,51,50,49,48,46,45,44,43,43,43,42,41]

plt.boxplot(x,vert=False)

plt.suptitle("Box Plot")
```

```
plt.xlabel("Age")
```

```
Out[ ]: Text(0.5, 0, 'Age')
```



Outliers

Example: For the data below, construct a **horizontal** box plot to identify any outliers.

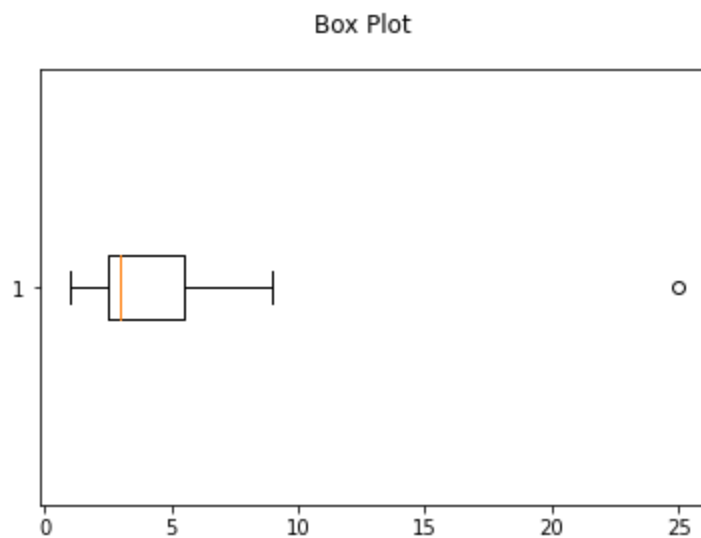
```
1, 2, 2, 3, 3, 3, 3, 4, 7, 9, 25
```

```
In [ ]: import matplotlib.pyplot as plt

x=[1, 2, 2, 3, 3, 3, 3, 4, 7, 9, 25]

plt.boxplot(x,vert=False)
plt.suptitle("Box Plot")
```

```
Out[ ]: Text(0.5, 0.98, 'Box Plot')
```



25 is identified as an outlier in the boxplot above marked with **o** . Any value below the left whisker or above the right whisker is considered an outlier.

8. Discrete Probability Distributions

A numerical variable whose value depends on the outcome of a chance or random event is called a **random variable**. If the set of possible values of a random variable is countable or finite, it is called a **discrete random variable**.

A discrete probability distribution function has two characteristics:

- Each probability is between zero and one, inclusive.
- The sum of the probabilities is one.

8.1 Expected Values and Standard Deviations

[Watch demo video](#)

If the probability distribution of the random variable X is given by $p(x)$, then:

1. The expected value of a random variable X is given by $E(X) = \mu = \sum xp(x)$
2. The standard deviation of the random variable X is given by
$$\sigma = \sqrt{E(X - \mu)^2} = \sqrt{\sum (x - \mu)^2 p(x)}$$

Syntax for expected value

```
import numpy as np
np.average(x, weights=prob)
```

Syntax for population standard deviation

```
import numpy as np
np.sqrt(np.cov(x, aweights=prob, ddof=0))
```

Example: The number of electronic devices per household in a neighborhood and their corresponding probabilities are given on the table below. Find the expected value and standard deviation of the number of devices per household in this neighborhood.

x	p(x)
0	.01
1	.02

x	p(x)
2	.13
3	.33
4	.51

```
In [ ]: import numpy as np

x = [0,1,2,3,4]
prob = [.01,.02,.13,.33,.51]

expected_value = np.average(x,weights=prob)
expected_value
```

Out[]: 3.31

Interpretation: The mean number of electronic devices per household is 3.31.

```
In [ ]: import numpy as np

x = [0,1,2,3,4]
prob = [.01,.02,.13,.33,.51]

standard_deviation = np.sqrt(np.cov(x, aweights=prob, ddof=0))
standard_deviation
```

Out[]: 0.8449260322655469

Interpretation: The mean number of electronic devices per household is 3.31 with a standard deviation of 0.845.

8.2 Binomial Probability

If an experiment consists of n identical and independent trials, with probability of success p (remains the same for each trial), then the random variable X defined as the number of successes in n trials is a binomial random variable and the probability of x successes is given by:

$$P(X = x) = \binom{n}{x} p^x (1 - p)^{n-x}$$

where $x = 0, 1, 2, 3, \dots, n$

The syntax for $P(X = x)$, given n and p is

```
from scipy import stats
stats.binom.pmf(x,n,p)
```

The syntax for cumulative probability $P(X \leq)$, given n and p is

```
from scipy import stats
stats.binom.cdf(x,n,p)
```

The syntax for mean and standard deviation is given by:

```
from scipy import stats
stats.binom.mean(n,p)
stats.binom.std(n,p)
```

Example: Find $P(X = 10)$ if $p = 0.6, n = 12$

$$P(x = 10) = \binom{12}{10} 6^{10} (1 - 0.6)^{12-10}$$

Here x=10, n=12 and p=0.6.

```
In [ ]: from scipy import stats
stats.binom.pmf(10, 12, 0.6)
```

```
Out[ ]: 0.06385228185599987
```

Example: For a Binomial distribution with 12 number of trials and probability of success 0.6, find the cumulative probabilities $P(x \leq 2) = \sum_1^6 \binom{12}{2} .6^2 (1 - .6)^{12-2}$. Here x=2, p=0.6 and n=12.

```
In [ ]: from scipy import stats
stats.binom.cdf(k=2, n=12, p=0.6) # same as stats.binom.cdf(2, 12, 0.6)
```

```
Out[ ]: 0.00281018368
```

Interpretation: $P(X \leq 2) = 0.00281018368$

Example: A manufacturing machine has a 5% defect rate. Write a `scipy.stats` program to compute the following:

- If 6 items are chosen at random, what is the probability of exactly 2 defects.

```
In [ ]: from scipy import stats
stats.binom.pmf(2,6,0.05)
```

```
Out[ ]: 0.030543984375000013
```

Interpretation: The probability of exactly two out of six defects is 0.030543984375000013

Example: A manufacturing machine has a 5% defect rate. Write a `scipy.stats` program to compute the following:

- If 6 items are chosen at random, what is the probability that 3 or less will have a defect?

```
In [ ]: from scipy import stats
stats.binom.cdf(3,6,0.05)
```

```
Out[ ]: 0.99991359375
```

Interpretation: The probability of 3 or less defects is 0.99991359375

Example: A manufacturing machine has a 5% defect rate. Write a `scipy.stats` program to compute the following:

- If 6 items are chosen at random, what is the probability that at least two defects?

```
In [ ]: from scipy import stats
1-stats.binom.cdf(1,6,0.05)
```

```
Out[ ]: 0.03277382812500007
```

Interpretation: The probability of at least two defects is 0.03277382812500007

Example: 20% of cars fail emission inspections in IL. If 15 cars are randomly selected, what is the probability that exactly three fail.

Here $x=3$, $p=.2$ and $n=15$

[Watch demo video](#)

```
In [ ]: from scipy import stats
stats.binom.pmf(3,15,.2)
```

```
Out[ ]: 0.2501388953190411
```

Interpretation: $P(X = 3) = 0.00281018368$

Example: 20% of cars fail emission inspections in IL. If 15 cars are randomly selected, what is the probability that at most five fail?

Here $x=5$, $p=.15$ and $n=15$ (Cumulative)

[Watch demo video](#)

```
In [ ]: from scipy import stats
stats.binom.cdf(5,15,.2)
```

```
Out[ ]: 0.938948570382336
```

Interpretation: $P(X \leq 5) = 0.938948570382336$

Example: 20% of cars fail emission inspections in IL. If 15 cars are randomly selected, what is the probability that at least five fail?

$$P(X \geq 5) = 1 - P(X \leq 4)$$

```
In [ ]: from scipy import stats
1 - stats.binom.cdf(4,15,.2)
```

```
Out[ ]: 0.16423372393676794
```

Interpretation: $P(X \geq 5) = 0.16423372393676794$

Example: 20% of cars fail emission inspections in IL. 15 cars are randomly selected from IL. Make a probability distribution table for each outcome.

```
In [ ]: from scipy import stats
import numpy as np
x = np.arange(0,15+1) #generates [0,1,2,...,15]
y = stats.binom.pmf(x,15,.2)

prob = zip(x,y)
list(prob)
```

```
Out[ ]: [(0, 0.035184372088831996),
(1, 0.13194139533312002),
(2, 0.23089744183296135),
(3, 0.2501388953190411),
(4, 0.18760417148928044),
(5, 0.10318229431910414),
(6, 0.04299262263296016),
(7, 0.013819057274880048),
(8, 0.0034547643187200112),
(9, 0.000671759728640003),
(10, 0.00010076395929600008),
(11, 1.1450449920000038e-05),
(12, 9.542041600000053e-07),
(13, 5.505024000000039e-08),
(14, 1.966080000000028e-09),
(15, 3.27680000000002e-11)]
```

Interpretation:

x	probability
0	0.035184372088831996
1	0.13194139533312002
2	0.23089744183296135
3	0.2501388953190411
4	0.18760417148928044

x	probability
5	0.10318229431910414
6	0.04299262263296016
7	0.013819057274880048
8	0.0034547643187200112
9	0.000671759728640003
10	0.00010076395929600008
11	1.1450449920000038e-05
12	9.542041600000053e-07
13	5.505024000000039e-08
14	1.966080000000028e-09
15	3.276800000000002e-11

Binomial Plot

Use the function `binomplot` below to plot a binomial distribution with probability p and number of trials n .

```
In [ ]: # binomial graph with n and p
def binomplot(n,p):
    from scipy import stats
    import numpy as np
    import matplotlib.pyplot as plt

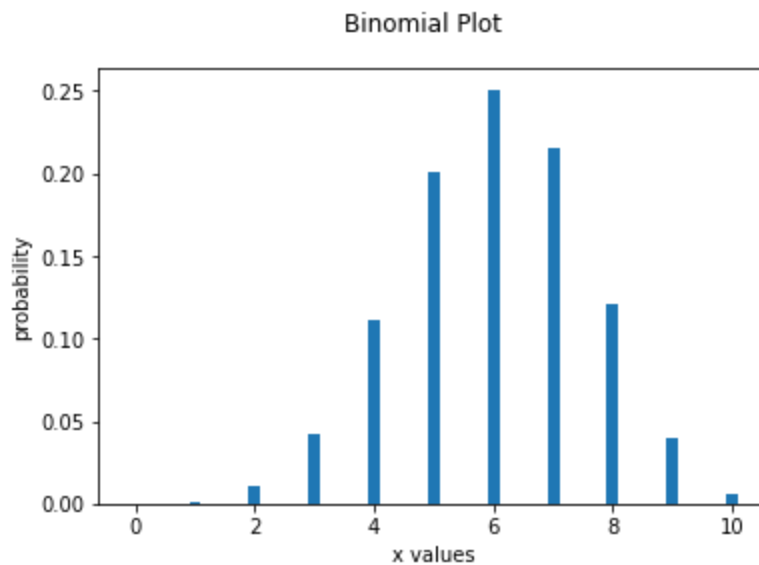
    x = np.arange(0,n+1)
    y = stats.binom.pmf(x,n,p)
    plt.bar(x,y,width=0.2)

    plt.suptitle("Binomial Plot")
    plt.xlabel("x values")
    plt.ylabel("probability")

    return plt.show()
```

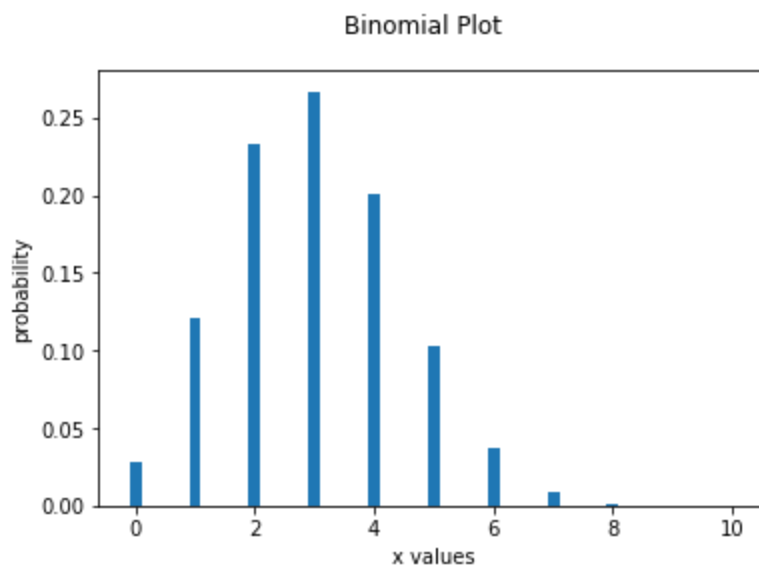
Example: plot a binomial distribution with $n=10$ and $p=0.6$

```
In [ ]: binomplot(10,.6) # run the binomplot function above first
```



Example: Graph a Binomial Probability Distribution with $n=10$ and $p=.3$.

```
In [ ]: binomplot(10,.3) # run the binomplot function above first
```



9. Continuous Probability Distributions

Let $f(x)$ be the probability distribution of a continuous random variable x . Then the probability that $a < x < b$ corresponds to the area under the curve $y = f(x)$ between a and b .

9.1 Uniform Distribution

The uniform probability distribution with minimum $\min$ and maximum $\max$ is given by:

$$f(x) = \frac{1}{\max - \min} \text{ for } \min < x < \max$$

- probability $P(a < x < b) = \frac{b-a}{\max - \min}$ area under $f(x)$ from a to b
- mean or $E(x) = \frac{\min + \max}{2}$ and
- standard deviation is $\sigma(x) = \sqrt{\frac{(\max - \min)^2}{12}}$

The syntax for cumulative probability, $P(X < x)$, inverse, mean and standard deviation:

```
from scipy import stats
stats.uniform.cdf(x,min,max-min) # cumulative prob P(X<x)
stats.uniform.ppf(p,min,max-min) # inverse of cdf or percentiles
stats.uniform.mean(min,max-min) # mean of X
stats.uniform.std(min,max-min) # std of X
```

Example: The amount of time, in minutes, that a person waits for a bus, in a bus station, is uniformly distributed between zero and 15 minutes, inclusive.

- What is the probability that a person waits fewer than 12.5 minutes?

$$P(X < 12.5) = \text{stats.uniform.cdf}(12.5, 0, 15)$$

```
In [ ]: from scipy import stats
stats.uniform.cdf(12.5,0,15)
```

```
Out[ ]: 0.8333333333333334
```

Interpretation: The probability of waiting less than 12.5 minutes is 0.8333333333333334

Example: The amount of time, in minutes, that a person waits for a bus, in a bus station, is uniformly distributed between zero and 15 minutes, inclusive.

- On the average, how long must a person wait? Find the mean, $\hat{\mu}$, and the standard deviation, $\hat{\sigma}$.

```
In [ ]: stats.uniform.mean(0,15) # mean
```

```
Out[ ]: 7.5
```

Interpretation: The mean waiting time is 7.5

```
In [ ]: stats.uniform.std(0,15) # std
```

```
Out[ ]: 4.330127018922194
```

Interpretation: The mean waiting time is 7.5 with a standard deviation of 4.33.

Example: The amount of time, in minutes, that a person waits for a bus is uniformly distributed between zero and 15 minutes, inclusive.

- Ninety percent of the time, the length of time a person must wait falls below what value?

90th percentile = stats.uniform.ppf(.9, 0, 15)

```
In [ ]: stats.uniform.ppf(.9,0,15)
```

```
Out[ ]: 13.5
```

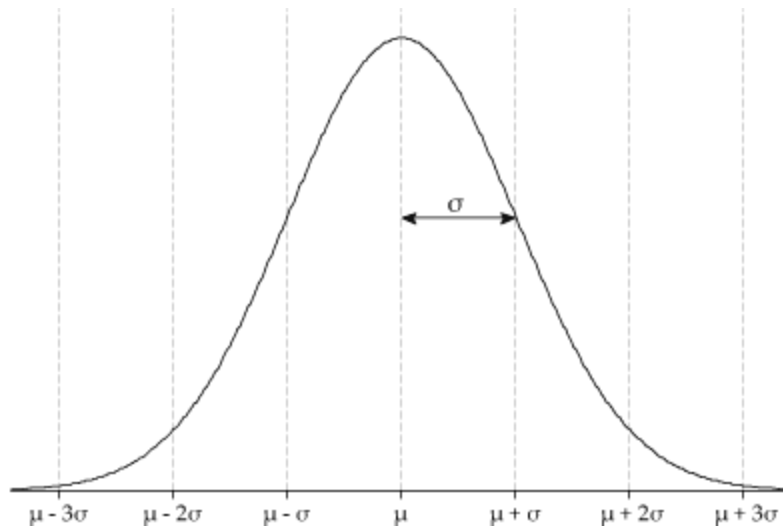
Interpretation: Ninety percent of the waiting time is less than or equal to 13.5

9.2 Normal Probability Distribution

The **normal** probability distribution with mean μ and standard deviation σ is given by:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

It is bell shaped and is symmetric about the mean μ .



The syntax for the normal cumulative probability $P(X < x)$ with μ =mean and σ =std is:

```
from scipy import stats
stats.norm.cdf(x,mean,std)
```

The syntax for inverse normal or pth percentile (with area to the left of p) and μ =mean and σ =std is:

```
from scipy import stats
stats.norm.ppf(p,mean,std)
```


Note: The **standard normal distribution** has a mean μ of zero and standard deviation σ of one.

Example: In a standard normal distribution what is the probability that the z value is less than 2?

[Watch demo video](#)

```
In [ ]: from scipy import stats
stats.norm.cdf(2,0,1)    #for standard normal mean is zero and std is one
```

```
Out[ ]: 0.9772498680518208
```

Interpretation: $P(z < 2) = 0.9772498680518208$

Example: In a standard normal distribution what is the probability that the z value is between -2 and 2.

$$P(-2 < z < 2) = P(z < 2) - P(z < -2)$$

$$= \text{stats.norm.cdf}(2,0,1) - \text{stats.norm.cdf}(-2,0,1)$$

```
In [ ]: from scipy import stats
stats.norm.cdf(2,0,1)-stats.norm.cdf(-2,0,1)
```

```
Out[ ]: 0.9544997361036416
```

Interpretation: $P(-2 < z < 2) = 0.9544997361036416$

Example: The midterm exam scores in a math class is normally distributed with a mean of 73 and a standard deviation of 5.

- Find the probability that a randomly selected student scored more than 65 on the exam.

$$P(X > 65) = 1 - P(X < 65) = 1 - \text{stats.norm.cdf}(65,73,5)$$

[Watch demo video](#)

```
In [ ]: from scipy import stats
1-stats.norm.cdf(65,73,5)
```

```
Out[ ]: 0.945200708300442
```

Interpretation: $P(X > 65) = 0.945200708300442$

Example: The midterm exam scores in a math class is normally distributed with a mean of 73 and a standard deviation of 5.

- Find the probability that a randomly selected student scored less than 85.

$$P(X < 85) = \text{stats.norm.cdf}(85, 73, 5)$$

```
In [ ]: from scipy import stats
stats.norm.cdf(85, 73, 5)
```

```
Out[ ]: 0.9918024640754038
```

Interpretation: $P(X < 85) = 0.9918024640754038$

Example: The midterm exam scores in a math class is normally distributed with a mean of 73 and a standard deviation of 5.

- Find the 90th percentile (that is, find the score k that has 90% of the scores below k and 10% of the scores above k).

$$90^{th} \text{percentile} = \text{stats.norm.ppf}(.9, 73, 5)$$

[Watch demo video](#)

```
In [ ]: from scipy import stats
stats.norm.ppf(.90, 73, 5)
```

```
Out[ ]: 79.407757827723
```

Interpretation: Ninety percent of scores are less than or equal to 79.407757827723

Example: The midterm exam scores in a math class is normally distributed with a mean of 73 and a standard deviation of 5.

- Find the probability that a randomly selected student has a score between 70 and 90.

$$P(70 < X < 90) = P(X < 90) - P(X < 70)$$

```
In [ ]: stats.norm.cdf(90, 73, 5) - stats.norm.cdf(70, 73, 5)
```

```
Out[ ]: 0.7254099529842495
```

Interpretation: $P(70 < X < 90) = 0.7254099529842495$

SciPy.Stats Summary of Probability Distributions

```
from scipy import stats
```

procedure	scipy.stats
Binomial pdf $P(X = x)$	<code>stats.binom.pmf(k, n, p)</code>
Binomial cdf $P(X \leq x)$	<code>stats.binom.cdf(k, n, p)</code>
Normal cdf $P(X \leq x)$	<code>stats.norm.cdf(x, mean, std)</code>

procedure	scipy.stats
Inverse Normal or p-th Percentile	<code>stats.norm.ppf(p, mean, std)</code>
t distribution cdf $P(X \leq x)$	<code>stats.t.cdf(t, df, 0, 1)</code>
Inverse t or p-th Percentile	<code>stats.t.ppf(p, df, 0, 1)</code>
chi-square cdf $P(\chi^2 \leq ch2)$	<code>stats.chi2.cdf(ch2, k-1)</code>
Inverse chi-square or p-th Percentile	<code>stats.chi2.ppf(p, k-1)</code>

9.3 Central Limit Theorem

[Watch demo video](#)

The sampling distribution of the sample mean is the probability distribution for the possible values of the mean that results when random sample of size n are repeatedly drawn from the population with mean μ and standard deviation σ .

When the sample size is large or the samples are drawn from a population that is normally distributed with mean μ and standard deviation σ , the sampling distribution of the sample mean $\bar{x}$ is approximately normal with mean μ and standard deviation of $\frac{\sigma}{\sqrt{n}}$

Example: A normally distributed population has a mean of 90 and a standard deviation of 15. Samples of size $n = 25$ are drawn randomly from the population and the mean is computed.

- Find the probability that sample mean is less than 85.

$$P(\bar{x} < 85) = \text{stats.norm.cdf}(85, 90, 15/25^{**}.5)$$

```
In [ ]: from scipy import stats
stats.norm.cdf(85, 90, 15/25**(1/2))
```

```
Out[ ]: 0.0477903522728147
```

Interpretation: $P(\bar{x} < 85) = 0.0477903522728147$

Example: A normally distributed population has a mean of 90 and a standard deviation of 15. Samples of size $n = 25$ are drawn randomly from the population and the mean is computed.

- Find the probability that the sample mean is between 85 and 92.

$$P(85 < \bar{x} < 92) = P(\bar{x} < 92) - P(\bar{x} < 85)$$

$$= \text{stats.norm.cdf}(92, 90, \frac{15}{\sqrt{25}}) - \text{stats.norm.cdf}(85, 90, \frac{15}{\sqrt{25}})$$

```
In [ ]: from scipy import stats
stats.norm.cdf(92,90,15/25**.5)-stats.norm.cdf(85,90,15/25**.5)
```

```
Out[ ]: 0.6997171101802624
```

Interpretation: $P(85 < \bar{x} < 92) = 0.6997171101802624$

Example: The mean age of tablet device users is 34 years. Suppose the standard deviation is 15 years. If a random sample of size $n = 100$ is drawn from this population:

- Find the probability that the sample mean age is more than 30 years.
- Find the 95th percentile for the sample mean age.

$$P(\bar{x} > 30) = 1 - P(\bar{x} < 30) = 1 - \text{stats.norm.cdf}(30, 34, \frac{15}{\sqrt{100}})$$

```
In [ ]: from scipy import stats
1-stats.norm.cdf(30,34,15/100**.5)
```

```
Out[ ]: 0.9961696194324102
```

Interpretation: $P(\bar{x} > 30) = 0.9961696194324102$

$$95^{th} \text{ percentile} = \text{stats.norm.ppf}(.95, 34, \frac{15}{\sqrt{100}})$$

```
In [ ]: from scipy import stats
stats.norm.ppf(.95,34,15/100**.5)
```

```
Out[ ]: 36.46728044042721
```

Interpretation: Ninety-five percent of the sample means are less than or equal to 36.46728044042721

Example

The average score for an IQ test is 100 with a standard deviation of 15. Assume the test scores are normally distributed.

1. If a random sample of 25 test scores is selected from this population, what is the probability that the sample mean is between 90 and 110.
2. Find the 95th percentile for the sample mean score.

```
In [ ]: from scipy import stats
stats.norm.cdf(110,100,15/(25**.5))-stats.norm.cdf(90,100,15/(25**.5))
```

```
Out[ ]: 0.9991418793336064
```

Interpretation: The probability that the mean is between 90 and 110 is 0.9991418793336064. That is, $P(90 < \bar{x} < 110) = 0.9991418793336064$

```
In [ ]: from scipy import stats
stats.norm.ppf(.95,100,15/(25**.5))
```

```
Out[ ]: 104.93456088085442
```

Interpretation: 95 percent of the test scores are less than or equal to 104.93456088085442.

10. Confidence Intervals

Inferential statistics deals with making inferences about a population parameter using information contained in the sample. Estimation and Hypothesis testing are two types of inferential statistics. There are two types of estimation: point and interval estimation.

1. The sample mean $\bar{x}$ is a point estimate of the population mean μ
2. The sample proportion $\hat{p}$ is a point estimate of the population proportion p .

An **interval estimate** or **confidence interval** provides two values, where all real numbers between these values are considered as an estimate to the population parameter.

1. $(\bar{x} - E, \bar{x} + E)$ interval estimate of the mean μ
2. $(\hat{p} - E, \hat{p} + E)$ interval estimate of the proportion p

10.1 Confidence Interval for Mean μ (z distribution)

[Watch demo video 1](#)

[Watch demo video 2](#)

Consider a random sample size of n from a population with an unknown mean μ and known standard deviation σ . To construct an interval estimate of the population mean μ with $(1 - \alpha)100\%$ confidence, use the following:

$$C = 1 - \alpha \text{ (the confidence level)}$$

$$E = z_{\frac{\alpha}{2}} \frac{\sigma}{\sqrt{n}} \text{ (the Margin of Error)}$$

$$\bar{x} - E < \mu < \bar{x} + E \text{ (the confidence interval)}$$

For large sample size the sample standard deviation can be used for the population standard deviation σ .

Use `scipy.stats` for obtaining the value of $z_{\frac{\alpha}{2}}$ where $C = 1 - \alpha$ as follows:

```
from scipy import stats
stats.norm.ppf(C+(1-C)/2,mean,std)
```

Margin of Error for μ (summary statistics given)

To find the margin of error for estimating the population mean when summary statistics such as the sample size and population standard deviation are given, use the `error_z_mean` function below with arguments: confidence level C, Population standard deviation std and sample size n.

$$E = z_{\frac{\alpha}{2}} \frac{\sigma}{\sqrt{n}}$$

```
In [ ]: # Margin of Error, given confidence level C, std, n
def error_z_mean(C,std,n):
    from scipy import stats
    er = (std/n**.5)*stats.norm.ppf(C+(1-C)/2,0,1)
    return print("The Margin of Error is",er)
```

Or you can use the code below by substituting `std`, `n` and `C` :

```
from scipy import stats
error = (std/n**.5)*stats.t.ppf(C+(1-C)/2,n-1,0,1)
```

Example: Midterm exam scores are normally distributed with an unknown population mean and a population standard deviation of three points. A random sample of 40 scores has mean of 72. Find a 90% margin of error and the confidence interval estimate for the population mean exam score.

- Find the margin of error for estimating the population mean.

We use the `error_z_mean` function above with arguments: C = 0.9, std = 3, n = 40.

1. Click `play` button in the above code `def error_z_mean(C,std,n)`
2. Click `+ Code`
3. Type ``error_z_mean(.90,3,40)`

```
In [ ]: error_z_mean(.90,3,40)
```

The Margin of Error is 0.780222581813336

Confidence Interval for μ (summary statistics given)

To find the confidence interval for the population mean when summary statistics such as the sample size, mean and population standard deviation are given, use the `conf_z_mean` function below with arguments: confidence level C, mean, std, and n.

```
In [ ]: # Confidence interval, given confidence level C, std, n
def conf_z_mean(C, mean, std, n):
    from scipy import stats
    ebm = (std/n**.5)*stats.norm.ppf(C+(1-C)/2, 0, 1)
    return print("(", mean-ebm, ", ", mean+ebm, ")")
```

Example: Midterm exam scores are normally distributed with an unknown population mean and a population standard deviation of three points. A random sample of 40 scores has mean of 72. Find a 90% confidence interval estimate for the population mean exam score.

- Find a 90% confidence interval for the true (population) mean of the exam scores.

We use the `conf_z_mean` function above with arguments: $C = 0.9$, $\text{mean} = 72$, $\text{std} = 3$, $n = 40$.

```
In [ ]: conf_z_mean(.90, 72, 3, 40)

( 71.21977741818667 , 72.78022258181333 )
```

Interpretation: $71.21977741818667 < \mu < 72.78022258181333$

10.2 Confidence Interval for Mean μ (t Distribution)

[Watch demo video](#)

Consider a random sample of n from a population with an unknown mean μ and unknown standard deviation σ . To construct an interval estimate of the population mean μ with $(1 - \alpha)100\%$ confidence, use the following: Confidence level, $C = 1 - \alpha$

$C = 1 - \alpha$ (level of confidence)

$E = t_{\frac{\alpha}{2}} \times \frac{s}{\sqrt{n}}$ (Margin of Error)

$\bar{x} - E < \mu < \bar{x} + E$ (confidence interval)

scipy.stats to obtain the value of $t_{\frac{\alpha}{2}}$

```
from scipy import stats
stats.t.ppf(C+(1-C)/2, n-1, 0, 1) # df=n-1
```

Note: When the sample size is large, the t-distribution is robust when the shape of the distribution is not normal.

Margin of Error for μ (t-distribution summary stats given)

To find the margin of error to estimate the population mean when summary stats is given, use the function `error_t_mean` below with arguments: level of confidence C, std and n. Here the population is assumed to be normal.

$$E = t_{\frac{\alpha}{2}} \frac{s}{\sqrt{n}}$$

```
In [ ]: # margin of error given the level of confidence C, mean, std and n
def error_t_mean(C,std,n):
    from scipy import stats
    ebm = (std/n**.5)*stats.t.ppf(C+(1-C)/2,n-1,0,1)
    return print("Margin of Error is",ebm)
```

You can also use the code below by substituting `std`, `n` and `C` :

```
from scipy import stats
error = (std/n**.5)*stats.t.ppf(C+(1-C)/2,n-1,0,1)
```

Example: In a study of a new pain relieving drug, the mean and standard deviation of pain levels for 15 subjects are 8 and 1.7 respectively. Find the margin of error for constructing a 95% confidence interval for the mean pain levels for the population (assume normal).

```
In [ ]: # Option 1

from scipy import stats
C = .95
std = 1.72
n = 15

error = (std/n**.5)*stats.t.ppf(C+(1-C)/2,n-1,0,1)
error
```

```
Out[ ]: 0.9525042514911835
```

```
In [ ]: # Option 2
error_t_mean(.95, 1.72, 15)
```

Margin of Error is 0.9525042514911835

Interpretation: In both cases, the margin of error is 0.9525042514911835.

To find the confidence interval for estimating the population mean when summary stats is given you can subtract and add the margin of error from above to the mean or use the function `conf_t_mean` below with arguments: level of confidence C, mean, std and n. Here the population is assumed to be normal.

```
In [ ]: # confidence interval given level of confidence C and x

def conf_t_mean(C,mean,std,n):
    from scipy import stats
    ebm = (std/n**.5)*stats.t.ppf(C+(1-C)/2,n-1,0,1)
```



```
print ('mean is',mean,'and std is',std)
return print("(", mean-ebm, ",", mean+ebm, ')'
```

Example:

Confidence Interval for μ (raw data given)

To find the confidence interval and margin of error for the population mean when raw data is given, first find the sample mean, std and size using numpy.

Example: In a study of a new pain relieving drug, the measures of pain levels for 15 subjects are recorded below. Use the sample data to construct a 95% confidence interval for the mean pain levels for the population (assume normal).

8.6; 9.4; 7.9; 6.8; 8.3; 7.3; 9.2; 9.6; 8.7; 11.4; 10.3; 5.4; 8.1; 5.5;
6.9

- Find the margin of error for 95% confidence interval

We will use the `error_t_mean` function above. But first we need the sample std and size.

```
In [ ]: x = [8.6, 9.4, 7.9, 6.8, 8.3, 7.3, 9.2, 9.6, 8.7, 11.4, 10.3, 5.4, 8.1,
           5.5, 6.9]

import numpy as np
s = np.std(x,ddof=1)
n = np.size(x)

print('std',s)
print('size',n)
```

```
std 1.6722383060978339
size 15
```

```
In [ ]: error_t_mean(.95,s,n) # values of s and n from above
```

Margin of Error is 0.9260547070143034

Example: In a study of a new pain relieving drug, the measures of pain levels for 15 subjects are recorded below. Use the sample data to construct a 95% confidence interval for the mean pain levels for the population (assume normal).

8.6; 9.4; 7.9; 6.8; 8.3; 7.3; 9.2; 9.6; 8.7; 11.4; 10.3; 5.4; 8.1; 5.5;
6.9

- Find the 95% confidence interval for the mean

use the `conf_t_mean` function above but first find the mean, std and n.

```
In [ ]: x = [8.6, 9.4, 7.9, 6.8, 8.3, 7.3, 9.2, 9.6, 8.7, 11.4, 10.3, 5.4, 8.1,
            5.5, 6.9]

import numpy as np
m = np.mean(x)
s = np.std(x, ddof=1)
n = np.size(x)
```

Call the function `conf_t_mean` with mean = m, std = s using results from the above code.

```
In [ ]: conf_t_mean(.95,m,s,n) # values of m,s and n from above
```

mean is 8.226666666666667 and std is 1.6722383060978339
(7.300611959652363 , 9.15272137368097)

Interpretation: $7.300611959652363 < \mu < 9.15272137368097$

10.3 Confidence Intervals for Proportion p (z-distribution)

Suppose a random sample of n (identical trials) is selected from a population with probability of success, p . The $(1 - \alpha)100\%$ confidence interval is given by:

$$C = 1 - \alpha \text{ (Confidence level)}$$

$$E = z_{\frac{\alpha}{2}} \sqrt{\frac{\hat{p}\hat{q}}{n}} \text{ (Margin of Error)}$$

$$\hat{p} - E < P < \hat{p} + E \text{ (Confidence interval)}$$

Confidence Interval and Margin of Error for p

[Watch demo video](#)

To find the confidence interval for the population proportion when summary statistics such as number of success in the sample and sample size, use the function `conf_prop` below with arguments: confidence level C , number of success and sample size n .

$$E = z_{\frac{\alpha}{2}} \sqrt{\frac{\hat{p}\hat{q}}{n}}$$

$$\hat{p} = \frac{x}{n} = p \text{ is the sample proportion}$$

```
In [ ]: # confidence interval and margin of error given sample proportion p and n

def conf_prop(C,p,n):
    from scipy import stats
    eb1 = ((p*(1-p)/n)**.5)*stats.norm.ppf((C+(1-C)/2,0,1))
```

```
return print(" Sample Proportion =",p,"\n Margin of Error = ", eb1,
            "\n Confidence Interval", "(" ,p - eb1,"",p + eb1,"")
```

Or you can use the code below by substituting $p = x/n$ for the sample proportion and n sample size.

```
error = ((p*(1-p)/n)**.5)*stats.norm.ppf(C+(1-C)/2,0,1)
```

Example: In a random sample of 35 students in a college, 20% are registered to vote. Construct the 90% confidence interval for the proportion of all students in the college registered to vote.

Use the `conf_prop` function with arguments: $C=.9$, $p=.2$ and $n=35$.

```
In [ ]: conf_prop(.90,.2,35)
```

```
Sample Proportion = 0.2
Margin of Error = 0.11121240329789352
Confidence Interval ( 0.0887875967021065 , 0.3112124032978935 )
```

Example: In a random sample of 35 students in a college, 20 are registered to vote. Construct the 95% confidence interval for the proportion of all students in the college registered to vote.

Before we use the `conf_prop` we need to find the sample proportion $p=x/n = 20/35$

```
In [ ]: p = 20/35
p
```

```
Out[ ]: 0.5714285714285714
```

Now call the function with $C = .95$, $p = 0.5714285714285714$ and $n = 35$

```
In [ ]: conf_prop(.95,0.5714285714285714,35)
```

```
Sample Proportion = 0.5714285714285714
Margin of Error = 0.16394819961335458
Confidence Interval ( 0.4074803718152168 , 0.735376771041926 )
```

Calculating the Sample Size n

****Sample Size for Estimating Proportions, *p*****

$$n = p(1 - p) * \left[\frac{z_{\frac{\alpha}{2}}}{E} \right]^2$$

To calculate the minimum sample size n for a given level of confidence C , estimated proportion p and margin of error E , use the function `samp_pro` below. $p=.5$ when no estimates are available.

```
In [ ]: def samp_pro(C,p,E):
        from scipy import stats
        sam = p*(1-p)*stats.norm.ppf(C+(1-C)/2,0,1)**2/E**2
        return print("sample size = ", sam)
```

Example: Suppose city officials want to determine the current percentage of residents vaccinated for Covid19. How many residents should the city officials survey in order to be 90% confident that the estimated (sample) proportion is within three percentage points of the true population proportion of residents who are vaccinated for Covid19?

use the `samp_pro` function above with arguments: $C = 0.90$, $p = 0.5$ (none is given), $E = 0.03$.

```
In [ ]: samp_pro(.90,.5,.03)
```

sample size = 751.5398483598369

Interpretation: 752 (always round up) residents are needed.

Sample Size for Estimating Mean, μ

$$n = \left[\frac{\sigma \times z_{\frac{\alpha}{2}}}{E} \right]^2$$

To calculate the minimum sample size n for a given level of confidence C , standard deviation σ as s and margin of error E , use the function `samp_mean` below.

```
In [ ]: def samp_mean(C,s,E):
        from scipy import stats
        sam = (s*stats.norm.ppf(C+(1-C)/2,0,1)/E)**2
        return print("sample size = ", sam)
```

```
In [ ]: samp_mean(.95,15,3)
```

sample size = 96.03647051735315

Example: A 4th grade elementary school teacher wants to estimate the mean IQ score of all 4th grade students. How many 4th grade students must be randomly selected if she wants a 90% confidence that the sample mean is within 5 IQ scores of the population mean? (Use $\sigma = 15$)

use the `samp_mean` function above with arguments: $C = 0.90$, $s = 15$ and $E = 3$.

```
In [ ]: samp_mean(0.9,15,3)
```

sample size = 67.63858635238533

Interpretation: 68 (round up) students are needed in the random sample.

11. Hypothesis Testing

Hypothesis testing involves making decisions on a **claim** or statement made about the value of population parameter.

First, the claim is used to define the null H_o and alternative H_a hypotheses. The alternative hypothesis is used to determine if the test is left (smaller), right (greater) or two-tailed test. A test-statistic and p-value are calculated with the assumption that the null hypothesis is true. A significant level or rejection region can be used to make a decision about the null-hypothesis.

11.1 Hypothesis Testing for Mean μ (z-test)

[Watch demo video](#)

Suppose a random sample of size n is drawn from a population with unknown mean μ and either the standard deviation σ is given or the sample size is large.

To test a hypothesis about the mean μ of the form:

$$H_0 : \mu = \mu_0$$

$$H_a : \mu < \mu_0 \text{ smaller}$$

OR:

$$H_0 : \mu = \mu_0$$

$$H_a : \mu > \mu_0 \text{ greater}$$

OR:

$$H_0 : \mu = \mu_0$$

$$H_a : \mu \neq \mu_0 \text{ two-sided}$$

we use the following test-statistic

$$z = \frac{\bar{x} - \mu_0}{\frac{\sigma}{\sqrt{n}}}$$

Use the function `z_test_mean` below with arguments: `null_mean =  $\mu_0$` , `sigma =  $\sigma$` , `sample_mean =  $\bar{x}$` , `n`, `alternative = 'smaller' OR 'greater' OR 'two-sided'`. By default it is set to 'two-sided'.

```
In [ ]: # z test with alternative 'smaller' or 'greater' or 'two-sided'
def z_test_mean(null_mean, sigma, sample_mean, n, alternative='two-sided'):
    from scipy import stats
    z = (sample_mean - null_mean) / (sigma / n**.5)
    if alternative == 'smaller':
        pv = stats.norm.cdf(z, 0, 1)
```

```

elif alternative == 'greater':
    pv = 1-stats.norm.cdf(z,0,1)
elif alternative == 'two-sided':
    if z < 0:
        pv = 2*stats.norm.cdf(z,0,1)
    else: pv = 2*(1-stats.norm.cdf(z,0,1))
else:
    print("alternative option error")
return print("z test statistic = ", z, "and p-value = ",pv)

```

Example: The president of a company claims that the average number of minutes employees waste time surfing the internet is more than 100 minutes a day. The population number of hours wasted is normally distributed with standard deviation of 10 minutes. A random sample of 19 employees from this company has a mean of 105 minutes. Do this sample data provide evidence that the mean wasted time for this company is more than 100 minutes? Use $\alpha = 0.05$.

$$H_o : \mu = 100$$

$$H_a : \mu > 100$$

Use the `z_test_mean` function above with arguments: `null_mean = 100`, `sigma = 10`, `sample_mean = 105`, `n = 19`, `alternative = 'greater'` (more than 100).

1. Click `play` button on code above `def z_test_mean`
2. Click `+ Code`
3. Type `z_test_mean(100,10,105,19,'greater')`
4. Click `play` button

```
In [ ]: z_test_mean(100,10,105,19,'greater')
```

```
z test statistic = 2.179449471770337 and p-value = 0.014649147202275281
```

Interpretation: $p\text{-value} = 0.014649 < \alpha = 0.05$. The null hypothesis is rejected.

Alternatively, compare `z = (sample_mean-null_mean)/(sigma/n**.5)` with the critical `stats.norm.ppf(1-.05,0,1) = 1.6448` see code below.

$z = 2.179449471770337 > 1.6448536269514722$ thus reject the null-hypothesis.

```
In [ ]: from scipy import stats
stats.norm.ppf(1-0.05,0,1) # right-tailed test
```

```
Out[ ]: 1.6448536269514722
```

Example: A government official claims that the average recovery time for Covid19 patients with severe symptoms is 6 weeks. A random sample of 23 patients with severe symptoms has a mean recovery time of 5.4 weeks with standard deviation of 1.1 weeks. Assume recovery time is normally distributed with standard deviation of 1.5 (σ) weeks. Test the government's official claim at α of 0.05. Use the `z_test_mean` function below.

```
In [ ]: # z test with alternative 'smaller' or 'greater' or 'two-sided'
def z_test_mean(null_mean,sigma,sample_mean,n,alternative='two-sided'):
    from scipy import stats
    z = (sample_mean-null_mean)/(sigma/n**.5)
    if alternative == 'smaller':
        pv = stats.norm.cdf(z,0,1)
    elif alternative == 'greater':
        pv = 1-stats.norm.cdf(z,0,1)
    elif alternative == 'two-sided':
        if z < 0:
            pv = 2*stats.norm.cdf(z,0,1)
        else: pv = 2*(1-stats.norm.cdf(z,0,1))
    else:
        print("alternative option error")
    return print("z test statistic = ", z, "and p-value = ",pv)
```

```
null_mean=6,sigma=1.5,sample_mean=5.4,n=23,alternative='two-sided'
```

```
In [ ]: z_test_mean(6,1.5,5.4,23,'two-sided')
```

z test statistic = -1.9183326093250865 and p-value = 0.05506885012079212

Interpretation: Because the p-value 0.05506885012079212 is not less than 0.05, reject the null hypothesis. There is no sufficient evidence to reject the official's claim.

Example: The work week of a particular organization workers is normally distributed with a mean of 47 hours. A newly hired manager at this company believes that employees average work hours is **different** from 47. She randomly asks 12 workers for the lengths in hours of their work week. Their responses are shown below. Test the claim using a 10% level of significance. The population standard deviation is 5.

Hours: 46, 43, 55, 50, 46, 67, 48, 47, 48, 49, 55, 54

$$H_o : \mu = 47$$

$$H_a : \mu \neq 47$$

We need to find the mean and sample size of the data before we call the function,

```
z_test_mean .
```

```
In [ ]: import numpy as np
x = [46, 43, 55, 50, 46, 67, 48, 47, 48, 49, 55, 54]

m = np.mean(x)
n = np.size(x)
print('mean is',m,'\n n is',n)
```

mean is 50.666666666666664

n is 12

Using the `sample_mean=m` and `n` from the above code, we call `z_test_mean` with arguments: `47,5,50.666667,12,'two-sided'`. But by default the function is set to 'two-sided'

```
In [ ]: z_test_mean(47,5,50.666667,12) #you need to run the ttestmean function above first
```

z test statistic = 2.540341207528364 and p-value = 0.011074436944822352

Interpretation: The p-value is not less than $\alpha = 0.10$. Fail to reject the null hypothesis.

11.2 Hypothesis Testing for Mean μ (t-test)

[Watch demo video](#)

Suppose a random sample of size n is drawn from a population that is approximately normal with unknown mean μ and unknown standard deviation σ . To test a hypothesis about the mean μ of the form:

$$H_0 : \mu = \mu_0$$

$$H_a : \mu < \mu_0 \text{ smaller}$$

OR:

$$H_0 : \mu = \mu_0$$

$$H_a : \mu > \mu_0 \text{ greater}$$

OR:

$$H_0 : \mu = \mu_0$$

$$H_a : \mu \neq \mu_0 \text{ two-sided}$$

we use the following test-statistic (t-distribution)

$$t = \frac{\bar{x} - \mu_0}{\frac{s}{\sqrt{n}}}$$

We use the `t_test_mean` function with arguments: `null_mean =  $\mu_0$` , sample mean = `sample_mean`, sample standard deviation = `sample_std`, sample size `n` and `alternative='smaller'`, `'greater'` or `'two-sided'` for two sided test.

```
In [ ]: # t test with alternative being 'smaller' or 'greater' or 'two-sided'
def t_test_mean(null_mean,sample_mean,sample_std,n,alternative='two-sided'):
    from scipy import stats
    t = (sample_mean-null_mean)/(sample_std/n**.5)
    if alternative == 'smaller':
        pv = stats.t.cdf(t,n-1,0,1)
```



```

elif alternative == 'greater':
    pv = 1-stats.t.cdf(t,n-1,0,1)
elif alternative == 'two-sided':
    if t< 0:
        pv = 2*stats.t.cdf(t,n-1,0,1)
    else: pv = 2*(1-stats.t.cdf(t,n-1,0,1))
else:
    print("alternative option syntax error")
return print("t test statistic = ", t, " p-value = ",pv)

```

Example: College administrators believe that the midterm exam score for Social Studies students is 70. A Social Studies instructor thinks the mean score is **lower** than 70. He samples ten students and obtains the scores 65; 65; 70; 67; 66; 63; 63; 68; 72; 71 . Performs a hypothesis test using a 5% level of significance. The data are assumed to be from a normal distribution.

$$H_o : \mu = 70$$

$$H_a : \mu < 70$$

Use the `t_test_mean` function above. But first we have to find the sample mean and standard deviation from the data.

```

In [ ]: import numpy as np
x = [65, 65, 70, 67, 66, 63, 63, 68, 72, 71]
sample_mean = float(np.mean(x))
sample_std = float(np.std(x,ddof=1))
n = len(x)
print('sample mean is',sample_mean,'sample std is',sample_std,'n is',n)

```

sample mean is 67.0 sample std is 3.197221015541813 n is 10

Now call `t_test_mean` with arguments: 70, sample_mean=67.0, sample_std = 3.1972, n=10, 'left'. The variables sample_mean, sample_std and n are computed in the above code.

```

In [ ]: t_test_mean(70,67.0,3.1972,10,"smaller")    #you need to run the ttestmean function al

```

t test statistic = -2.9672316340876823 p-value = 0.007886584766274029

Interpretation: $p\text{-value} < \alpha = 0.05$, reject the null. Or compare the t test statistics with t critical `stats.t.ppf(.05/2,n-1,0,1)` = -2.26 (see code below)

```

In [ ]: from scipy import stats
stats.t.ppf(.05/2,10-1,0,1)

```

```

Out[ ]: -2.262157162740992

```

Example: A college administrator claims that the average grade point for all students in the college is greater than 3.0. A random sample of 20 students has a sample mean of 3.4 with standard deviation of 0.8. Assume average grade points are normally distributed. Test the college administrator's claim at α of 0.10. Use the `t_test_mean` function below.

```
In [ ]: # t test with alternative being 'smaller' or 'greater' or 'two-sided'
def t_test_mean(null_mean,sample_mean,sample_std,n,alternative='two-sided'):
    from scipy import stats
    t = (sample_mean-null_mean)/(sample_std/n**.5)
    if alternative == 'smaller':
        pv = stats.t.cdf(t,n-1,0,1)
    elif alternative == 'greater':
        pv = 1-stats.t.cdf(t,n-1,0,1)
    elif alternative == 'two-sided':
        if t < 0:
            pv = 2*stats.t.cdf(t,n-1,0,1)
        else: pv = 2*(1-stats.t.cdf(t,n-1,0,1))
    else:
        print("alternative option syntax error")
    return print("t test statistic = ", t, " p-value = ",pv)
```

```
null_mean=3,sample_mean=3.4,sample_std=0.8,n=20,alternative='greater'
```

```
In [ ]: t_test_mean(3,3.4,0.8,20,'greater')
```

```
t test statistic = 2.2360679774997894 p-value = 0.01877027477426252
```

Interpretation: Since the p-value 0.01877027477426252 is less than $\alpha = 0.1$, reject the null hypothesis. There is enough evidence to support the administrator's claim.

Proportions

The sample proportion is defined as the number of individuals in the sample with the specified characteristic, x , divided by the sample size, n .

$$\hat{p} = \frac{x}{n}$$

The population proportion is defined as the number of individuals in the population with the specified characteristic, x , divided by the population size, N .

$$p = \frac{x}{N}$$

Example: 20 students were asked if they were registered to vote. Their responses are listed below. Find the proportion of YES'.

```
NO, NO, YES, YES, NO, YES, NO, NO, YES, NO, YES, NO, NO, YES, YES, YES,
YES, NO, YES, YES
```

```
In [ ]: x = ['NO', 'NO', 'YES', 'YES', 'NO', 'YES', 'NO', 'NO', 'YES', 'NO',
            'YES', 'NO', 'NO', 'YES', 'YES', 'YES', 'YES', 'NO', 'YES', 'YES']
x_df = pd.DataFrame({'vote':x})
x_df.value_counts()
```

```
Out[ ]: vote
        YES      11
        NO       9
        dtype: int64
```

```
In [ ]: x_df.value_counts(normalize=True)
```

```
Out[ ]: vote
        YES      0.55
        NO      0.45
        dtype: float64
```

```
In [ ]: proportion = 11/20 # here yes is counted as "success"
        print("proportion is",proportion)
```

proportion is 0.55

11.3 Hypothesis Testing for Proportion P (z-test)

[Watch demo video](#)

Suppose a random sample of n (identical trials) is drawn from a population with probability of success p . The sample proportion, $\hat{p}$, has approximately normal distribution with mean p and standard deviation of $\sqrt{p(1-p)/n}$ for large n .

To test a hypothesis of the form:

$$H_0 : p = p_0$$

$$H_a : p < p_0 \text{ smaller}$$

OR:

$$H_0 : p = p_0$$

$$H_a : p > p_0 \text{ greater}$$

OR:

$$H_0 : p = p_0$$

$$H_a : p \neq p_0 \text{ two-sided}$$

use the following test-statistics (z-distribution)

$$z = \frac{\hat{p} - P_o}{\sqrt{\frac{P_o(1-P_o)}{n}}}$$

Hypothesis Testing for Proportions p (raw data given)

Hypothesis testing for the population proportion when raw data such as the sample size and number of success are given use the `z_test_prop` function below. This function needs the following arguments: `null_prop` the proportion from the null hypothesis, `x`, the number of "successes", `n` sample size, and `alternative` the direction of the alternative as "left", "right" or "two" for two sided test.

```
In [ ]: def z_test_prop(null_prop,x,n,alternative='two-sided'):
    from scipy import stats
    p = x/n
    z = (p-null_prop)/(null_prop*(1-null_prop)/n)**.5
    if alternative == 'smaller':
        pv = stats.norm.cdf(z,0,1)
    elif alternative == 'greater':
        pv = 1-stats.norm.cdf(z,0,1)
    elif alternative == 'two-sided':
        if z < 0:
            pv = 2*stats.norm.cdf(z,0,1)
        else: pv = 2*(1-stats.norm.cdf(z,0,1))
    else:
        print("alternative option syntax error")
    return print("z test statistic = ", z, " p-value = ",pv)
```

Example: The president believes that 65% of voters approve his job performance. A pollster performs a hypothesis test to determine if the percentage is the same or different from 65%. She samples 100 voters and 61 reply that they approve the president's performance. Test the hypothesis at 1% level of significance.

$$H_0 : p = 0.65 \text{ vs } H_a : p \neq 0.65$$

Use the `z_test_prop` function above with arguments:0.65,61,100,'two'

```
In [ ]: z_test_prop(0.65,61,100,'two-sided')
```

z test statistic = -0.8386278693775355 p-value = 0.4016781664697726

Interpretation: $p\text{-value} > \alpha = 0.01$, fail to reject the null

Example: The college president claims that the proportion of students registered to vote is more than 55%. A random sample of 20 students were asked if they are registered to vote. The results are listed below. Test the president's claim at 10% level of significance. NO, NO, YES, YES, NO, YES, NO, NO, YES, NO, YES, NO, NO, YES, YES, YES, YES, NO, YES, YES

Before we use the `z_test_prop` function, we need to find x (the number of success). We can use the `'x_df.value_count()'` to find x.

```
In [ ]: x = ['NO', 'NO', 'YES', 'YES', 'NO', 'YES', 'NO', 'NO', 'YES', 'NO',
            'YES', 'NO', 'NO', 'YES', 'YES', 'YES', 'YES', 'NO', 'YES', 'YES']
```

```
x_df = pd.DataFrame({'vote':x})
x_df.value_counts()
```

```
Out[ ]: vote
YES      11
NO        9
dtype: int64
```

$H_o : p = .55$ vs $H_a : p < 0.55$

use the `z_test_prop` with arguments: .55, 11, 20, 'right'

```
In [ ]: z_test_prop(.55,11,20,'greater')
```

```
z test statistic = 0.0 p-value = 0.5
```

p-value = 0.5 is not $< \alpha = 0.05$. Do not reject the null.

Hypothesis Testing for Proportions p (summary stats given)

Hypothesis testing when raw data such as sample size, proportion are given, use the

`z_test_stats` function below with arguments: null_prop, p, n, alternative.

```
In [ ]: # z proportion test ,altenative can only be "left" or "right" or "two"
```

```
def z_test_prop_stats(null_prop,p,n,alternative):
    from scipy import stats
    z = (p-null_prop)/(null_prop*(1-null_prop)/n)**.5
    if alternative == 'left':
        pv = stats.norm.cdf(z,0,1)
    elif alternative == 'right':
        pv = 1-stats.norm.cdf(z,0,1)
    elif alternative == 'two':
        if z < 0:
            pv = 2*stats.norm.cdf(z,0,1)
        else: pv = 2*(1-stats.norm.cdf(z,0,1))
    else:
        print("altrnative option syntax error")
    return print("z test statistic = ", z, "and p-value = ",pv)
```

Example: The college president claims that the proportion of students registered to vote is more than 60%. A random sample of 50 students were asked if they are registered to vote. 62% were registered voters. Test the president's claim at 5% level of confidence.

$H_o : p = 0.60$ vs $H_a : p > 0.60$

Use the `z_test_prop_stats` function with arguments:

null_prop=.60,p=.62,n=50,alternative='right'

```
In [ ]: z_test_prop_stats(.6,.62,50,'right')
```

```
z test statistic = 0.28867513459481314 and p-value = 0.38641499634222365
```

Interpretation: p-value is not less than $\alpha = 0.05$. Fail to reject the null hypothesis.

Example: The mayor of a city believes that more than 60% of the residents are fully vaccinated for Covid19. In a survey of 120 residents, 81 were fully vaccinated. Test the mayor's claim at α level of 0.05. Use the `z_test_prop` function below.

```
In [ ]: def z_test_prop(null_prop,x,n,alternative): #altenative can only be "smaller"
                                                # or "greater" or "two-sided"

    from scipy import stats
    p = x/n
    z = (p-null_prop)/(null_prop*(1-null_prop)/n)**.5
    if alternative == 'smaller':
        pv = stats.norm.cdf(z,0,1)
    elif alternative == 'greater':
        pv = 1-stats.norm.cdf(z,0,1)
    elif alternative == 'two-sided':
        if z < 0:
            pv = 2*stats.norm.cdf(z,0,1)
        else: pv = 2*(1-stats.norm.cdf(z,0,1))
    else:
        print("altrnative option syntax error")
    return print("z test statistic = ", z, " p-value = ",pv)
```

```
z_test_prop(null_prop,x,n,alternative) with
null_prop=.6,x=81,n=120,alternative='greater'
```

```
In [ ]: z_test_prop(.6,81,120,'greater')
```

```
z test statistic = 1.6770509831248437 p-value = 0.04676625634454645
```

Interpretation: Since the p-value 0.04676625634454645 is less than α of 0.05, reject the null hypothesis. There is enough evidence to support the mayor's claim.

12. Linear Regression and Correlation

How do we measure a relationship between two quantitative variables? For example, what is the relationship between students' time spent studying for a test and their test scores? In general, the more time spent studying (more hours) would yield a higher test score. There is a direct or positive relationship between time spent and test scores. But how strong is the relationship?

Another example is the relationship between the age (years) of a car and its price (in dollars). In general, the older the car (more years) the cheaper (less dollars) it costs. The relationship is indirect or negative. What type and how strong is the relationship?

12.1 Scatter plot

One way to check for a relationship between two variables is to draw a scatter plot. One of the variables (independent/explanatory) can be on the x-axis and the dependent(response) on the y-axis. To draw a scatterplot of two variables x and y use the following code

```
import matplotlib.pyplot as plt
x = [x1,x2,...]
y = [y1,y2,...]
plt.scatter(x, y)
```

12.2 The Pearson Correlation Coefficient

The (Pearson) correlation coefficient is a measure of the strength and direction of the *linear* relationship between two variables.

$\rho = \frac{cov(x,y)}{\sigma_x \sigma_y}$ for population correlation coefficient

$r = \frac{cov(x,y)}{s_x s_y}$ or $r = \frac{n \sum xy - \sum x \sum y}{\sqrt{n \sum x^2 - (\sum x)^2} \sqrt{n \sum y^2 - (\sum y)^2}}$ for sample correlation coefficient

Where ρ and r are the population and sample Pearson correlation coefficients respectively. $cov(x, y)$ is the covariance of the two variables.

The value of r is between -1 and 1 and it is unitless. That is, it is not affected by the unit of measurements of both variables.

1. if r or ρ is close to +1, we conclude strong positive relationship
2. If r or ρ is close to -1, we conclude strong negative relationship
3. If r or ρ is close to zero, we conclude weak or no relationship

To compute the correlation coefficient, you can use of the following two options.

```
from scipy import stats
stats.linregress(x, y) # rvalue the 3rd from the output
```

or

```
import numpy as np
x = [x1,x2,...]
y = [y1,y2,...]
np.corrcoef(x,y)
```

returns a matrix form of correlations between x and x, x and y, y and x, y and y

Example: The midterm exam out of a total of 100 points, x, and final exam out of a total of 200 points, y, scores for a random sample of 11 students is listed below. Find the correlation

coefficient and draw a scatter plot.

x	65	67	71	71	66	75	67	70	71	69	69
y	145	133	185	163	126	198	153	163	159	151	159

[Watch demo video 1](#) Scatter Plot

[Watch demo video 2](#) Correlation Coefficient

```
In [ ]: # Pearson Correlation Coefficient
import numpy as np
x = [65, 67, 71, 71, 66, 75, 67, 70, 71, 69, 69]
y = [145, 133, 185, 163, 126, 198, 153, 163, 159, 151, 159]
np.corrcoef(x,y)
```

```
Out[ ]: array([[1.          , 0.87974511],
               [0.87974511, 1.          ]])
```

Interpretation: Because $r=0.879745$ is positive and close to one, we can conclude that there is a strong and direct relationship between x midterm scores and y final exam scores.

Note: The correlation coefficient between x and x is 1. Similarly between y and y is 1.

```
In [ ]: x = [65, 67, 71, 71, 66, 75, 67, 70, 71, 69, 69]
y = [145, 133, 185, 163, 126, 198, 153, 163, 159, 151, 159]

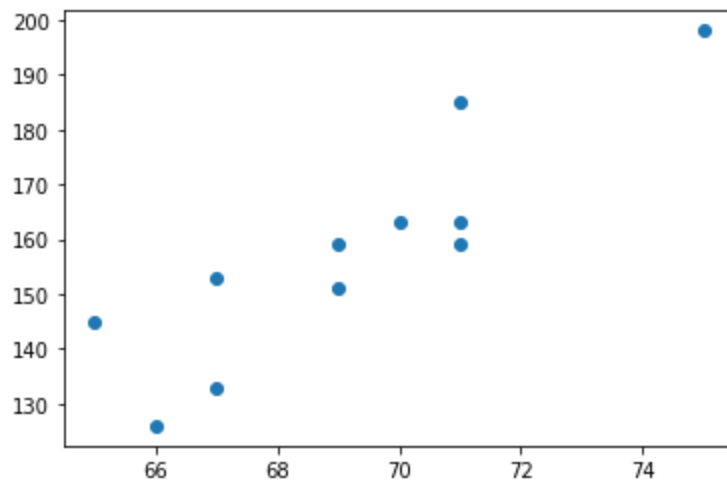
from scipy import stats
stats.linregress(x, y)
```

```
Out[ ]: LinregressResult(slope=6.364142538975501, intercept=-282.5556792873051, rvalue=0.879
7451071731136, pvalue=0.00035593568153235875, stderr=1.1464698617902889)
```

Interpretation: for now we only need the third output $rvalue=0.8797451071731136$

```
In [ ]: # scatterplot
import matplotlib.pyplot as plt
x = [65, 67, 71, 71, 66, 75, 67, 70, 71, 69, 69]
y = [145, 133, 185, 163, 126, 198, 153, 163, 159, 151, 159]
plt.scatter(x, y)
```

```
Out[ ]: <matplotlib.collections.PathCollection at 0x7f9c4241aa10>
```

There is a positive or direct relationship.

Example: The number of hours studied, x , and test scores, y , for 10 students are listed below. Find the correlation coefficient and display the scatter plot.

x	2	0	3	5	4	7	4	6	5	1
y	60	55	65	69	70	90	71	88	79	51

```
In [ ]: import numpy as np
```

```
x=[2,0,3,5,4,7,4,6,5,1]
y=[60,55,65,69,70,90,71,88,79,51]

np.corrcoef(x,y)
```

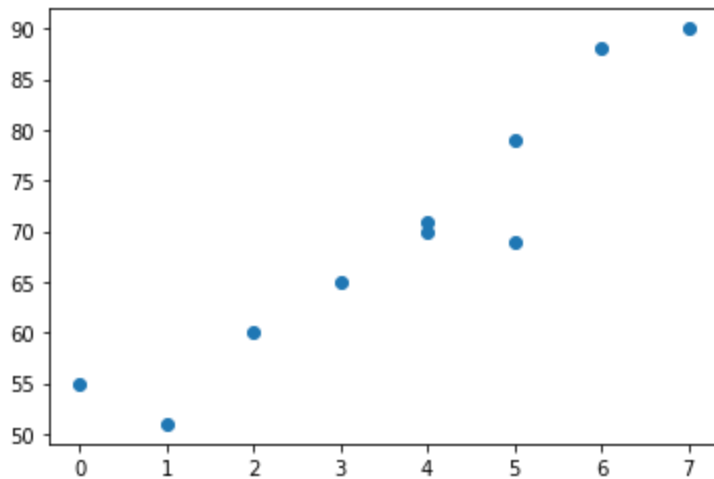
```
Out[ ]: array([[1.          , 0.94858577],
               [0.94858577, 1.          ]])
```

```
In [ ]: import matplotlib.pyplot as plt
```

```
x=[2,0,3,5,4,7,4,6,5,1]
y=[60,55,65,69,70,90,71,88,79,51]

plt.scatter(x,y)
```

```
Out[ ]: <matplotlib.collections.PathCollection at 0x7fa3da036dd0>
```



12.3 Linear Regression

[Watch demo video](#)

We can use the variables x and y to **model** a linear relationship between x and y with a linear equation of the form $y = b + mx$.

The Least-Squares method can be used to obtain the predicted y values $\hat{y}$ as follows:

$$\hat{y} = a + bx$$

Where

$$b = \text{slope} = r \times \frac{s_y}{s_x} \text{ and } a = \bar{y} - b\bar{x}$$

Where s_y and s_x are std for x and y . The following code can be used to obtain the slope and intercept.

```
from scipy import stats
stats.linregress(x, y)
```

Example: The midterm exam out of a total of 100 points, x , and final exam out of a total of 200 points, y , scores for a random sample of 11 students is listed below. Find the slope and intercept of the regression line. Test if the correlation between x and y is significant at 0.05 level of significance.

x	65	67	71	71	66	75	67	70	71	69	69
y	145	133	185	163	126	198	153	163	159	151	159

```
In [ ]: x = [65, 67, 71, 71, 66, 75, 67, 70, 71, 69, 69]
y = [145, 133, 185, 163, 126, 198, 153, 163, 159, 151, 159]
from scipy import stats
stats.linregress(x, y)
```

```
Out[ ]: LinregressResult(slope=6.364142538975501, intercept=-282.5556792873051, rvalue=0.8797451071731136, pvalue=0.00035593568153235875, stderr=1.1464698617902889)
```

Interpretation: slope=6.364142538975501, intercept=-282.5556792873051, rvalue=0.8797451071731136 (this is Pearson correlation coefficient), pvalue=0.00035593568153235875 (this p-value is for testing the significance of the correlation coefficient), stderr=1.1464698617902889

Since the p-value is less 0.05, there is significant correlation between x and y.

slope=6.364142538975501, intercept=-282.5556792873051 .

Thus, $\hat{y} = -282.56 + 6.36x$ is the regression line with values rounded to two decimal places.

Example: the number of hours studied, x, and test scores, y, for 10 students are listed below. Find the correlation coefficient (r-value), estimated slope and y-intercept of the regression line and the p-value for testing the significance of correlation.

x	2	0	3	5	4	7	4	6	5	1
y	60	55	65	69	70	90	71	88	79	51

```
In [ ]: from scipy import stats

x=[2,0,3,5,4,7,4,6,5,1]
y=[60,55,65,69,70,90,71,88,79,51]

stats.linregress(x,y)
```

```
Out[ ]: LinregressResult(slope=5.564625850340137, intercept=49.21088435374149, rvalue=0.9485857685252971, pvalue=2.8725087369610658e-05, stderr=0.6564713967897522)
```

Interpretation:

slope=5.564625850340137, intercept=49.21088435374149, r-value=0.9485857685252971, pvalue=2.8725087369610658e-05

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np

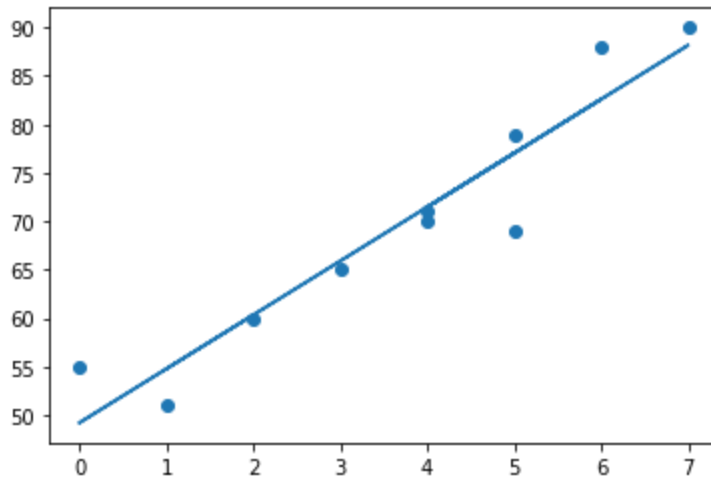
x=[2,0,3,5,4,7,4,6,5,1]
y=[60,55,65,69,70,90,71,88,79,51]

x = np.array(x)
y = np.array(y)

y_hat = 5.564625850340137*x + 49.21088435374149

plt.scatter(x,y)
plt.plot(x,y_hat)
```

```
Out[ ]: [matplotlib.lines.Line2D at 0x7fa3d937f8d0]
```



Regression Line

The function `lin_reg(x,Y)` below provides the slope, y-intercept of the least-squares regression line. In addition it returns the R-square, p-value, t-test statistic for testing the significance of the correlation coefficient.

```
In [ ]: def lin_reg(x,y):
    import statsmodels.api as sm
    import pandas as pd

    x_df = pd.DataFrame({'indpt':x})

    # to get intercept
    X = sm.add_constant(x_df)

    # fit the regression model
    model = sm.OLS(y, X)
    results = model.fit()
    print()
    if results.params[1]>0:
        r = results.rsquared**.5
    else: r = -1*results.rsquared**.5
    print('corr coeff r: ', r)
    print('R squared:', results.rsquared)
    print()
    print('estimated slope:', results.params[1])
    print('estimated y-intercept:', results.params[0])
    print()
    print('t test statistic:', results.tvalues[1])
    print('p-value:', results.f_pvalue)
    print()

    import matplotlib.pyplot as plt
    y_hat = results.predict(X)
    fig, ax = plt.subplots(figsize=(8,6))
    ax.plot(x, y, 'o', label="observed")
```

```
ax.plot(x, y_hat, 'b-', label="reg line")
ax.set_xlabel('indpt')
ax.set_ylabel('dept')
ax.legend(loc='best');
```

Example: The midterm exam, x, and final exam, y, scores for a random sample of 11 students is listed below. Find the slope and intercept of the regression line, correlation coefficient and draw a scatter plot.

x	65	67	71	71	66	75	67	70	71	69	69
y	145	133	185	163	126	198	153	163	159	151	159

```
In [ ]: x = [65, 67, 71, 71, 66, 75, 67, 70, 71, 69, 69]
        y = [145, 133, 185, 163, 126, 198, 153, 163, 159, 151, 159]
```

```
In [ ]: lin_reg(x,y)
```

/usr/local/lib/python3.7/dist-packages/statsmodels/tools/_testing.py:19: FutureWarning:

pandas.util.testing is deprecated. Use the functions in the public API at pandas.testing instead.

corr coeff r: 0.8797451071731136

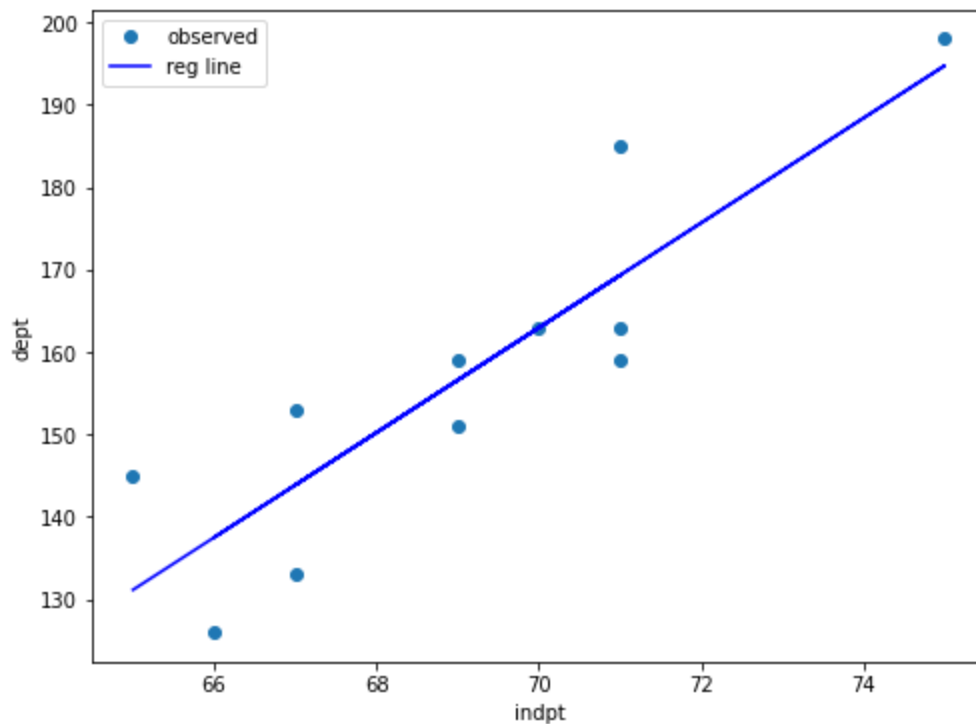
R squared: 0.7739514535950331

estimated slope: 6.364142538975502

estimated y-intercept: -282.5556792873052

t test statistic: 5.55107705058855

p-value: 0.0003559356815323585



Interpretation: $r=0.8797451071731136$ indicates strong positive linear relationship. R squared: 0.7739514535950331 indicates 77.4% of the variation can be explained by the linear model. p-value: 0.0003559356815323585 indicates the correlation coefficient is significant.

Predictions

Suppose you want to estimate, or predict, the final exam score of a student with midterm exam score of 70. The midterm exam scores (x-values) range from 65 to 75. Since 70 is inside this range, we can substitute $x = 70$ into the regression equation to find the predicted y value.

$$\hat{y} = a + bx$$

from the results (see above)

$$\hat{y} = -282.5556792873052 + 6.364142538975502x$$

$$\hat{y} = -282.5556792873052 + 6.364142538975502 \times 70$$

```
In [ ]: -282.5556792873052 + 6.364142538975502*70
```

```
Out[ ]: 162.9342984409799
```

A student with a midterm exam score of 70 is predicted to score 162.9 on the final exam.

13. Analysis of Categorical Data

Qualitative data can be classified consisting of counts or number of observations that fall into each category.

13.1 Goodness of Fit Test

[Watch demo video](#)

Categorical or qualitative data can be organized by frequency distribution of a list of each category and the corresponding counts or relative frequencies.

A Chi-square goodness-of-fit is used to test whether a frequency distribution follows a given specific distribution.

H_o : The random variable follows a certain given distribution or (the probability for each category, P_i , as given is *true*).

H_a : The random variable does not follow the distribution in the null hypothesis

The Chi-square test statistic is given as:

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$$

Where

O_i is the observed value for category i

$E_i = nP_i$ is the expected value for category i assuming H_o is true. n is sample size.

$$df = k - 1$$

For large sample sizes χ^2 is approximately follows the **chi-square probability distribution**

Note: In order to perform the Chi-Square goodness of fit, we assume:

- samples are randomly selected
- sample data consist of frequency counts for each category
- For each category, the expected frequency is at least 5

For chi-square goodness of fit test, use the following syntax with observed frequency stored in `obs` and expected frequency stored in `exp` :

```
from scipy import stats
stats.chisquare(obs,exp)
```

returns `Power_divergenceResult(statistic, pvalue)`

Example: A store manager claims that a plain M&M bag of candies contains an equal number of each of the six colors: brown, yellow, red, blue, orange, and green. The counts for each color of a randomly selected bag of plain M&M is given below. Test the store managers claim at 5% level of significance.

color	frequency
brown	57
yellow	64
red	54
blue	75
orange	86
green	64

The above table contains the observed (O_i) frequencies. We need to find the expected frequencies before we call the function. If the manager's claim is true there will be equal number of candies for each color. We need the total number of candies and divide it to 6 to get the expected frequencies.

```
In [ ]: obs = [57,64,54,75,86,64]
```

```
In [ ]: sum(obs) #total number of candies in the bag
```

```
Out[ ]: 400
```

If the null is true, we expect an equal number in each category. That is, $P_i = \frac{1}{6}$ for all i .
 $E_i = nP_i = 400/6$

color	observed	expected
brown	57	400/6
yellow	64	400/6
red	54	400/6
blue	75	400/6
orange	86	400/6
green	64	400/6

```
In [ ]: exp = [400/6,400/6,400/6,400/6,400/6,400/6]
```

```
In [ ]: obs = [57,64,54,75,86,64]
```



```
In [ ]: from scipy import stats

obs = [57,64,54,75,86,64]
exp = [400/6,400/6,400/6,400/6,400/6,400/6]

stats.chisquare(obs,exp)
```

```
Out[ ]: Power_divergenceResult(statistic=10.67, pvalue=0.05833003695944934)
```

Interpretation: The Chi-square test statistic is equal to 10.67 . At the p-value is not less than 0.05 . We would fail to reject the null hypothesis.

Example: The number of sales of electric vehicles (EV) in the USA is given in the table below. Test a claim that the distribution of sales is uniform with $\alpha = 0.05$.

Brand	Sales
Ford	2400000
Toyota	2200000
Chevrolet	2020000
Honda	1500000

```
In [ ]: import numpy as np
from scipy import stats

obs = [2400000,2200000,2020000,1500000] # observed frequency
sum = np.sum(obs) # sum of observed freq
exp = [sum/4,sum/4,sum/4,sum/4] # expected freq

stats.chisquare(obs,exp)
```

```
Out[ ]: Power_divergenceResult(statistic=220098.52216748768, pvalue=0.0)
```

Interpretation: The chi-square test statistics is 220098.52216748768 and the p-value 0.0. Reject the null hypothesis. There is enough evidence to conclude that the distribution is not uniform.

13.2 Test of Independence

[Watch demo video](#)

Sometimes, categorical data can be organized by a two-way classification or contingency table. Each cell (intersection of row and column) is the observed count.

H_o : The row and column variables are independent

H_a : The row and column variables are dependent

The Chi-square test statistic is given by:

$$\chi^2 = \sum_{i,j=1}^{rc} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

where

r and c are the number of rows and columns respectively

$E_{ij} = \frac{r_i c_j}{n}$ where r_i, c_j row i , column j totals.

$$df = (r - 1)(c - 1)$$

Note: In order to perform the Chi-Square test of independence, we assume:

- samples are randomly selected
- sample data consist of frequency counts for each two-way table
- For each cell, the expected frequency is at least 5

Use the function `chi2_test_of_indp` below with argument: each column from the observed frequency combined as `obs = [c1, c2, ...]`. Where `c1` is the list from column 1, `c2` is the list from column 2 ...

```
In [ ]: def chi2_test_of_indp(obs):  
        from scipy import stats  
        chi2, p, dof, ex = stats.chi2_contingency(obs)  
  
        print("chi-square test statistic:", chi2)  
        print("p-value:", p)
```

Example: The contingency table below shows a random sample of white, black and Hispanic college students based on whether their family borrowed money to pay their college education. Test the hypothesis that borrowing money for college and race are independent at 1% level of significance. (adapted from Larson and Farber)

race/borrow	Yes	No
White	49	64
Black	85	123
Hispanic	85	180

H_0 : Borrowing money for college and race are independent

H_a : Borrowing money for college and race are dependent

```
In [ ]: c1 = [49, 85, 85]  
        c2 = [64, 123, 180]  
  
        obs = [c1, c2]
```

```
In [ ]: chi2_test_of_indp(obs)
```

```
chi-square test statistic: 5.993527985414218  
p-value: 0.04994844064430499
```

Swapping the column and row still gives the same result (see below).

```
In [ ]: c1 = [49,64]  
c2 = [85,123]  
c3 = [85,180]  
  
obs = [c1,c2,c3]  
chi2_test_of_indp(obs)
```

```
chi-square test statistic: 5.993527985414217  
p-value: 0.04994844064430501
```

At $\alpha = 0.05$, the **p-value** is less than .05. Reject the null hypothesis that race and borrowing college money are independent.

Example: The contingency table below shows the effects of Covid19 by race in the USA. Test the hypothesis that effects of Covid19 is independent of race at 5% level of significance.

Covid19\Race	White	Native	Asian	Black	Hispanic	Cases
Hospitalization	100	170	70	110	190	640
Death	100	340	100	280	280	1000

Use the **chi2_test_of_indp** function below.

```
In [ ]: def chi2_test_of_indp(obs):  
        from scipy import stats  
        chi2, p, dof, ex = stats.chi2_contingency(obs)  
  
        print("chi-square test statistic:",chi2)  
        print("p-value:", p)
```

```
In [ ]: r1 = [100,170,70,110,190]  
r2 = [100,340,100,280,280]  
r3 = [100,240,100,200,230]  
  
obs = [r1,r2,r3]  
  
chi2_test_of_indp(obs)
```

```
chi-square test statistic: 36.38048143358525  
p-value: 1.4955960363094355e-05
```

Interpretation: Since the p-value 1.4955960363094355e-05 is less than 5%, reject the null hypothesis. That is, there is enough evidence to reject the claim that the effects of Covid10 and race are independent.

14. Advanced Statistics Topics

14.1 Descriptive Statistics with Pandas

[Watch demo video](#)

Pandas Library

Pandas is another Python software library that is very useful for data analysis. Pandas is typically imported with alias `np`.

Pandas DataFrame

The Pandas DataFrame function is used for entering the data values in a *spreadsheet* like format with rows and columns labeled 0, 1, 2,... by default. Pandas output is typically displayed as DataFrame too.

Note: We will use the variable name `x` to store dataset in *Python List*. We will use the variable name `x_df` to store dataset in *Pandas DataFrame*.

To convert the Python list `x` to pandas dataframe `x_df` and display the results:

1. Click `+ Code`
2. Type `import pandas as pd`
3. Type `x = [x1, x2, ...]`
4. Type `x_df = pd.DataFrame({'column_name':x})`
5. Type `x_df`
6. Click `play` button

Warning: Like all Python variables `x_df` will only contain dataset that was stored last. Old values are replaced(updated) by the last data values stored.

Example: The dataset below is the waiting time, in minutes, of customers at a fast food restaurant. Write a pandas program to store and display the dataset in DataFrame.

```
5, 3, 4, 1, 5, 3, 5, 2, 3, 2, 5, 4, 4, 6, 4, 3, 4, 1, 2, 5
```

1. Click `+ Code`
2. Type `import pandas as pd`
3. Type `x = [5, 3, 4, 1, 5, 3, 5, 2, 3, 2, 5, 4, 4, 6, 4, 3, 4, 1, 2, 5]`
4. Type `x_df = pd.DataFrame({'waiting_time':x})`
5. Type `print(x_df)`
6. Click `play` button

```
In [ ]: import pandas as pd
x = [5, 3, 4, 1, 5, 3, 5, 2, 3, 2, 5, 4, 4, 6, 4, 3, 4, 1, 2, 5]
x_df = pd.DataFrame({'waiting_time':x})
print(x_df)
```

	waiting_time
0	5
1	3
2	4
3	1
4	5
5	3
6	5
7	2
8	3
9	2
10	5
11	4
12	4
13	6
14	4
15	3
16	4
17	1
18	2
19	5

Note: Unlike Numpy, Pandas DataFrame output is given in a spreadsheet like format. It has both column(s) and row(s). The first column is called the `index` column and it is automatically generated by Python. In the output above, the first column `0, 1, 2, ..., 19` is the `index` column. **Make a special note that it does NOT start at 1. It starts at 0.** `waiting_time` is the name of the second column after the `index` column. We only have one column with dataset here: `waiting_time`. The variable name for the whole DataFrame, including the columns and rows, is `x_df`.

Numpy vs. Pandas

Both numpy and pandas can be used for most computations in descriptive statistics. Unlike numpy, Pandas requires converting the dataset list to Pandas dataframe before any computation or analysis. But Pandas has more statistical computing and plotting capabilities.

Value Counts and Relative Frequency using Pandas

1. Pandas syntax for frequency

```
import pandas as pd # pd alias for pandas
x=[x1,x2,x3,...]
x_df=pd.DataFrame({'column_name':x})
x_df.value_counts()
```

The above returns counts of unique values (frequencies).

2. Pandas syntax for a relative frequency

```
import pandas as pd
x=[x1,x2,x3,...]
x_df=pd.DataFrame({'column_name':x})
x_df.value_counts(normalize=True)
```

The above returns a relative frequency of the unique values (Frequency $\hat{f}$ · Total Frequency)

Example: The dataset below is the waiting time, in minutes, of customers at a fast food restaurant. Write a pandas program to construct a frequency distribution.

5, 3, 4, 1, 5, 3, 5, 2, 3, 2, 5, 4, 4, 6, 4, 3, 4, 1, 2, 5

1. Click + Code
2. Type `import pandas as pd`
3. Type `x= [5, 3, 4, 1, 5, 3, 5, 2, 3, 2, 5, 4, 4, 6, 4, 3, 4, 1, 2, 5]`
4. Type `x_df = pd.DataFrame(x,columns = ['waiting_time'])`
5. Type `x_df.value_counts(sort=False)`
6. Click play button

```
In [ ]: import pandas as pd
x= [5, 3, 4, 1, 5, 3, 5, 2, 3, 2, 5, 4, 4, 6, 4, 3, 4, 1, 2, 5]
x_df = pd.DataFrame(x,columns = ['waiting_time'])
x_df.value_counts(sort=False)    # if sort is True it will sort by frequency
```

```
Out[ ]: waiting_time
1          2
2          3
3          4
4          5
5          5
6          1
dtype: int64
```

Interpretation from the above output:

waiting time	frequency
1	2
2	3
3	4
4	5
5	5
6	1

Example: The dataset below is the waiting time, in minutes, of customers at a fast food restaurant. Write a pandas program to construct a relative frequency distribution.

```
5, 3, 4, 1, 5, 3, 5, 2, 3, 2, 5, 4, 4, 6, 4, 3, 4, 1, 2, 5
```

```
In [ ]: import pandas as pd

x = [5, 3, 4, 1, 5, 3, 5, 2, 3, 2, 5, 4, 4, 6, 4, 3, 4, 1, 2, 5]
x_df = pd.DataFrame({'waiting_time':x})
x_df.value_counts(normalize=True, sort=False)    # add normalize=True for relative fi
```

```
Out[ ]: waiting_time
1          0.10
2          0.15
3          0.20
4          0.25
5          0.25
6          0.05
dtype: float64
```

Interpretation from the above output

waiting time	rel-freq
1	0.10
2	0.15
3	0.20
4	0.25
5	0.25
6	0.05

Example: 20 students were asked if they were registered to vote. Their responses are given below. Write a program to construct a frequency and relative frequency distribution using pandas library.

```
NO, NO, YES, YES, NO, YES, NO, NO, YES, NO, YES, NO, NO, YES, YES, YES,
YES, NO, YES, YES
```

```
In [ ]: # non-numerical values should be in 'quotes'
x = ['NO', 'NO', 'YES', 'YES', 'NO', 'YES', 'NO', 'NO', 'YES', 'NO',
      'YES', 'NO', 'NO', 'YES', 'YES', 'YES', 'YES', 'NO', 'YES', 'YES']
x_df = pd.DataFrame({'vote':x})
x_df.value_counts()
```

```
Out[ ]: vote
YES      11
NO        9
dtype: int64
```

Interpretation from the above output

vote

Yes	NO
11	9

```
In [ ]: # non-numerical values should be in 'quotes'
x = ['NO', 'NO', 'YES', 'YES', 'NO', 'YES', 'NO', 'NO', 'YES', 'NO',
      'YES', 'NO', 'NO', 'YES', 'YES', 'YES', 'YES', 'NO', 'YES', 'YES']
x_df = pd.DataFrame({'vote':x})
x_df.value_counts(normalize=True)
```

```
Out[ ]: vote
YES      0.55
NO       0.45
dtype: float64
```

Interpretation from the above output

Yes	No
0.55	0.45

Descriptive Statistics with Pandas

Pandas syntax for the mean:

```
import pandas as pd
x = [x1,x2,x3,...,xn]
x_df = pd.DataFrame({'column_name':x})
x_df.mean()
```

Example: Find the mean of the data set `1,2,3` using pandas.

```
In [ ]: import pandas as pd
x = [1,2,3]
x_df = pd.DataFrame({'x':x})
x_df.mean()
```

```
Out[ ]: x      2.0
dtype: float64
```

Interpretation: the mean is 2.0

Pandas syntax for the median:

```
import pandas as pd
x = [x1,x2,x3,...,xn]
x_df = pd.DataFrame(x)
x_df.median()
```

Example: find the median of the data set `1,2,3,4`


```
In [ ]: import pandas as pd
x = [1,2,3,4]
x_df = pd.DataFrame({'x':x})
x_df.median()
```

```
Out[ ]: x      2.5
dtype: float64
```

Interpretation: the median is 2.5

Mode

The mode(s) of a data set is the most frequent value(s).

pandas syntax for mode

```
import pandas as pd
x = [x1,x2,x3,...]
x_df = pd.DataFrame(x)
x_df.mode()
```

Example: find the mode of the data set 1,2,3,3,4,4

```
In [ ]: import pandas as pd
x = [1,2,3,3,4,4]
x_df = pd.DataFrame({'x':x})
x_df.mode()
```

```
Out[ ]:    x
0  3
1  4
```

Interpretation: There are two modes here, 3 and 4. It is bimodal. Recall that the columns and rows in pandas are 0, 1, 2,3,...

In this case, there is one column x and two rows 0 and 1.

Example: The dataset below is scores of 20 students in a Math class. Find the mean, median and mode.

```
81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52, 85,
28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19
```

```
In [ ]: import pandas as pd
x = [81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40,
      32, 52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]
x_df = pd.DataFrame({'score':x})
```

```
print("The mean",x_df.mean())
print('the median',x_df.median())
print('the mode',x_df.mode())
```

```
The mean score    42.866667
dtype: float64
the median score   36.0
dtype: float64
the mode    score
0         36
```

Interpretation: The mean is 42.866667, the median is 36.0 and the mode is 36

Example: The dataset below is scores of 21 students in a Math class. Find mode.

```
17, 81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52,
85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19
```

```
In [ ]: import pandas as pd
x = [17, 81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40,
      32, 52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]
x_df = pd.DataFrame({'score':x})

print("Mode",x_df.mode())
```

```
Mode    score
0       17
1       36
```

There are two modes here: 17 and 36.

pandas syntax for variance and std:

```
import pandas as pd
x = [x1,x2,x3,...]
x_df = pd.DataFrame({'column_name':x})
x_df.var(ddof=0)      #pop variance
x_df.var()            #sample variance
x_df.std(ddpf=0)      #pop std
x_df.std()            #sample std
```

Note: ddof=1 by default for pandas

Example: find the population variance of the data set 1,2,3,4,5

1. Click + Code
2. Type `import pandas as pd`
3. Type `x = [1,2,3,4,5]`
4. Type `x_df =pd.DataFrame(x)`
5. Type `x_df.var(ddof=0)`
6. Click play

```
In [ ]: import pandas as pd
x = [1,2,3,4,5]
x_df =pd.DataFrame(x)
x_df.var(ddof=0)           #in Pandas ddof=1 by default
```

```
Out[ ]: 0    2.0
dtype: float64
```

Interpretation: the population variance is 2.0

Example: find the sample variance of the data set 1,2,3,4,5

```
In [ ]: import pandas as pd
x = [1,2,3,4,5]
x_df =pd.DataFrame(x)
x_df.var()
```

```
Out[ ]: 0    2.5
dtype: float64
```

Interpretation: the sample variance is 2.5

Example: find the population standard deviation of the data set 1,2,3,4,5

```
In [ ]: import pandas as pd
x = [1,2,3,4,5]
x_df =pd.DataFrame(x)
x_df.std(ddof=0)
```

```
Out[ ]: 0    1.414214
dtype: float64
```

Interpretation: the population standard deviation is 1.414214

Example: find the sample standard deviation of the data set 1,2,3,4,5

```
In [ ]: import pandas as pd
x = [1,2,3,4,5]
x_df =pd.DataFrame(x)
x_df.std()
```

```
Out[ ]: 0    1.581139
dtype: float64
```

Interpretation: the sample standard deviation is 1.581139

Example: The dataset below is scores of 20 students in a Math class. Find the sample standard deviation and variance.

```
81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52, 85,
28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19
```

```
In [ ]: import pandas as pd

x = [81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32,
      52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]

x_df = pd.DataFrame({'score':x})

x_df.std()           #sample standard deviation
```

```
Out[ ]: score    23.555852
dtype: float64
```

In the above output, 23.555852 is the sample standard deviation.

```
In [ ]: import pandas as pd

x = [81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32,
      52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]

x_df = pd.DataFrame({'score':x})

x_df.var()           #sample variance
```

```
Out[ ]: score    554.878161
dtype: float64
```

Interpretation: the sample variance is 554.878161

Example: The dataset below is scores of 20 students in a Math class. Find the population standard deviation and variance.

```
81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52, 85,
28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19
```

```
In [ ]: import pandas as pd

x = [81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32,
      52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]

x_df = pd.DataFrame({'score':x})
```

```
x_df.std(ddof=0)      # population standard deviation ddof degrees of freedom
```

```
Out[ ]: score      23.159927
dtype: float64
```

In the above output, 23.159927 is the population standard deviation.

```
In [ ]: import pandas as pd

x = [81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32,
      52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]

x_df = pd.DataFrame({'score':x})

x_df.var(ddof=0)      # population variance
```

```
Out[ ]: score      536.382222
dtype: float64
```

Interpretation: The population variance is 536.382222

Percentiles

pandas syntax for percentile

```
import pandas as pd
x_df.quantile(p)
```

Example: The dataset below is scores of 20 students in a Math class.

```
81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52, 85,
28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19
```

- Find the 70th percentile.

Note: These results by pandas could be different from other results that use a different method of computing percentiles.

```
In [ ]: import pandas as pd
x=[81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32,
    52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]
x_df = pd.DataFrame({'score':x})
x_df.quantile(.70)
```

```
Out[ ]: score      50.6
Name: 0.7, dtype: float64
```

Interpretation: The 70th percentile is approximately 50.6

Note: In the output above, `score` is the name of the variable/column, `50.6` is the calculated 70th percentile, and `dtype: float64` tells us that this result `50.6` is of type `float64` is a double precision float.

Example: The dataset below is scores of 20 students in a Math class.

```
81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52, 85,
28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19
```

- Find the 83rd percentile.

Note: These results by pandas could be different from other results that use a different method of computing percentiles.

```
In [ ]: import pandas as pd
x=[81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32,
    52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]
x_df = pd.DataFrame({'score':x})
x_df.quantile(.83)
```

```
Out[ ]: score      79.07
Name: 0.83, dtype: float64
```

Interpretation: The 83rd percentile is 79.07

Example: The dataset below is scores of 20 students in a Math class.

```
81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52, 85,
28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19
```

1. Find the 70th percentile.
2. Find the 83rd percentile.
3. Find the 1st quartile.
4. Find the 3rd quartile.

Note: These results by pandas could be different from other results that use a different method of computing percentiles.

```
In [ ]: import pandas as pd
x=[81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32,
    52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]
x_df = pd.DataFrame({'score':x})
x_df.quantile([.7,.83,.25,.75]) # this will print all of them at once
```

```
Out[ ]:
```

	score
0.70	50.60
0.83	79.07
0.25	25.25
0.75	52.75

Interpretation: In the output above, the first column is the percentiles given in decimals and the second column is the corresponding values. For example, **0.70** 50.60 tells us the **70th** percentile is **50.60**.

Percentile	score
70th	50.60
83rd	79.07
25th	25.25
75th	52.75

Summary Statistics, Skewness and Kurtosis

use `x_df.describe()` to get summary stats of a data. This code returns: size(count), mean, std, min, 1st, 2nd, 3rd quartile, max.

use `x_df.skew()` for skewness and `x_df.kurtosis()` for kurtosis.

Skewness is a measure of symmetry/asymmetry of the shape and kurtosis is a measure of heavy/light tail.

Example: The dataset below is scores of 20 students in a Math class. Use the `x_df.describe()` to find summary stats.

```
81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32, 52, 85,
28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19
```

```
In [ ]: import pandas as pd
x=[81, 21, 53, 32, 80, 13, 26, 42, 35, 41, 14, 25, 87, 62, 46, 40, 32,
    52, 85, 28, 17, 36, 83, 79, 18, 50, 17, 36, 36, 19]
x_df = pd.DataFrame({'data':x})

x_df.describe()
```

Out[]:

data	
count	30.000000
mean	42.866667
std	23.555852
min	13.000000
25%	25.250000
50%	36.000000
75%	52.750000
max	87.000000

In the above output, count is the number of data values, std is sample standard deviation, 25% is the 1st quartile or 25th percentile.

interpretation	pandas	value
n	count	30.000000
mean	mean	42.866667
standard deviation	std	23.555852
minimum	min	13.000000
25th percentile	25%	25.250000
50th percentile	50%	36.000000
75th percentile	75%	52.750000
maximum	max	87.000000

```
In [ ]: x_df.skew()
```

Out[]: data 0.694929
dtype: float64

Interpretation: The skewness index is 0.694929. If the value of skew is positive, the shape of the data is skewed to the right. Negative implies skewed to the left and zero indicates no skewness.

```
In [ ]: x_df.kurtosis()
```

Out[]: data -0.71888
dtype: float64

Interpretation: The kurtosis index is -0.71888. For kurtosis, the general guideline is that if the value is greater than 3, the distribution has heavier tails and if it is less than 3, it indicates a

distribution that has lighter tails as compared to the normal distribution.

14.2 Uploading Dataset

Uploading Excel or csv Files from Your Computer

[Watch demo video](#)

If your data file is in Excel spreadsheet or csv (comma separated value) format stored in your local computer, you can run the code-cell below to upload the file in to **Google Colab**.

```
from google.colab import files
uploaded = files.upload()
```

After running the code, click the **Choose Files** button and browse your computer's folder to locate your Excel or csv file. Open the file. Colab will upload it with the same file name, for example `file_name.xlsx`. Next, you will need to run the code-cell below to convert it to pandas dataframe.

```
import io
import pandas as pd
```

```
excel_file = io.BytesIO(uploaded['file_name.xlsx'])
x_df = pd.read_excel(excel_file)
```

Make sure the file names match. Now your dataframe, `x_df`, is ready for analysis and plotting.

For a csv file run the code-cell below

```
import io
import pandas as pd
```

```
csv_file = io.StringIO(uploaded['MyData.csv'].decode('utf-8'))
x_df = pd.read_csv(csv_file)
```

Uploading csv Dataset From URL

[Watch demo video](#)

If you want to upload a csv file from the internet with url address, use the code below. Replace the `web_link_address` with your url address first.

```
import pandas as pd
url = `www_link_address`
```

```
x_df = pd.read_csv(url)
```

Example: Write a program to read the dataset (URL below) and display the shape (size) of the dataset. In addition, display the first and last five rows of the dataset.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import pandas as pd

url = 'https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)
```

```
In [ ]: x_df.shape # displays the number of rows and columns
```

```
Out[ ]: (180, 8)
```

Interpretation: There are 180 rows and 8 columns in `x_df`.

```
In [ ]: x_df # displays the first 5 and the last 5 rows
```

```
Out[ ]:
```

	id	gender	section	time	level	test1_score	test2_score	test3_score
0	1	female	C	17.3	low	65.4	84.3	51.9
1	2	female	D	5.1	low	72.4	86.1	52.0
2	3	female	A	2.4	low	73.8	84.2	53.1
3	4	female	A	8.8	low	70.2	87.9	52.0
4	5	male	B	15.0	low	60.3	81.4	55.5
...	...	...	...	...	...	...	...	...
175	176	male	B	13.2	high	78.8	89.3	52.5
176	177	female	B	12.6	high	76.5	83.4	50.0
177	178	male	D	8.4	high	78.1	86.3	51.0
178	179	male	C	9.5	high	76.6	86.6	52.4
179	180	male	C	16.2	high	83.1	89.4	54.1

180 rows & 8 columns

Cleaning Data

Common cleaning data techniques include removing duplicate columns or observations (rows), combining or summarizing columns and replacing or deleting missing values.

1. To drop a column use

```
x_df.drop(['column_name'],axis=1,inplace=True)
```

2. To drop duplicate rows by keeping only the first observation, use

```
x_df.drop_duplicates(subset='column_name', keep="first", inplace=True)
```

3. To combine columns and make a new one, for example `average` use

```
df['average']=(x_df['test1']+x_df['test2']+x_df['test3])/3
```

4. To drop the rows where at least one element is missing, use

```
x_df.dropna()
```

5. To drop the columns where at least one element is missing, use

```
x_df.dropna(axis='columns')
```

6. `x_df.isnull().sum()` returns the number of missing values for each column

7. To replace missing (NaN) values with the mean, use

```
x_df['column_name'].fillna(value=df['column_name'].mean(), inplace=True)
```

Downloading Data as csv file to your computer

If you make changes to your `x_df` in Google colab and want to save or download your updated `x_df` to your local computer you can follow the steps below. For more on updating your dataframe, see the Cleaning Data section above.

first run the code-cell below

```
from google.colab import drive
```

then on a new code-cell run

```
drive.mount('/drive')
```

You will receive a message that says go to the url link to get authorization from your Google account. Click the link, then choose your Google Account (if you have more than one), then scroll down and click `allow`. Next you will need to copy the authorization code. Then go back to your Colab, paste the code where it says `Enter Authorization Code` and press enter.

You will see a new message: `Mounted at /drive`

Now you will need to choose a folder in your Google Drive where the file is to be downloaded or saved. For example:

```
x_df.to_csv('/drive/My Drive/Yiakl/my_csv_file.csv')
```

your dataframe name is `x_df` and your file will be downloaded to your Google Drive under the `Yiak1` folder. The name of the downloaded file is chosen here to be `my_csv_file.csv`.

14.3 Descriptive Statistics (multiple columns)

[Demo Vidoe Link](#)

Sample Dataset

We will use the students test scores dataset found at https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv. See below on how to upload it. This is "clean" dataset.

1. This is simulated data (not real)
2. 180 rows (observations) and 8 columns (variables)
3. `time` is completion time for `test1_score`
4. `id`, `gender`, `section` and `level` are categorical data
5. `time`, `test1_score`, `test2_score`, and `test3_score` are quantitative data

Basic Info about the Dataset

`x_df.info()` gives basic information about the dataset stored in `x_df`: column names, non-empty data values, and data types.

Example: Write a program to display basic information about the dataset (URL below).

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import pandas as pd

url = 'https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)

In [ ]: x_df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 180 entries, 0 to 179
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id               180 non-null    int64
1   gender           180 non-null    object
2   section          179 non-null    object
3   time             180 non-null    float64
4   level            180 non-null    object
5   test1_score      180 non-null    float64
6   test2_score      180 non-null    float64
7   test3_score      180 non-null    float64
dtypes: float64(4), int64(1), object(3)
memory usage: 11.4+ KB

```

Interpretation:

1. The first row tells us the object `x_df` is of class `DataFrame` from pandas
2. The second, `RangeIndex`, tells us there are 180 entries 0 to 179

Descriptive Stats: mean, std, quartiles, frequencies, correlations etc

Example: Write a program to read the dataset (URL below) and display the descriptive statistics - count, mean, std, 1st quartile, 2nd quartile, 3rd quartile and max of 'test1_score'.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```

In [ ]: import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)

```

```

In [ ]: x_df['test1_score'].describe()

```

```

Out[ ]: count    180.000000
mean      74.410000
std        5.890378
min       58.800000
25%       70.700000
50%       74.150000
75%       78.625000
max       90.300000
Name: test1_score, dtype: float64

```

Example: Write a program to read the dataset (URL below) and display the descriptive statistics - count, mean, std, 1st quartile, 2nd quartile, 3rd quartile and max of `test1_score` and `test2_score`.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)
```

```
In [ ]: x_df[['test1_score', 'test2_score']].describe()
```

```
Out[ ]:
```

	test1_score	test2_score
count	180.000000	180.000000
mean	74.410000	86.150000
std	5.890378	2.545332
min	58.800000	78.500000
25%	70.700000	84.450000
50%	74.150000	86.650000
75%	78.625000	88.000000
max	90.300000	90.700000

Descriptive Statistics: All quantitative variables

Example: Write a program to read the dataset (URL below) and display the descriptive statistics - count, mean, std, 1st quartile, 2nd quartile, 3rd quartile and max of all variables(columns).

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)
```

```
In [ ]: x_df.describe()    # summary for all quantitative columns
```

	id	time	test1_score	test2_score	test3_score
count	180.000000	180.000000	180.000000	180.000000	180.000000
mean	90.500000	10.321111	74.410000	86.150000	53.785000
std	52.105662	4.725398	5.890378	2.545332	2.693101
min	1.000000	-4.500000	58.800000	78.500000	49.900000
25%	45.750000	7.075000	70.700000	84.450000	51.700000
50%	90.500000	9.950000	74.150000	86.650000	53.200000
75%	135.250000	13.500000	78.625000	88.000000	55.350000
max	180.000000	22.500000	90.300000	90.700000	64.300000

The summary for the variable `id` is meaningless and should be ignored.

Grouped Frequencies

Example: Write a program to read the dataset (URL below) and display the frequency distribution by 'gender'. Change the frequencies to percent.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)

In [ ]: x_df.value_counts(subset=['gender']) # value counts by the variable(column) gender

Out[ ]: gender
male      98
female    82
dtype: int64

In [ ]: x_df.value_counts(subset=['gender'],normalize=True).mul(100).round(1).astype(str)+'%'

Out[ ]: gender
male      54.4%
female    45.6%
dtype: object
```

Summary Statistics Grouped by Gender

Example: Write a program to read the dataset (URL below) and display a table of `test1_score` mean, min, max, std, size grouped by `gender`.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import pandas as pd
```

```
url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'
```

```
x_df = pd.read_csv(url)
```

```
In [ ]: x_df.groupby(['gender']).agg({'test1_score': ['mean', 'min', 'max', 'std', 'size']})
```

```
Out[ ]:
```

	test1_score				
	mean	min	max	std	size
gender					
female	74.013415	58.8	87.6	6.234309	82
male	74.741837	60.3	90.3	5.597126	98

Example: Write a program to read the dataset (URL below) and display a table of `test1_score` mean, min, max, std, size grouped by `level` .

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import pandas as pd
```

```
url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'
```

```
x_df = pd.read_csv(url)
```

```
In [ ]: x_df.groupby(['level']).agg({'test1_score': ['mean', 'min', 'max', 'std', 'size']})
```

```
Out[ ]:
```

	test1_score				
	mean	min	max	std	size
level					
high	79.613333	70.9	90.3	3.617306	60
low	68.775000	58.8	76.1	4.350282	60
medium	74.841667	66.6	85.7	3.638881	60

Example: Write a program to read the dataset (URL below) and display a table of `test2_score` mean, min, max, std, size grouped by `section` .

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: x_df.groupby(['section']).agg({'test1_score': ['mean', 'min', 'max', 'std', 'size']})
```


Out[]:

	test1_score					
	mean	min	max	std	size	
section						
A	73.869565	58.8	87.6	6.367552	46	
B	73.695833	60.3	90.3	6.155398	48	
C	73.443243	61.7	84.2	6.343568	37	
D	76.241667	66.4	85.7	4.312961	48	

Grouped by Two Categorical Variables

Example: Write a program to read the dataset (URL below) and display a table of `test1_score` mean, min, max, std, size grouped by `level` and `gender` .

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

In []:

```
import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)
```

In []:

```
x_df.groupby(['level', 'gender']).agg({'test1_score': ['mean', 'min', 'max', 'std', 'size']})
```

Out[]:

		test1_score				
		mean	min	max	std	size
level	gender					
high	female	79.684000	70.9	87.6	3.637728	25
	male	79.562857	71.6	90.3	3.654979	35
low	female	68.469697	58.8	76.1	4.876377	33
	male	69.148148	60.3	76.1	3.662741	27
medium	female	75.729167	71.4	81.4	2.804884	24
	male	74.250000	66.6	85.7	4.031271	36

Grouped by Three Categorical Variables

Example: Write a program to read the dataset (URL below) and display a table of `test1_score` mean, min, max, std, size grouped by `level` , `gender` and `section` .

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import pandas as pd
```

```
url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'
```

```
x_df = pd.read_csv(url)
```

```
In [ ]: x_df.groupby(['level', 'section', 'gender']).agg({'test1_score': ['mean', 'min', 'max']})
```

Out[]:

			test1_score				
			mean	min	max	std	size
level	section	gender					
high	A	female	80.857143	75.7	87.6	4.595236	7
		male	78.571429	72.9	86.1	4.312661	7
	B	female	78.266667	74.0	81.6	2.838074	6
		male	80.171429	73.6	90.3	5.362125	7
	C	female	79.350000	70.9	84.2	4.749210	6
		male	79.477778	71.6	83.1	3.622422	9
	D	female	80.066667	78.0	82.3	1.709581	6
		male	79.709091	75.2	84.2	2.279673	11
low	A	female	67.788889	58.8	74.1	5.528889	9
		male	66.683333	62.1	69.4	3.491370	6
	B	female	66.233333	60.5	72.1	4.596375	6
		male	68.950000	60.3	74.9	4.021401	10
	C	female	66.900000	61.7	74.0	4.232021	9
		male	69.400000	65.6	71.2	1.925271	7
	D	female	72.211111	66.4	76.1	3.262072	9
		male	72.900000	69.9	76.1	2.970971	4
medium	A	female	76.360000	71.8	81.3	3.856553	5
		male	74.166667	70.8	78.0	2.133002	12
	B	female	76.022222	72.6	81.4	3.175601	9
		male	73.550000	66.6	81.8	4.818541	10
	C	female	75.650000	75.6	75.7	0.070711	2
		male	71.700000	69.7	73.3	1.489966	4
	D	female	75.025000	71.4	77.2	2.205027	8
		male	76.070000	69.5	85.7	5.153219	10

Correlation Coefficient (Pearson)

Example: Write a program to read the dataset (URL below) and display the correlation-coefficient between `time` and `test1_score`

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)
```

```
In [ ]: # correlation coefficient between time and test1_score
x_df['time'].corr(x_df['test1_score'])
```

Out[]: 0.040726134954403075

Example: Write a program to read the dataset (URL below) and use `x_df.corr()` to display the correlation-coefficient table between all columns.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)
```

```
In [ ]: x_df.corr()
```

Out[]:

	id	time	test1_score	test2_score	test3_score
id	1.000000	0.040144	0.738910	0.072236	0.024916
time	0.040144	1.000000	0.040726	0.061515	-0.080249
test1_score	0.738910	0.040726	1.000000	0.004367	0.030067
test2_score	0.072236	0.061515	0.004367	1.000000	0.096539
test3_score	0.024916	-0.080249	0.030067	0.096539	1.000000

Correlation Coefficient (Spearman)

Example: Write a program to read the dataset (URL below) and display the Spearman correlation-coefficient between `test1_score` and `test1_score`.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)
```

```
In [ ]: from scipy import stats
stats.spearmanr(x_df["test1_score"],x_df["test2_score"] ) # Spearman's rho - ranked correlation coefficient

Out[ ]: SpearmanrResult(correlation=0.008971805516759307, pvalue=0.9048529194438715)
```

14.4 Visualization

Example: The dataset below (URL) contains students test1_score, test2_score, test3_score, gender, level, section etc.

Write a program to display a frequency bar graph for gender.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

Bar graphs

```
In [ ]: import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

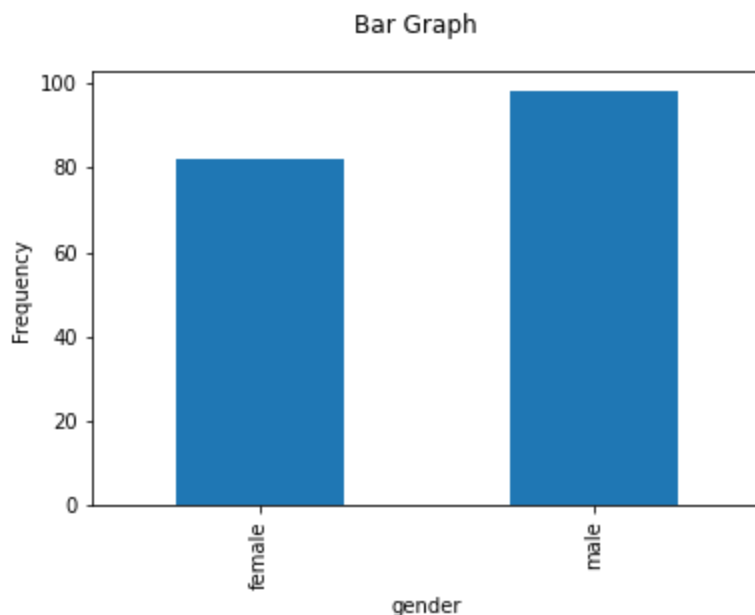
x_df = pd.read_csv(url)
```

```
In [ ]: x_df.groupby(['gender']).size().plot(kind='bar')

import matplotlib.pyplot as plt

plt.suptitle("Bar Graph")
plt.ylabel("Frequency")
```

```
Out[ ]: Text(0, 0.5, 'Frequency')
```



Example: The dataset below (URL) contains students test1_score, test2_score, test3_score, gender, level, section etc.

Write a program to display a frequency pie chart for section.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

Pie Chart

```
In [ ]: import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

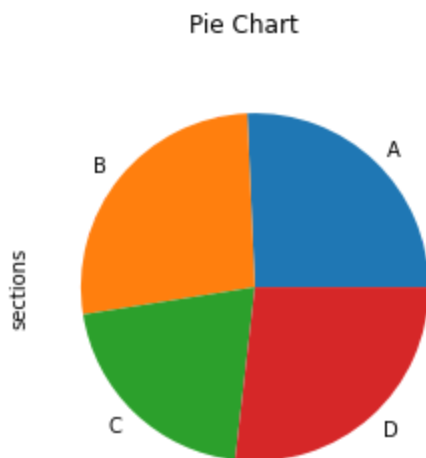
x_df = pd.read_csv(url)
```

```
In [ ]: x_df.groupby(['section']).size().plot(kind='pie')

import matplotlib.pyplot as plt

plt.suptitle("Pie Chart")
plt.ylabel("sections")
```

```
Out[ ]: Text(0, 0.5, 'sections')
```



Example: The dataset below (URL) contains students test1_score, test2_score, test3_score, gender, level, section etc.

Write a program to display a box plot for test1_socre.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

Box Plots

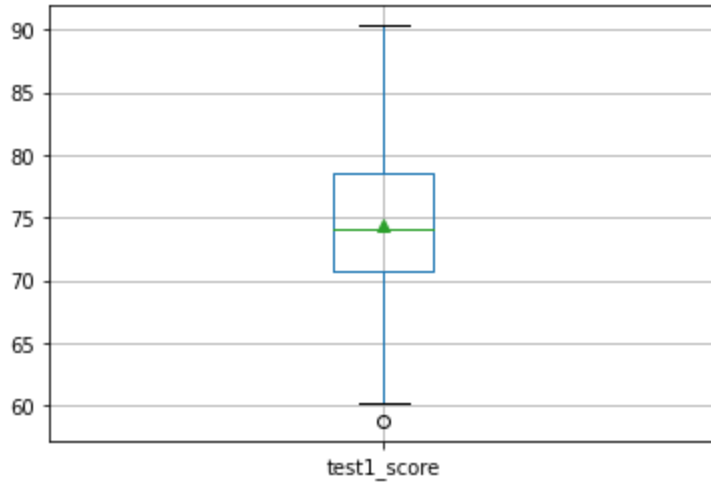
```
In [ ]: import pandas as pd
```

```
url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'
```

```
x_df = pd.read_csv(url)
```

```
In [ ]: x_df.boxplot(column=['test1_score'], showmeans = True)
```

```
Out[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7f9c424fd650>
```



Example: The dataset below (URL) contains students test1_score, test2_score, test3_score, gender, level, section etc.

Write a program to display box plot for test1_score grouped by level.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

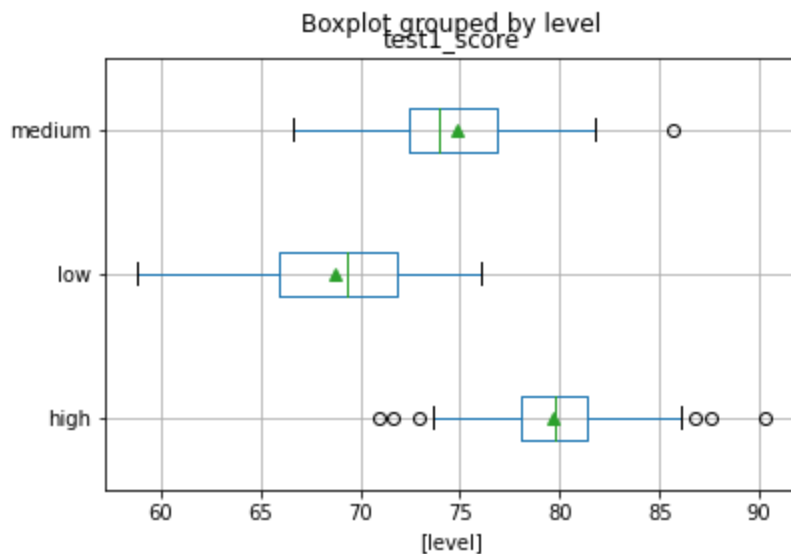
```
In [ ]: import pandas as pd
```

```
url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'
```

```
x_df = pd.read_csv(url)
```

```
In [ ]: x_df.boxplot(column=['test1_score'], by = ['level'], showmeans = True, vert = False)
```

```
Out[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7f9c329f73d0>
```



Example: The dataset below (URL) contains students test1_score, test2_score, test3_score, gender, level, section etc.

Write a program to display box plot for test1_score grouped by gender.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import pandas as pd
```

```
url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'
```

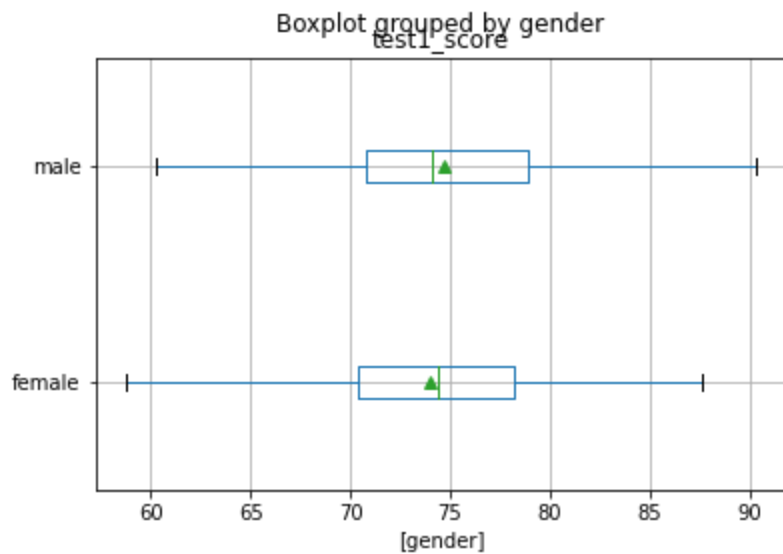
```
x_df = pd.read_csv(url)
```

```
In [ ]: x_df.boxplot(column=['test1_score'], by = ['gender'], showmeans = True, vert = False)
```

/usr/local/lib/python3.7/dist-packages/numpy/core/_asarray.py:83: VisibleDeprecationWarning:

Creating an ndarray from ragged nested sequences (which is a list-or-tuple of lists-or-tuples-or ndarrays with different lengths or shapes) is deprecated. If you meant to do this, you must specify 'dtype=object' when creating the ndarray

```
Out[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7f9c32a1de10>
```

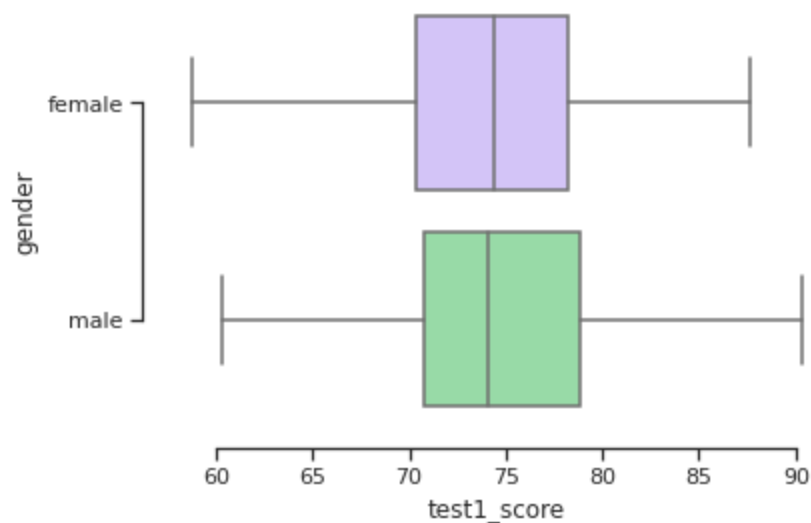
Alternative solution to the above example using the `seaborn` visualization library

```
In [ ]: import pandas as pd
import seaborn as sns
sns.set_theme(style="ticks", palette="pastel")

# Load the example tips dataset
url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)

sns.boxplot(x="test1_score", y="gender", hue_order="smoker", palette=["m", "g"], data=x_df)
sns.despine(offset=10, trim=True)
```



Example: The dataset below (URL) contains students test1_score, test2_score, test3_score, gender, level, section etc.

Write a program to display a histogram for test1_score with 6 bins(classes).

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

Histograms

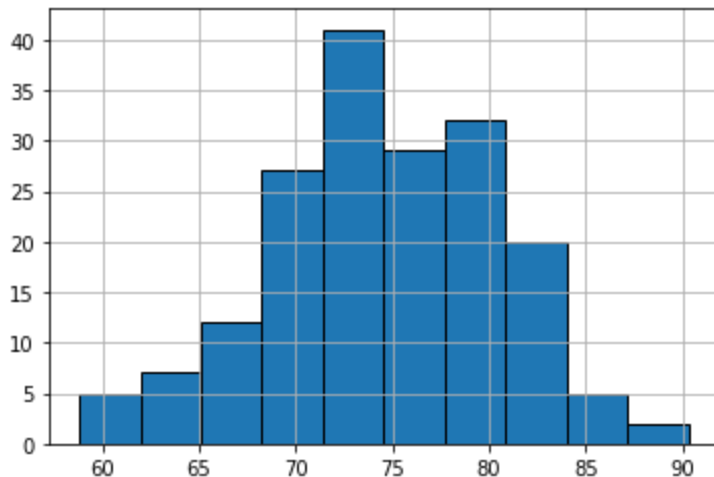
```
In [ ]: import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)
```

```
In [ ]: x_df.test1_score.hist(bins=10, ec='k')
```

```
Out[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7f8fcd56bf50>
```



Scatter plots

Example: The dataset in the website (URL below) includes 20 columns about home sale, including price, number of bedrooms and bathrooms, areas etc. The data includes 21,613 entries.

1. Write a program to read the dataset and display the basic information.
2. Write a program to display a scatter plot of sqft_living and price.
3. Write a program to display a scatter plot of bathrooms and bedrooms.

https://raw.githubusercontent.com/Shreyas3108/house-price-prediction/master/kc_house_data.csv

```
In [ ]: import pandas as pd

url = 'https://raw.githubusercontent.com/Shreyas3108/house-price-prediction/master/kc_house_data.csv'

x_df = pd.read_csv(url)

x_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21613 entries, 0 to 21612
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    21613 non-null  int64
1   date                 21613 non-null  object
2   price                21613 non-null  float64
3   bedrooms            21613 non-null  int64
4   bathrooms            21613 non-null  float64
5   sqft_living          21613 non-null  int64
6   sqft_lot             21613 non-null  int64
7   floors              21613 non-null  float64
8   waterfront          21613 non-null  int64
9   view                 21613 non-null  int64
10  condition            21613 non-null  int64
11  grade                21613 non-null  int64
12  sqft_above           21613 non-null  int64
13  sqft_basement        21613 non-null  int64
14  yr_built             21613 non-null  int64
15  yr_renovated         21613 non-null  int64
16  zipcode              21613 non-null  int64
17  lat                  21613 non-null  float64
18  long                 21613 non-null  float64
19  sqft_living15        21613 non-null  int64
20  sqft_lot15           21613 non-null  int64
dtypes: float64(5), int64(15), object(1)
memory usage: 3.5+ MB
```

Example: The dataset in the website (URL below) includes 20 columns about home sale, including `price` , number of `bedrooms` and `bathrooms` , areas etc. The data includes 21,613 entries.

Write a program to display a scatter plot of `sqft_living` and `price` .

https://raw.githubusercontent.com/Shreyas3108/house-price-prediction/master/kc_house_data.csv

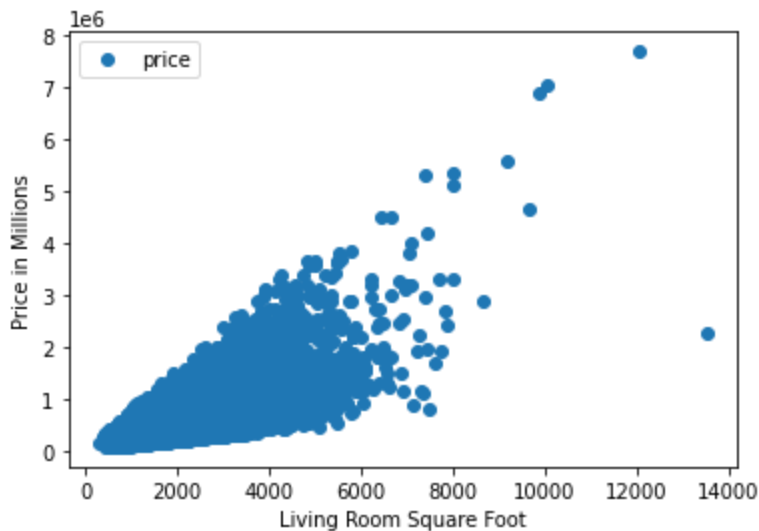
```
In [ ]: import pandas as pd

url = 'https://raw.githubusercontent.com/Shreyas3108/house-price-prediction/master/kc_house_data.csv'

x_df = pd.read_csv(url)
```

```
In [ ]: x_df.plot('sqft_living','price',style='o', xlabel='Living Room Square Foot', ylabel='Price')
```

Out[]: <matplotlib.axes._subplots.AxesSubplot at 0x7f8fcd489c90>



Example: The dataset in the website (URL below) includes 20 columns about home sale, including `price`, number of `bedrooms` and `bathrooms`, areas etc. The data includes 21,613 entries.

Write a program to display a scatter plot of `bathrooms` and `bedrooms`

https://raw.githubusercontent.com/Shreyas3108/house-price-prediction/master/kc_house_data.csv

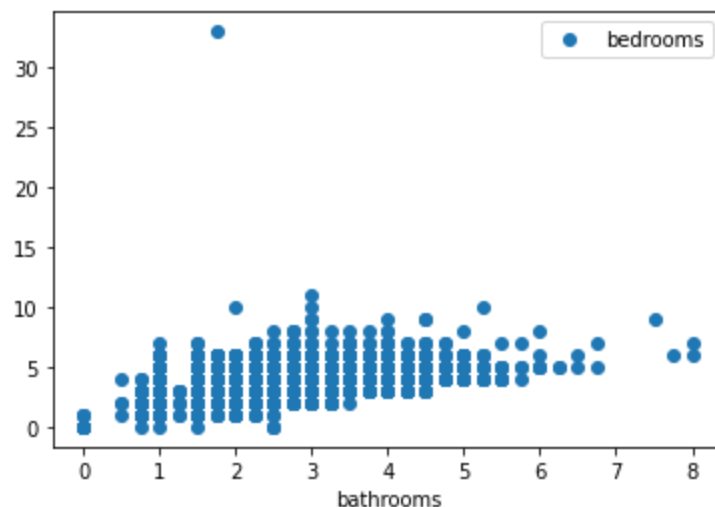
```
In [ ]: import pandas as pd

url = 'https://raw.githubusercontent.com/Shreyas3108/house-price-prediction/master/kc_house_data.csv'

x_df = pd.read_csv(url)
```

```
In [ ]: x_df.plot('bathrooms', 'bedrooms', style='o')
```

Out[]: <matplotlib.axes._subplots.AxesSubplot at 0x7f9c323b21d0>



14.5 Regression

Multiple Linear Regression

[Watch demo video](#)

The **General Linear Model** $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k + \epsilon$ for multiple regression analysis where y is the response variable we want to predict, $\beta_0, \beta_1, \dots, \beta_k$ are unknown constants, $x_1, x_2, \dots, x_k$ are independent predictor variables that are measured without error, and ϵ is the random error.

$\hat{y} = b_0 + b_1 x_1 + b_2 x_2 + \dots + b_k x_k$ can be determined using the method of Least Squares as follows.

```
import statsmodels.formula.api as smf
model = smf.ols(formula='y ~ x1 + x2 +...+ xk', data=x_df)
results = model.fit()

print(results.summary())
```

Example: The dataset in the website (URL below) includes 20 columns about home sale, including price, number of bedrooms and bathrooms, areas etc. The data includes 21,613 entries.

Write a program to display basic information and the first five rows. `x_df.info()` and `x_df.head()`.

https://raw.githubusercontent.com/Shreyas3108/house-price-prediction/master/kc_house_data.csv

```
In [ ]: import pandas as pd

url = 'https://raw.githubusercontent.com/Shreyas3108/house-price-prediction/master/kc_house_data.csv'
x_df = pd.read_csv(url)
```

Interpretation: The above code uses pandas to read and save the dataset as pandas dataframe.

```
In [ ]: x_df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21613 entries, 0 to 21612
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                     21613 non-null  int64
1   date                   21613 non-null  object
2   price                  21613 non-null  float64
3   bedrooms               21613 non-null  int64
4   bathrooms              21613 non-null  float64
5   sqft_living            21613 non-null  int64
6   sqft_lot               21613 non-null  int64
7   floors                 21613 non-null  float64
8   waterfront             21613 non-null  int64
9   view                   21613 non-null  int64
10  condition              21613 non-null  int64
11  grade                  21613 non-null  int64
12  sqft_above             21613 non-null  int64
13  sqft_basement          21613 non-null  int64
14  yr_built               21613 non-null  int64
15  yr_renovated           21613 non-null  int64
16  zipcode                21613 non-null  int64
17  lat                   21613 non-null  float64
18  long                  21613 non-null  float64
19  sqft_living15          21613 non-null  int64
20  sqft_lot15             21613 non-null  int64
dtypes: float64(5), int64(15), object(1)
memory usage: 3.5+ MB

```

Interpretation: There are 20 columns and 21,613 entries.

```
In [ ]: x_df.head()
```

```

Out[ ]:

```

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floo
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	1
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	1
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	1

Example: The dataset in the website (URL below) includes 20 columns about home sale, including price, number of bedrooms and bathrooms, areas etc. The data includes 21,613 entries.

1. Write a program to find the least-squares prediction equation,

$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \epsilon$ for these data where, y is **price**, x_1 is **bedrooms**, x_2 **bathrooms** and x_3 is number of **floors**.

2. Use the overall F test to determine whether the model contributes significant information to the prediction of y with 1% level of significance.
3. What is the value of R-squared. That is, the coefficient of determination.
4. Does the number of `floors`, x_3 , contribute significant information for the prediction of the `price`, y , given that x_1 and x_2 are already in the model? (use $\alpha = 0.05$)

https://raw.githubusercontent.com/Shreyas3108/house-price-prediction/master/kc_house_data.csv

```
In [ ]: import statsmodels.formula.api as smf
model = smf.ols(formula='price ~ bedrooms+bathrooms+floors', data=x_df)
results = model.fit()

print(results.summary())
```

```

OLS Regression Results
=====
Dep. Variable:          price      R-squared:                0.278
Model:                  OLS       Adj. R-squared:           0.278
Method:                 Least Squares   F-statistic:            2769.
Date:                  Tue, 07 Sep 2021   Prob (F-statistic):      0.00
Time:                  21:32:40    Log-Likelihood:         -3.0409e+05
No. Observations:      21613      AIC:                   6.082e+05
Df Residuals:          21609      BIC:                   6.082e+05
Df Model:               3
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-2.91e+04	9209.213	-3.160	0.002	-4.72e+04	-1.11e+04
bedrooms	2.002e+04	2680.838	7.469	0.000	1.48e+04	2.53e+04
bathrooms	2.385e+05	3681.888	64.766	0.000	2.31e+05	2.46e+05
floors	-1737.4719	4569.452	-0.380	0.704	-1.07e+04	7218.991

```

=====
Omnibus:                17359.610    Durbin-Watson:           1.961
Prob(Omnibus):           0.000      Jarque-Bera (JB):        905901.534
Skew:                   3.476      Prob(JB):                0.00
Kurtosis:               33.946      Cond. No.                20.4
=====

```

Warnings:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Interpretation:

1. See `x_df.info()` and `x_df.head()` above
2. $\hat{y} = b_0 + b_1x_1 + b_2x_2 + b_3x_3$ where $b_0 = -2.91e + 04$,
 $b_1 = 2.002e + 04$, $b_2 = 2.385e + 05$, $b_3 = -1737.4719$

3. The F-statistic is 2769 with p-value of Prob (F-statistic) 0.00. Thus, the model has a significant contribution to the prediction of price.
4. The coefficient of determination (R-squared:) is 0.278.
5. The p-value ($P > |t|$) for the floors is 0.704 which is not less than $\alpha = 0.05$. It is not significant.

Logistic Regression

Logistic regression is the type of regression analysis used to find the probability of a certain event occurring. Best suited for cases where we have a categorical dichotomous dependent variable.

$p = \frac{1}{1 + e^{\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k}}$ where p is probability that $y = 1$ and y is the response variable with values 1s and 0s.

Example: The data set below (URL) contains the occurrence of coronary heart disease (`chd`), tobacco, obesity, age, alcohol, etc. For `chd`, 0 means patient has no `chd` and 1 means patient has `chd`.

1. Write a program to read the data set and display the first five rows.
2. Write a program to fit a logistic regression model where the dependent variable is `chd` (coronary heart disease) and the predictor variables are `tobacco` , `obesity` and `age` .
3. Use the LLR p-value test to determine whether the model contributes significant information to the prediction of y , `chd` , with 1% level of significance.

<https://raw.githubusercontent.com/simeonaman/demo/master/heart.csv> this dataset came from <https://web.stanford.edu/~hastie/ElemStatLearn//datasets/SAheart.data>

```
In [ ]: import pandas as pd
x_df = pd.read_csv('https://raw.githubusercontent.com/simeonaman/demo/master/heart.c
x_df.head()
```

```
Out[ ]:
```

	row.names	sbp	tobacco	ldl	adiposity	famhist	typea	obesity	alcohol	age	chd
0	1	160	12.00	5.73	23.11	Present	49	25.30	97.20	52	1
1	2	144	0.01	4.41	28.61	Absent	55	28.87	2.06	63	1
2	3	118	0.08	3.48	32.28	Present	52	29.14	3.81	46	0
3	4	170	7.50	6.41	38.03	Present	51	31.99	24.26	58	1
4	5	134	13.60	3.50	27.78	Present	60	25.99	57.34	49	1


```
In [ ]: x_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 462 entries, 0 to 461
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   row.names    462 non-null    int64
1   sbp          462 non-null    int64
2   tobacco      462 non-null    float64
3   ldl          462 non-null    float64
4   adiposity    462 non-null    float64
5   famhist      462 non-null    object
6   typea        462 non-null    int64
7   obesity      462 non-null    float64
8   alcohol      462 non-null    float64
9   age          462 non-null    int64
10  chd          462 non-null    int64
dtypes: float64(5), int64(5), object(1)
memory usage: 39.8+ KB
```

Example: The data set below (URL) contains the occurrence of coronary heart disease (`chd`), tobacco, obesity, age, alcohol, etc. For `chd`, 0 means patient has no `chd` and 1 means patient has `chd`.

1. Write a program to fit a logistic regression model where the dependent variable is `chd` (coronary heart disease) and the predictor variables are `tobacco` , `obesity` and `age` .
2. Use the LLR p-value test to determine whether the model contributes significant information to the prediction of y , `chd` , with 1% level of significance.

<https://raw.githubusercontent.com/simeonaman/demo/master/heart.csv> this dataset came from <https://web.stanford.edu/~hastie/ElemStatLearn//datasets/SAheart.data>

```
In [ ]: import pandas as pd
x_df = pd.read_csv('https://raw.githubusercontent.com/simeonaman/demo/master/heart.c
```

```
In [ ]: import statsmodels.formula.api as smf
model = smf.logit(formula='chd ~ tobacco + obesity + age', data=x_df)
results = model.fit()

print(results.summary())
```

Optimization terminated successfully.
 Current function value: 0.557771
 Iterations 6

Logit Regression Results						
=====						
Dep. Variable:	chd		No. Observations:	462		
Model:	Logit		Df Residuals:	458		
Method:	MLE		Df Model:	3		
Date:	Tue, 07 Sep 2021		Pseudo R-squ.:	0.1354		
Time:	21:32:40		Log-Likelihood:	-257.69		
converged:	True		LL-Null:	-298.05		
Covariance Type:	nonrobust		LLR p-value:	2.142e-17		
=====						
	coef	std err	z	P> z	[0.025	0.975]

Intercept	-3.4213	0.753	-4.541	0.000	-4.898	-1.945
tobacco	0.0788	0.026	3.061	0.002	0.028	0.129
obesity	0.0019	0.026	0.074	0.941	-0.050	0.054
age	0.0538	0.009	5.808	0.000	0.036	0.072
=====						

Interpretation:

$$1. \hat{p} = \frac{1}{1 + e^{-3.4213 + 0.0788x_1 + 0.0019x_2 + 0.0538x_3}}$$

2. LLR p-value 2.142e-17 is less than $\alpha = 0.01$. Thus, the model has a significant contribution to the prediction of price.

14.6 ANOVA (Analysis of Variance)

[Watch demo video](#)

One-Way ANOVA

ANOVA (analysis of variance) can be used to test whether the means of more than two groups are statistically different from each other.

Example: The data set below (URL) contains the students test1_score, gender, level, section etc.

1. Write a program to read the data set and display the first and last five rows.
2. Write a program to find `test1_score` means, std and size for each level

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'
```

```
x_df = pd.read_csv(url)

x_df # print the first and last five rows
```

```
Out[ ]:
```

	id	gender	section	time	level	test1_score	test2_score	test3_score
0	1	female	C	17.3	low	65.4	84.3	51.9
1	2	female	D	5.1	low	72.4	86.1	52.0
2	3	female	A	2.4	low	73.8	84.2	53.1
3	4	female	A	8.8	low	70.2	87.9	52.0
4	5	male	B	15.0	low	60.3	81.4	55.5
...	...	...	...	...	...	...	...	...
175	176	male	B	13.2	high	78.8	89.3	52.5
176	177	female	B	12.6	high	76.5	83.4	50.0
177	178	male	D	8.4	high	78.1	86.3	51.0
178	179	male	C	9.5	high	76.6	86.6	52.4
179	180	male	C	16.2	high	83.1	89.4	54.1

180 rows Ãƒâ€” 8 columns

```
In [ ]: x_df.groupby(['level']).agg({'test1_score': ['mean', 'std', 'size']})
```

```
Out[ ]:
```

	test1_score		
	mean	std	size
level			
high	79.613333	3.617306	60
low	68.775000	4.350282	60
medium	74.841667	3.638881	60

Example: The data set below (URL) contains the students test1_score, gender, level, section etc.

- Write a program to perform ANOVA to test if there is sufficient evidence to indicate a difference in the averages of test1_scores among the three levels (low, medium, high). Use 10% level of significance.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import statsmodels.api as sm
from statsmodels.formula.api import ols
model = ols('test1_score ~ C(level)', data = x_df).fit()
```

```
table = sm.stats.anova_lm(model,type=2) # type 2 for equal or non-equal sample sizes
print(table)
```

	df	sum_sq	mean_sq	F	PR(>F)
C(level)	2.0	3540.854333	1770.427167	117.372972	3.555808e-33
Residual	177.0	2669.827667	15.083772	NaN	NaN

Interpretation: Since the p-value $3.555808e-33 < \alpha = 0.10$, reject the null hypothesis that the averages for all three levels are equal. That is, there is a significant difference between the averages of test1_score of the three levels. The sum of squares residual is 2669.827667. The sum of squares treatment (level) is 3540.854333.

Tukey's Paired Mean Test

Example: For the above example, use Tukey's method for paired comparisons to determine which pairs of the three level means are different.

```
In [ ]: import statsmodels.stats.multicomp as mc
comp = mc.MultiComparison(x_df['test1_score'], x_df['level'])
post_hoc_res = comp.tukeyhsd(alpha=0.05)
print(post_hoc_res.summary())
```

```
Multiple Comparison of Means - Tukey HSD, FWER=0.05
=====
group1 group2 meandiff p-adj lower upper reject
-----
high low -10.8383 0.001 -12.5144 -9.1622 True
high medium -4.7717 0.001 -6.4478 -3.0956 True
low medium 6.0667 0.001 4.3906 7.7428 True
-----
```

Interpretation: All pairwise mean comparisons are significant. That is, the mean score1_difference between high and low, high and medium, and low and medium are significantly different.

Two-way ANOVA (Block Design)

Example: The data set below (URL) contains students test1_score, gender, level, section etc.

1. Write a program to read the dataset and display the first rows.
2. Write a program to find `test1_score` means grouped by `level` and `gender`.
3. Write a program to perform ANOVA block design to test if there is sufficient evidence to indicate a difference in the averages of test1_scores among the three levels (low, medium, high) using gender as blocks. Use 10% level of significance.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)

x_df.head() # print the first five rows
```

```
Out[ ]:
```

	id	gender	section	time	level	test1_score	test2_score	test3_score
0	1	female	C	17.3	low	65.4	84.3	51.9
1	2	female	D	5.1	low	72.4	86.1	52.0
2	3	female	A	2.4	low	73.8	84.2	53.1
3	4	female	A	8.8	low	70.2	87.9	52.0
4	5	male	B	15.0	low	60.3	81.4	55.5

```
In [ ]: x_df.groupby(['level', 'gender']).agg({'test1_score': ['mean', 'std', 'size']})
```

```
Out[ ]:
```

		test1_score		
		mean	std	size
level	gender			
high	female	79.684000	3.637728	25
	male	79.562857	3.654979	35
low	female	68.469697	4.876377	33
	male	69.148148	3.662741	27
medium	female	75.729167	2.804884	24
	male	74.250000	4.031271	36

```
In [ ]: import statsmodels.api as sm
from statsmodels.formula.api import ols
model = ols('test1_score ~ C(level) + C(gender)', data = x_df).fit()
table = sm.stats.anova_lm(model, type=2) # type 2 for non-equal sample sizes
print(table)
```

	df	sum_sq	mean_sq	F	PR(>F)
C(level)	2.0	3540.854333	1770.427167	116.878417	5.044391e-33
C(gender)	1.0	3.850572	3.850572	0.254203	6.147613e-01
Residual	176.0	2665.977095	15.147597	NaN	NaN

Interpretation: There is significant difference of the test1_score means between the three levels after blocking by gender. No significant difference in the average test1_scores between the two genders.

Two-Factor ANOVA (Interactions)

Example: The dataset below (URL) contains students `test1_score`, `gender`, `level`, `section` etc.

1. Write a program to read the dataset and display the first 5 rows.
2. Write a program to display the means grouped by `level` and `interaction`
3. Write a program to perform a two-factor ANOVA with `level` and `section`. Do the data provide sufficient evidence to indicate an interaction between `level` and `section`? Use 10% level of significance.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

```
In [ ]: import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)
```

```
In [ ]: x_df.groupby(['level','section']).agg({'test1_score': ['mean', 'std', 'size']})
```

```
Out[ ]:
```

		test1_score		
		mean	std	size
level	section			
high	A	79.714286	4.442601	14
	B	79.292308	4.325402	13
	C	79.426667	3.944339	15
	D	79.835294	2.047542	17
low	A	67.346667	4.704840	15
	B	67.931250	4.311646	16
	C	67.993750	3.560238	16
	D	72.423077	3.067614	13
medium	A	74.811765	2.812002	17
	B	74.721053	4.207081	19
	C	73.016667	2.343857	6
	D	75.605556	4.043073	18

```
In [ ]: import statsmodels.api as sm
from statsmodels.formula.api import ols
```

```
model = ols('test1_score ~ C(level)*C(section)',data = x_df).fit()
table = sm.stats.anova_lm(model,type=2) # type 2 for non-equal sample sizes
print(table)
```

	df	sum_sq	mean_sq	F	PR(>F)
C(level)	2.0	3494.967554	1747.483777	121.179409	3.062871e-33
C(section)	3.0	125.235378	41.745126	2.894819	3.687668e-02
C(level):C(section)	6.0	133.100312	22.183385	1.538309	1.685520e-01
Residual	167.0	2408.245695	14.420633	NaN	NaN

Interpretation: There is no significant interaction effect between `level` and `section` of the average `test1_score`, because the p-value 1.685520e-01 is not less than the level of significance .10.

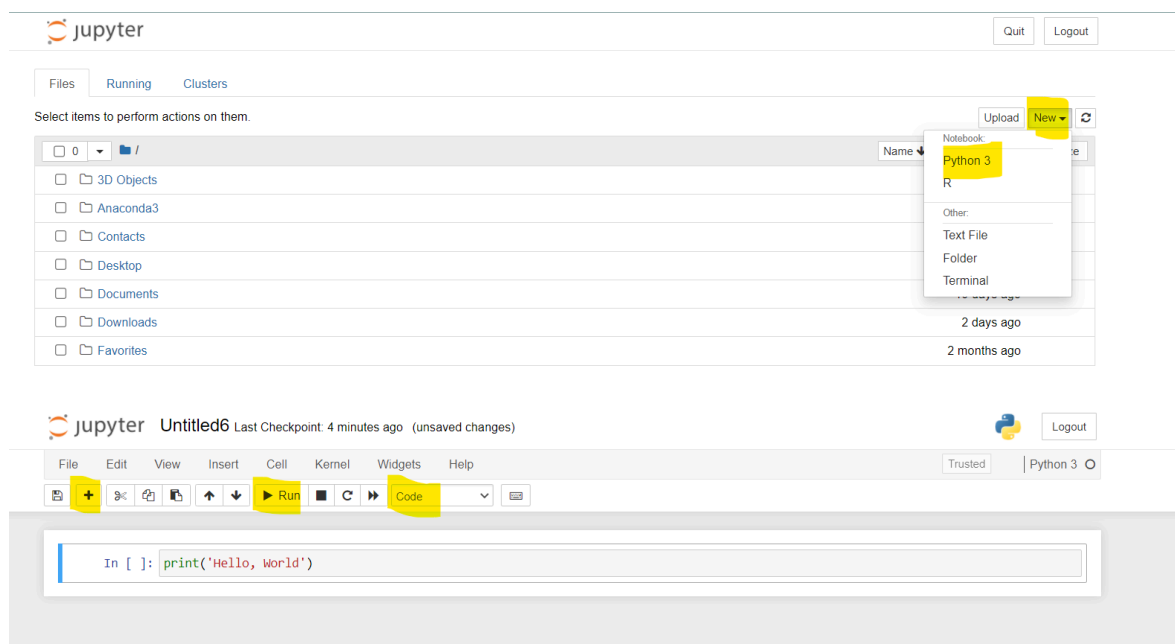
The End

Appendix 1: Python Installing Steps

Python Computer Installation

If you prefer not to use Google Colab Notebook, you can install free Python in your computer and use the **Anaconda Jupyter Notebook**. Go to [this website](#) and select Windows, MacOS or Linux, depending on your computer. When the installation is completed, open **jupyter Notebook(anaconda3)** from the **Anaconda** folder on your computer. You can also watch YouTube videos on how to install Anaconda for [Windows 10](#) or [MacOS](#).

Open **Jupyter Notebook** from your Windows computer Start menu under **Anaconda** and click **new**. You can add new code-cell or text-cell by clicking + and code (see pictures below). You can run a code-cell by clicking **run**.



Appendix 2: Random Integer Generator

[Watch demo video](#)

"A random number generator, generates a sequence of numbers or symbols that cannot be reasonably predicted better than by a random chance." Wikipedia

To randomly generate n integers between a and b , inclusive, use the following function from the [numpy library](#). Here n , a and b are integers.


```
import numpy as np
np.random.randint(a, b+1, size=n)
```

The above code returns an array of n random integers from a to $b + 1$, inclusive. Notice that every time you run the program, it will generate different sets of numbers. To create the same set of random numbers you can add the line `np.random.seed(123)`.

Example: Write a program to randomly select one integer between 1 and 10.

1. Click `+ Code`
2. Type `import numpy as np`
3. Type `np.random.randint(1,10+1,size=1)`
4. Click `play` button

```
In [ ]: import numpy as np
np.random.randint(1,10+1,size=1)
```

```
Out[ ]: array([5])
```

Interpretation: The number 5 is randomly selected from all integers between 1 and 10. If you run this program again, it will, most likely, generate a different result.

Example: Write a program to randomly select 3 integers between 1 and 32.

```
In [ ]: import numpy as np
np.random.randint(1,32+1,size=3)
```

```
Out[ ]: array([22, 13,  5])
```

Interpretation: 22, 13, and 5 are randomly selected from all integers between 1 and 32.

Example: Write a program to randomly select 10 integers between 1 and 100. Save the result in a variable named `x` and print `x`.

```
In [ ]: import numpy as np
x = np.random.randint(1,100+1,size=10)
x
```

```
Out[ ]: array([59, 65, 14, 79, 42, 44,  4, 30, 38, 40])
```

Interpretation: The 10 numbers randomly selected from all integers between 1 and 100 are 59, 65, 14, 79, 42, 44, 4, 30, 38, 40.

To get the same result in every run, add `np.random.seed(n)` where `n` is any non-negative integer.

Example: Write a program to randomly select 3 integers between 1 and 32. Use the seed number 123.

```
In [ ]: import numpy as np
        np.random.seed(123) # fix seed to get the same result
        np.random.randint(1,32+1,size=3)
```

```
Out[ ]: array([31, 14, 31])
```

Interpretation: The 3 numbers randomly selected with the given seed are 31, 14 and 31.

```
In [ ]: import numpy as np
        np.random.seed(123) # fix seed to get the same result
        np.random.randint(1,32+1,size=3)
```

```
Out[ ]: array([31, 14, 31])
```

Interpretation: By adding `np.random.seed(123)`, we are able to generate the same set of numbers all the time.

Appendix 3: Counting, Permutations and Combinations

In probability experiment, counting the number of outcomes in a trial or the total number of all possible outcomes is essential.

Factorial, Permutation, Combination

[Watch demo video](#)

The number of ways we can arrange n distinct objects is given by:

$$n! = n \times (n - 1) \times (n - 2) \dots 2 \times 1$$

The number of ways we can arrange 5 distinct objects is:

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

Syntax for factorial:

```
import numpy as np
np.math.factorial(n)
```

The number of ways we can arrange n distinct objects, taking them r at a time is given by:

$$nPr = \frac{n!}{(n-r)!}$$

```
import numpy as np
np.math.factorial(n)/np.math.factorial(n-r)
```

The number of distinct combinations of n distinct objects that can be formed by taking r at a time is given by:

$${}^nC_r = \frac{n!}{r!(n-r)!}$$

```
import numpy as np
np.math.factorial(n)/(np.math.factorial(r)*np.math.factorial(n-r))
```

Example: For $n = 6$ and $r = 3$, compute the following:

1. $n!$
2. nP_r
3. nC_r

```
In [ ]: import numpy as np
np.math.factorial(6)
```

Out[]: 720

Interpretation: $6!=720$. There 720 different arrangement of 6 distinct objects.

```
In [ ]: import numpy as np
np.math.factorial(6)/np.math.factorial(6-3)
```

Out[]: 120.0

Interpretation: there are 120 permutations of 6 objects taking 3 at a time.

```
In [ ]: import numpy as np
np.math.factorial(6)/(np.math.factorial(3)*np.math.factorial(6-3))
```

Out[]: 20.0

Interpretation: There are 20 combinations (without regard to order) of 6 objects taking 3 at a time.

Code References

Arithmetic Operators

Name	Operator	Example	Result
Addition	+	5+6	11

Name	Operator	Example	Result
Subtraction	-	5-6	-1
Multiplication	*	5*6	30
Division	/	5/6	0.83333
Exponentiation	**	5**2	5 ² = 25

Descriptive Stats

```
import numpy as np
import pandas as pd
```

```
x=[x1,x2,x3,...]
```

```
x_df=pd.DataFrame({'column_name':x})
```

procedure	numpy	pandas
frequency counts	<code>np.unique(x,return_counts=True)</code>	<code>x_df.value_counts()</code>
size	<code>np.size(x)</code>	<code>x_df.count()</code>
min	<code>np.amin(x)</code>	<code>x_df.min()</code>
max	<code>np.amax(x)</code>	<code>x_df.max()</code>
mean	<code>np.mean(x)</code>	<code>x_df.mean()</code>
mode	NA	<code>x_df.mode()</code>
median	<code>np.median(x)</code>	<code>x_df.median()</code>
sample variance	<code>np.var(x,ddf=1)</code>	<code>x_df.var()</code>
pop variance	<code>np.var(x)</code>	<code>x_df.var(ddof=0)</code>
sample std	<code>np.std(x,ddof=1)</code>	<code>x_df.std()</code>
pop std	<code>np.std(x)</code>	<code>x_df.std(ddof=0)</code>
weighted mean	<code>np.average(x,weights=freq)</code>	NA
weighted sample std	<code>(np.cov(x,fweights=freq,ddof=1))**(1/2)</code>	NA
weighted pop std	<code>(np.cov(x,fweights=freq))**(1/2)</code>	NA
percentiles	<code>np.quantile(x,p)</code>	<code>x_df.quantile(p)</code>
summary stats	NA	<code>x_df.describe()</code>
factorial, n!	<code>np.math.factorial(n)</code>	NA

procedure	numpy	pandas
n random integers from a to b	<code>np.random.randint(a,b+1,size=n)</code>	NA
weighted mean	<code>np.average(x,weights=y)</code>	NA
Weighted std	<code>(np.cov(x,aweights=y,ddof=0)**(1/2)</code>	NA
correlation coef.	<code>np.corrcoef(x,y)</code>	<code>x_df.corr()</code>

NA = Not Available

Summary statistics by grouping with categorical columns:

```
x_df.groupby(['level','gender']).agg({'test1_score':
['mean','std','size']})
```

Graphs

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
x = [x1, x2, x3, ...]
x_df = pd.DataFrame({'column_name',x})
```

CHOOSE A CODE FROM BELOW (Bar, Pie, Histogram etc)

```
plt.suptitle("Plot Title")
plt.xlabel("x-axis label")
plt.ylabel("y-axis label")
```

1. Bar graph (raw data) `x_df.groupby(['column_name']).size().plot(kind='bar')`

2. Pie Chart (raw data) `x_df.groupby(['column_name']).size().plot(kind='pie', autopct='%1.1f%%')`

3. Bar graph (frequency table) `x_df.plot.bar(x='column_name',y='frequency')`

4. Pie chart (frequency table)

```
x_df.set_index('column_name',inplace=True)
x_df.plot.pie(y='frequency')
```

5. Side by Side Bar (frequency table)

```
x_df.set_index('column_name',inplace=True)
x_df.plot.bar()
```

6. Histogram

```
freq,bins,patches = plt.hist(x,bins=5,ec='k',range=
[ min(x),max(x)+1])
plt.xticks(bins) # df['column_name']=x
```

7. Scater Plot (x, y list given) `plt.scatter(x,y)`

Linear Regression

```
from scipy import stats
stats.linregress(x_list, y_list)
```

Multiple Linear Regression

```
import statsmodels.formula.api as smf
model = smf.ols(formula='y ~ x1 + x2 +...+ xk', data=x_df)
results = model.fit()

print(results.summary())
```

Probability Distributions

```
from scipy import stats
```

procedure	scipy.stats
Binomial pdf $P(X = x)$	<code>stats.binom.pmf(x,n,p)</code>
Binomial cdf $P(X \leq x)$	<code>stats.binom.cdf(x,n,p)</code>
Normal cdf $P(X \leq x)$	<code>stats.norm.cdf(x,mean,std)</code>
Inverse Normal or p-th Percentile	<code>stats.norm.ppf(p,mean,std)</code>
t distribution cdf $P(X \leq x)$	<code>stats.t.cdf(t,df,0,1)</code>
Inverse t or p-th Percentile	<code>stats.t.ppf(p,df,0,1)</code>
chi-square cdf $P(\chi^2 \leq ch2)$	<code>stats.chi2.cdf(ch2,k-1)</code>
Inverse chi-square or p-th Percentile	<code>stats.chi2.ppf(p,k-1)</code>

Margin of Error for Confidence Interval

```
from scipy import stats
```

Mean μ with z distribution with confidence level C , (population) standard deviation std and sample size n :

$$E = (std/n^{.5}) * stats.norm.ppf(C+(1-C)/2, 0, 1)$$

Mean μ with t distribution with confidence level C , sample standard deviation std and sample size n :

$$E = (std/n^{.5}) * stats.t.ppf(C+(1-C)/2, 0, 1)$$

Proportion p with z distribution with confidence level C , sample proportion p and sample size n :

$$E = (p(p*(1-p)/n)^{.5}) * stats.norm.ppf(C+(1-C)/2, 0, 1)$$

Resources

- [Python.org](https://www.python.org/)
- [Matplotlib](https://matplotlib.org/)
- [Python eBook](#)
- [Introductory Statistics eBook](#)
- [Python Tutorials](#)
- [Terminology](#)
- [Numpy](#)
- [Scipy Stats -Pandas](#)